

S2 Appendix. Protocol of the controlled studies

Contents

1	Contents of this appendix	3
2	Pre-registration	3
	Pre-registration — controlled evaluation of executable specifications and a reviewer-derived audit checklist for agent-run interpretability	3
3	Amendments	6
	Amendment 1 to the pre-registration (written before Studies A and B were run)	6
	Amendment 2 (exploratory arm; written after the Study A T2 results were seen)	6
	Amendment 3 (exploratory Haiku arm re-run with one folder per agent; written before the re-run)	7
	Note on Amendment 3 (written 2026-10-03, after the re-run)	7
4	Freeze log	8
	Freeze log	8
5	The generic checklist	10
	Methodological audit checklist (ten recurring reviewer objections)	10
6	The checklist as deployed	15
	Audit pipeline — recurring reviewer-flagged issues	15
7	Agent prompts	27
	7.1 Study A (tools/studyA_workflow.js)	27
	7.2 Study B (tools/studyB_workflow.js)	30
	7.3 Study C (tools/studyC_workflow.js)	31
	7.4 Exploratory arm (tools/studyA_haiku_isolated_workflow.js)	34
8	Study A task briefs	35
	8.1 Task T1	35
	8.2 Task T2	37
	8.3 Task T3	39
	8.4 Task T4	41
9	Study A answer keys	43
	9.1 Task T1	43
	9.2 Task T2	46
	9.3 Task T3	50
	9.4 Task T4	54
10	Study B items	57
	10.1 Pair P02 (flawed B9296, clean B3100)	58
	10.2 Pair P03 (flawed B3954, clean B8827)	59
	10.3 Pair P04 (flawed B7148, clean B7392)	60
	10.4 Pair P05 (flawed B2977, clean B4481)	62
	10.5 Pair P06 (flawed B1144, clean B2101)	63
	10.6 Pair P07 (flawed B8612, clean B6874)	65
	10.7 Pair P08 (flawed B9709, clean B8045)	66
	10.8 Pair P09 (flawed B8833, clean B2418)	68

10.9	Pair P10 (flawed B1249, clean B4174)	69
10.10	Pair P11 (flawed B4668, clean B8104)	71
10.11	Pair P12 (flawed B7037, clean B4328)	73
11	Study C reference set	75
12	Exploratory arm with Claude Haiku 4.5	76

1 Contents of this appendix

This appendix gives the protocol of the three controlled studies of LLM agents (Studies A, B and C). Section 2 is the pre-registration, and Section 3 gives its three amendments. Section 4 is the freeze log: the time and the sha256 hash of each file that was frozen before use. Section 5 is the generic ten-pattern checklist, and Section 6 is the checklist as it was deployed. Section 7 gives the prompt templates sent to the agents and the fields of their structured output. Section 8 gives the Study A task briefs, and Section 9 the Study A answer keys. Section 10 lists the Study B items and their keys. Section 11 lists the 68 known errors that form the Study C reference set. Section 12 covers the exploratory arm with Claude Haiku 4.5. Sections 2 to 6 and the briefs in Section 8 are copied word for word; only their heading levels were changed. Sections 9 to 11 lay out the key files without changing their text. All files are also in the code and data release, in the folder `controlled-evaluation/`. Absolute local paths are shown with the placeholders used in the release: `<EVAL_ROOT>` is the evaluation workspace and `<CODE_ROOT>` the folder that held it (`PATH_MAP.md` in the release lists all placeholders). The freeze-log hashes are those of the original files.

2 Pre-registration

The pre-registration is copied word for word below. Its sha256 hash is in the freeze log (Section 4).

Pre-registration — controlled evaluation of executable specifications and a reviewer-derived audit checklist for agent-run interpretability

Frozen before any subject agent is run. Task-specific answer keys are frozen separately (one file per task or item); their sha256 hashes are written to `protocol/FREEZE_LOG.md` before the corresponding study is run.

1. Questions

- **Q1 (Study A).** For the same agent model, tools, data, task statement and source-method material, does adding the markdown pipeline specification ("contract") make the agent's final scientific conclusion more often correct?
- **Q2 (Study A).** Applied to the same executor outputs, does a review using the reviewer-derived ten-pattern checklist, followed by repair, produce correct conclusions more often than (a) no review and (b) a generic critical review of the same budget, followed by the same repair step?
- **Q3 (Study B).** On analyses that were not used to build the checklist, how often does a checklist review find a real error, how often does it raise a false alarm on a correct analysis, and how does it compare with a generic review of the same budget?
- **Q4 (Study C).** On a blind copy of the MaxToki deployment as it stood before its audit, which of the errors later documented in the error ledger are found by (a) the checklist as deployed (with project-specific examples), (b) the generic checklist, (c) a generic review?

2. Subjects and context control (all studies)

- Every subject agent (executor, reviewer, repairer, grader) is Claude Opus 5.5 (`claude-opus-5-5`), run as a Claude Code workflow subagent of the built-in read-only type with the model set explicitly to Opus. This agent type starts without project instruction files and without a memory index (verified by a probe on 2026-10-01). It can read files and run shell commands, but it does not write files: analyses run as inline Python, and all deliverables are returned as structured output. The harness writes them to disk verbatim from the workflow journal.
- Python interpreter: `maxtoki-framework-eval/bin/python` (numpy 1.26.4, pandas 2.3.3, scikit-learn 1.8.0, scipy, torch 2.11.0 — CPU only for subjects).
- Each subject is told to read only inside the folders its prompt names. After the runs, every transcript is scanned for file reads outside those folders. A run that read the repository,

another task, another item, an answer key or the paper is excluded and repeated once; the number of exclusions is reported.

- Reasoning effort is not overridden; it is the session default for all arms.

3. Study A — 2×3 factorial on four analysis tasks

Arms

- Executor factor:
 - paper: task brief + data + source-method paper (PDF) of the method's original study.
 - contract: the same, plus the deployed pipeline specification for that method (the version dated 2026-04-15, as used in the deployment). The brief, data, deliverable format, compute limits and instructions are word-for-word identical across the two arms except for one sentence pointing to the specification.
- Review factor (applied to every executor output):
 - none: the executor output is final.
 - generic: a fresh reviewer reads the task package and the executor's deliverable and reviews it critically for errors and weaknesses; a fresh repairer then revises the deliverable given the review.
 - checklist: the same, except the reviewer applies the generic ten-pattern checklist (protocol/CHECKLIST_GENERIC.md). Reviewer prompts carry the same budget sentence and the same output schema. The repair prompt is identical across the two review arms.
- Replicates: 5 independent executor runs per task $\times$ executor arm. Total executor runs: 4 tasks $\times$ 2 arms $\times$ 5 = 40. Each yields three final deliverables (none / generic / checklist): 120 graded deliverables.

Tasks (data come from the MaxToki-217M deployment, after correction)

- T1: Do attention-derived scores recover curated regulator-target pairs (TRRUST) in the 217M RPE1 run?
- T2: Do SAE-circuit-derived predictions get the direction of change after a CRISPRi knockdown right (K562)?
- T3: Are SAE features that respond to a transcription factor's knockdown specific to that factor's target genes (K562)?
- T4: Does a developmental (hematopoietic stage) ordering read out from the model's internal state transfer to donors not used to fit it?

Outcomes

- **Primary:** verdict correct (1/0) against the frozen answer key.
- Secondary:
 1. trap score: fraction of the task's pre-listed traps handled correctly;
 2. key numbers within the key's tolerance (deterministic, from the returned results object where the key defines a matching quantity);
 3. number of materially false statements in the report;
 4. reviews: number of findings; findings judged valid / invalid (false alarm) against the key;
 5. repair: wrong→right and right→wrong verdict changes;
 6. reproducibility: the returned analysis script is run by the harness on the task data; numbers in results are reproduced within tolerance (1/0);
 7. cost: tokens, tool calls, wall time per agent from transcripts;
 8. human interventions: none by design (recorded as 0).

Grading

- Two independent graders per deliverable (same agent type and model), each given the task brief, the answer key and the deliverable; neither is told the arm. Before grading, the harness redacts arm-revealing words from the deliverable (case-insensitive: specification, spec, contract, checklist, audit, reviewer, review, repair, revised, pattern P1-P10 labels). Graders score verdict, traps and false statements with written justification. Disagreement on the primary outcome → a third grader decides. Agreement is reported (Cohen's kappa on the primary outcome; ICC or weighted kappa on the trap score).

Analysis

- Primary contrasts (Holm-corrected across the four):
 1. contract vs paper at review = none (unpaired; Fisher exact test; risk difference with Newcombe 95% CI; also a task-stratified Cochran-Mantel-Haenszel test);
 2. checklist vs generic (paired on the same executor output; exact McNemar);
 3. checklist vs none (paired; exact McNemar);
 4. generic vs none (paired; exact McNemar).
- Secondary outcomes: descriptive with 95% CIs (Wilson for proportions; bootstrap over executor runs, stratified by task, for means).
- All results are reported, whatever their direction. No arm, task or outcome is dropped after seeing results.
- Power note: with 20 runs per executor arm the study can detect only large differences in the primary outcome. This is stated with the results.

4. Study B — held-out audit benchmark

Items

- Flawed items: analyses from other projects in the author's repository (single-cell foundation models other than MaxToki, protein language models, text-LLM-based cell models, GRN benchmarks) whose error was found and fixed after the fact, rebuilt as self-contained packages (data, code, outputs and the results summary as originally written), with hint comments removed. Plus a small number of planted causal-overreach items (pattern P9 has no natural example in the pool). Each item has exactly one documented error.
- Clean items: the corrected versions of the same analyses where available, plus other analyses checked to be correct.
- Every item is verified by code before freezing: a flawed item reproduces its documented wrong result; a clean item reproduces the correct one.
- Items are labelled by whether the documented error falls inside the checklist's ten patterns or outside them (e.g. pure code bugs).

Arms

- checklist (generic checklist) vs generic review; same budget sentence, same schema; 3 independent reviews per item per arm.

Outcomes

- Flawed items: detected (1/0): a finding that names the documented error's nature and location well enough that a reader would fix it.
- All items: false alarms: findings that assert a load-bearing error that the key marks as not an error. Disputed findings go to a verifier agent that checks them by running code on the package; its verdict is final.
- Per arm: sensitivity, false-alarm rate per clean item, precision of load-bearing findings; split by in-checklist vs outside-checklist errors.

Analysis

- Item-level means over replicates; bootstrap over items for CIs; paired comparison of arms by item (Wilcoxon signed-rank on item-level detection; also a mixed logistic model with item random effect if it converges).

5. Study C — blind retrospective re-audit of the deployment

- Snapshot: the MaxToki project as it stood before the audit started (2026-05-07 16:57), built from file timestamps; audit files, paper folders and every later file excluded; summaries edited later reconstructed by stripping the dated audit additions. Manifest kept outside the snapshot.
- Ground truth: rows of the error ledger with `layer = run_outputs` and `present_in_pre_audit_state = yes` (frozen before the run).

- Arms (3 independent runs each): `deployed-checklist` (the audit specification exactly as used in the deployment, including its project-specific examples), `checklist` (generic), generic review. Same budget sentence and schema.
- Outcomes: per ledger item, detection by each run (graded by two graders with the ledger); critical-item detection; unmatched findings classified by a verifier as (a) a real error missing from the ledger, (b) false alarm, (c) cannot be decided.
- Comparison: per-arm item-level detection with bootstrap CIs over items, and the original audit's detection on the same items (from the ledger).

6. Things not done (stated in advance)

- No human baseline (no human analysts or reviewers).
- No other model family as executor or reviewer.
- No measurement of human supervision time (the deployment did not log it).

3 Amendments

The pre-registration has three amendments. They are given in order, word for word. Amendments 2 and 3 concern the exploratory arm with Claude Haiku 4.5 (Section 12).

Amendment 1 to the pre-registration (written before Studies A and B were run)

1. **Study A, task T4 replaced.** The developmental-manifold transfer task was replaced by a cross-model gene-embedding alignment task: "Is MaxToki-217M's gene-embedding geometry aligned with scGPT's, and how does that alignment compare with its alignment to Geneformer V2-316M?" Reason: an unambiguous answer key for the manifold task could not be written, because the manifold gates are also passed by a pure cell-type-label lookup (found in Study C and confirmed on the full repository), so the correct verdict depends on analyses that were still running. The new T4 has a clear key and contains a real deployed error (wrong embedding rows for scGPT).
2. **Study A runs in two batches** with the identical protocol: batch 1 = T1, T3, T4; batch 2 = T2 (its data exist only after the circuit-tracing re-run with corrected hooks). Results are pooled.
3. **Package staging.** Subject packages are copied to a neutral folder (`<CODE_ROOT>/analysis-tasks/<opaque id>/`), so folder names do not reveal task, arm or study. The Python wrapper there caps BLAS threads at 2 and suppresses Python warnings (which printed a local path). Package content is byte-identical to the frozen build except the interpreter path in BRIEF.md.
4. **Known property of the contract arm.** The deployed specifications are given verbatim, as the treatment. They carry expectations from earlier studies (for example, the attention specification states that a failing verdict is the expected outcome). This is part of what a contract is in this framework and is reported with the results.
5. **Study B, pair P01 excluded** (packages B1131, B1448): independent verification found that the documented error was not the real defect of the flawed version, so the item failed the pre-registered verification and is not frozen. Study B has 11 pairs (22 items): 8 with natural errors and 3 with planted causal-overreach summaries.
6. **Budget sentences.** Study A reviewers: "Spend about the effort of a careful two-hour expert review; you may use up to roughly 60 tool calls." Study B reviewers: "Spend about the effort of a careful one-hour expert review; you may use up to roughly 50 tool calls." Identical across arms within a study.

Amendment 2 (exploratory arm; written after the Study A T2 results were seen)

After batch 2 of Study A (task T2) finished, every deliverable in every cell had the correct verdict (30/30), so with Claude Opus 5.5 the task showed a ceiling. To learn whether the specification or the reviews help a less capable agent, we add an exploratory replication of

Study A with **Claude Haiku 4.5** as executor, reviewer and repairer, on all four tasks, with the identical packages, prompts, budget sentences, five replicates per cell, and the same Opus 5.5 graders and answer keys. It is reported separately from the pre-registered Study A and labelled exploratory; its results do not enter the pre-registered contrasts. It was added after one task's results were known, which is stated in the paper.

Amendment 3 (exploratory Haiku arm re-run with one folder per agent; written before the re-run)

Written 2026-10-02, before any agent of the re-run started. No result of the re-run had been seen.

Why

A scan of the first exploratory Haiku 4.5 run (Amendment 2) found that its agents did not keep their work separate:

- 48 Haiku subject agents wrote temporary files to the shared /tmp folder under common names (for example /tmp/analysis_script.py), so concurrent agents could read or overwrite each other's scripts.
- Two Haiku executors wrote analysis_script.py into the shared task folders of T1 (contract) and T2 (contract), and Haiku reviewers of other runs then read and ran it. A resumed Haiku agent later wrote verify_analysis.py into the T1 (contract) folder.

The prompts asked the agents not to write files and to read only inside the task folder; the Haiku agents did not follow this. The first Haiku run is therefore set aside. Its outputs are kept unchanged on disk but are not analysed.

The same scan of the pre-registered Opus 5.5 studies (A, B and C) found no agent that read a file written by another agent, with one exception: two Study A repair agents (T1, contract, run 2 after the checklist review and run 5 after the generic review) printed the first 30 to 50 lines of the stray Haiku script in the T1 (contract) folder, which they took for part of the package. Graders of T1 (contract) deliverables also opened it. This is reported in the paper. The stray files were moved to quarantine/2026-10-02_stray_files_in_task_folders/, and all Study A and Study B task folders were checked to be identical to their sources (apart from the interpreter path in the Study A briefs, which was set when the folders were built).

What changes in the re-run

1. **One folder per agent.** Every agent (executor, reviewer, repairer and grader) gets its own folder with an opaque name under analysis-tasks/. The folder mirrors its task package: real sub-folders, and one symbolic link per package file. A file an agent writes therefore stays in its own folder. The package files are checked against their source copies after the run.
2. **Prompt wording.** In place of "do not write files" and "read only inside <package>", every prompt says that the folder is the agent's own, that it may write files only inside it, and that it must not read or write anywhere else (naming /tmp and scratch folders). Everything else in the prompts is unchanged.
3. **Labels.** Agents are labelled AI: instead of AH:.

Unchanged: subject model (Claude Haiku 4.5), tasks, packages, answer keys, review and repair instructions, budget sentences, five replicates per cell, Opus 5.5 graders (two per deliverable, a third on disagreement), review-point grading, and the analysis code. The arm stays exploratory and is reported separately from the pre-registered Study A.

A dated note, written after the re-run and also hashed in the freeze log, corrects one count in Amendment 3. It is copied word for word below.

Note on Amendment 3 (written 2026-10-03, after the re-run)

Amendment 3 says that two Study A repair agents (T1, contract arm, run 2 after the checklist review and run 5 after the generic review) printed the first lines of the stray script w6252/

analysis_script.py. A complete scan of all Study A transcripts made afterwards (tools/package_write_check.py, and a direct search of the ten T1 contract-arm repair transcripts) found that all ten repair agents of T1 in the contract arm (runs 1-5, after both review styles) opened that script; most printed its first 30 to 50 lines. No executor or reviewer of Study A opened it, and no agent of Studies B or C. All ten deliverables reached the correct verdict, and leaving out the T1 contract-arm runs does not change any Study A result (for example, checklist minus no review in pitfalls handled: +0.029, 95% bootstrap interval 0.007 to 0.051, over 35 instead of 40 executor runs). The paper reports the corrected count.

4 Freeze log

The freeze log lists the UTC time and the sha256 hash of each frozen file. It is copied word for word below.

Freeze log

Each line: UTC time, sha256, file. Written before the study that uses the file is run.

- 2026-10-01T04:58:21Z
175d922edef4cf424744e91d51eeb1a69ae840d53f0ab7a635179e24dc3409ad
PREREGISTRATION.md
- 2026-10-01T04:58:21Z
17d858b2541650b6b89ba9475d8570e382708925e6fc17e6bec0931738de2b56 CHECKLIST_
GENERIC.md
- 2026-10-01T04:59:27Z
adab48b845d888844b00f4d5565b7650cea3dc2a03bf059048d9780fe905162f STUDYC_KEY.
csv
- 2026-10-01T04:59:27Z
dc2740a0fef6feb3f6e95d06a79f54a60fa53e397855852a21b01523f7cf80cb CHECKLIST_
DEPLOYED.md
- 2026-10-01T10:56:43Z
e3f266d4f038639466752b0465775b55607df44460f0d18a64b590412dad7139 AMENDMENT_
1.md
- 2026-10-01T10:56:43Z
da10d2d21d04498685c3880a77356e10d750526833b18b96f75cc46eb0c2b28f STUDYA_
PACKAGE_MAP.json
- 2026-10-01T10:56:43Z
045de13ff4eb0520aeb128eb078f9fec3223c42d3e3df90dc863218fe4c8aaa6 STUDYB_
ITEMS.json
- 2026-10-01T10:56:43Z
9edc7ed1a0a69c5336438f5aee4f519dbe99771340f1452692a90139fa3402b9 ../studyA/
keys/T1/key.json
- 2026-10-01T10:56:43Z
290d211c28bf1afa490d06e32d6312c3b7c487da2ca895e9da868a47dc51d29a ../studyA/
keys/T3/key.json
- 2026-10-01T10:56:43Z
e69a98329f762e18d69b763cc7a648d8d9a80382246aea4276734cc657e71c92 ../studyA/
keys/T4/key.json
- 2026-10-01T10:56:43Z
3595a1fbc40c8e930e4d0a7816889a630bb6b1c31f55a949e58c8b91293c41cc ../studyB/
keys/B1144.json
- 2026-10-01T10:56:43Z
0cdd1d8328f3fa0af27e366184336536dacbff4ccccf95a13470b67432ed5f49 ../studyB/
keys/B1249.json
- 2026-10-01T10:56:43Z
c615f53568a9008beeba4ded64d0f53850d19d88f667eff0007fe201eb6c6df9 ../studyB/
keys/B2101.json
- 2026-10-01T10:56:43Z
252fdf53145781169d483eac4cd21fdadb1cfd21c9149df7e740ea7c5724530 ../studyB/
keys/B2418.json

- 2026-10-01T10:56:43Z
59285c465e5d9ae4bcd0864ff7ce371fc3d46a790543293a8d2e347657c69edc ../studyB/keys/B2977.json
- 2026-10-01T10:56:43Z
3bb431b6b40e995473fd242436fd1ca63a2bbd3bc5ea66ccaa23e4b815d36dab ../studyB/keys/B3100.json
- 2026-10-01T10:56:44Z
92b6e2a387916d203480a3a616810e4754a6e2943630f606d9ed7dc0fa067663 ../studyB/keys/B3954.json
- 2026-10-01T10:56:44Z
68375f6a8ee475d0d514869ac63b78dba52155d5a3b73d90ec42e0871aab51d3 ../studyB/keys/B4174.json
- 2026-10-01T10:56:44Z
b77b4bcb6abb82fd22554d24992b85822294d86655ffd4af9d85a3b55fb793aa ../studyB/keys/B4328.json
- 2026-10-01T10:56:44Z
9f172812f76f97e9a6036572201f3b2198a0b964b7301298713b2e5df5669eb0 ../studyB/keys/B4481.json
- 2026-10-01T10:56:44Z
1888312924985d260029907d662a21d9e355e0f911f64c7e1c1a6876f284b759 ../studyB/keys/B4668.json
- 2026-10-01T10:56:44Z
4b237b8e9067dfebc2fb030babf88be596708437ea39dc6449db7bb83c9fcda5 ../studyB/keys/B6874.json
- 2026-10-01T10:56:44Z
d1004a2aff828d348f12aedb97b0819aabe394a3c74282aab2b317906c35d5a0 ../studyB/keys/B7037.json
- 2026-10-01T10:56:44Z
5a12a8a4c7946110a9b83c8bdc7da016c26d6abac13bb8eb321f15deb50e13ca ../studyB/keys/B7148.json
- 2026-10-01T10:56:44Z
15c70964872f40b13dee887423ad4101a92708092151578c53bd826f95ad9300 ../studyB/keys/B7392.json
- 2026-10-01T10:56:44Z
1a1ae8cc0d4205904b122066e3118061f8c64c08f90d829a779abef84628fd87 ../studyB/keys/B8045.json
- 2026-10-01T10:56:44Z
ba1e241c7080c21595bbee4034a55964c7af180d9d1a293b99775d75c852195f ../studyB/keys/B8104.json
- 2026-10-01T10:56:44Z
12f3fc22ded6a56376740827124e75afedb16510557152c0d8c4f77f33e2873f ../studyB/keys/B8612.json
- 2026-10-01T10:56:44Z
9bfa93f02c5595f1f0f7a0218f4fba39fc88a6e16f46717a5f405787563b8312 ../studyB/keys/B8827.json
- 2026-10-01T10:56:44Z
03db5dde99ecdcfbe5acb6fff8d568c1193c20f1d154c0740feca7ceb5905f4 ../studyB/keys/B8833.json
- 2026-10-01T10:56:44Z
b70be069b55393f9f997a21740e740a0c21f4e4c7adc08c2472525bffe017ff7 ../studyB/keys/B9296.json
- 2026-10-01T10:56:44Z
9b016da2aad1c89d067335d80468df2d2d1a2bdd3fff77aec8766a5101fa2cfb1 ../studyB/keys/B9709.json
- 2026-10-01T11:50:29Z
711d2be0cfc38b445d0d351564d988c6da2104129f2bd07852f8862e24784f1a ../studyA/keys/T2/key.json (batch 2)
- 2026-10-01T11:50:29Z
8d67ae1fb08e072797e71bae98db7372e19ef0cafedbba5fe2823e8901c34b7a STUDYA_PACKAGE_MAP.json (batch 2)
- 2026-10-01T16:12:32Z

- 2bd24b2acd6748c291eb645c367a62059addcf004537a993fd29c3f067ed3d24 AMENDMENT_2.md
- 2026-10-02T15:26:46Z
375e485d9c3c6859b67a412d24ad059050cdd3591029557f407cd221d012e368 AMENDMENT_3.md
- 2026-10-03T08:50:41Z
18407d880e8352e1b6110f39325a20709e346156e60a2a7fdc445092240b9e9 AMENDMENT_3_NOTE.md (note, not an amendment)

When this appendix was built, the build script recomputed the sha256 hash of each of the 36 files in the log. All 36 match their last entry in the log. STUDYA_PACKAGE_MAP.json has two entries; the file on disk matches the second (batch 2) entry.

5 The generic checklist

This is the generic ten-pattern checklist. It was the checklist used by the checklist review arm in Studies A and B and by the checklist arm of Study C. It is copied word for word below.

Methodological audit checklist (ten recurring reviewer objections)

Apply this checklist to the analysis you are given. For each of the ten patterns below, decide whether the analysis shows the pattern, using the diagnostic signature, and say what the prescription requires. Go from each load-bearing claim to its numerical evidence to the code that produced it, in that order. Record a verdict for every pattern (clean / present but acknowledged / present and unaddressed), with evidence.

Wherever the text below mentions FINAL_SUMMARY.md, README.md, a pipeline section number (§) or a project folder, read it as "the analysis's results summary", "its method description" or "its files", whatever they are called in the material you are given.

P1. Null model is too weak / not anchored to chance

Reviewer signature. *"No baseline given for what 0.53 shared ontology means — anchor it to a chance baseline." "The expression-matched null may be insufficient — control for variance and dropout, not just mean." "How much of the effect survives a stricter null?"*

Why it recurs. Single-cell foundation-model artefacts are dense, so a naive shuffle (gene-label permutation, feature-shuffle) produces a null much weaker than the structure that actually drives the observed result. Reviewers want to see (i) every quantitative claim *anchored* to a chance value, and (ii) a hierarchy of nulls that progressively rules out confounders.

Diagnostic signature.

- A claim of the form " $X = 0.NN$ " without an adjacent "vs chance baseline $0.MM \pm SD$ ".
- A null implemented only as `np.random.permutation(labels)` or `np.random.shuffle(features)` with no degree preservation, no composition matching, and no coexpression matching.
- A "p-value" without a stated null family.
- A " $\Delta AUROC$ " reported only against the trivial 0.5 baseline rather than against a coexpression / matched-degree null.

Prescription.

- For every numeric claim in FINAL_SUMMARY.md that is meant to be meaningful (i.e., load-bearing), attach a "(null mean $\pm$ SD; p95)" suffix. If the suffix would be misleading because the relevant null isn't computed, compute it (cheap; reuse phase-12-style strict-max-null harness).
- Adopt the strict max-null framing as the default for any claim that appears in an abstract or executive summary. Per-domain breakdowns at the loose null are fine in supplementary detail.
- Where a single null family was used, name it inline ("...vs feature-shuffle null; the same comparison against degree-preserving rewiring is in §X").

P2. Trivial / gene-level baseline absent

Reviewer signature. *"What does a simpler model already give you?" "Does the foundation-model embedding add **incremental value** over PCA on gene expression / marginal correlation / gene-level mean expression?"*

Why it recurs. Foundation-model claims are most credible when stated as "this beats the simple thing." Without a trivial baseline, the reader cannot distinguish "model encodes X" from "X is recoverable from raw expression statistics."

Diagnostic signature.

- Headline AUROC reported in absolute terms with no comparison to PCA, raw-correlation, or gene-mean baselines.
- "Cross-model alignment $r=0.78$ " reported without checking whether r between the two models' *static input embeddings* (which are trained independently but on similar token sets) already produces $r \approx 0.78$ — i.e., the alignment is a property of the input layer, not learned dynamics.
- Discussion section claims biological understanding without naming a simpler model that would or would not produce the same number.

Prescription.

- Add to the validation step (§9 of any pipeline) an explicit "trivial baseline" line: PCA-on-expression, marginal correlation, or gene-mean — whichever is the natural simple comparator for that endpoint.
 - Convert any absolute-AUROC claim to a " Δ AUROC over trivial baseline" claim in FINAL_SUMMARY.md. If the delta is zero or negative, that *is* the finding and should be stated explicitly.
 - For cross-model claims, always also report the static-input-only alignment as a floor.
-

P3. Endpoint-object mismatch

Reviewer signature. *"Perturbation DE measures direct + indirect; you can't infer direct regulation from this." "AUROC vs TRRUST is an edge-recovery task; perturbation-target prediction is an effect-magnitude task; they are not the same thing." "Embedding shift is a model-internal quantity; what makes you think it tracks biological importance?" "TRRUST is curated literature; ChIP-seq in the matched cell type is closer to truth."*

Why it recurs. Mech-interp work routinely conflates (a) what the metric arithmetically measures, (b) what the model is internally doing, and (c) the biological claim made in the abstract. Reviewers home in on this gap.

Diagnostic signature.

- An abstract sentence "the model encodes regulatory relationships" supported only by an AUROC against a curated database.
- "Direct target" used loosely without a ChIP-seq peak or a perturbation + time-resolved direct-effect filter.
- Any claim that "the model causes X" supported by a correlational embedding-shift analysis.
- Reuse of an annotation database for *both* feature selection and evaluation (annotation-database circularity).

Prescription.

- In every summary of results, restate what the endpoint *literally* measures before the biological claim. Use an explicit two-objective framing: "Objective A (mechanistic / edge-recovery against curated database)" vs "Objective B (perturbation-response prediction)" — and state which objective each claim speaks to.
 - Tag annotation databases used as ground truth with the cell-type / context in which they were curated. Note explicitly when that context does not match the analysis context.
 - Replace "encodes" / "causes" with "is consistent with" / "co-varies with" unless an intervention-based test was actually done.
-

P4. Pseudoreplication / wrong unit-of-inference / leaky CV

Reviewer signature. *"You have 500 cells but only 3 donors — your inference unit is donors, not cells, and your power is ~27%." "Edge-level 5-fold CV leaks gene-level features; use GroupKFold by TF." "Cell-type labels are imputed from markers; treat as exploratory, not confirmatory."*

Why it recurs. Single-cell data is hierarchical (cell $\subset$ donor $\subset$ cohort, edge $\subset$ TF, etc.) and the natural unit at which the *biological* claim lives is rarely the same as the unit at which sample size N is large. CV / nulls that ignore this hierarchy systematically inflate significance.

Diagnostic signature.

- A p-value computed over cells when the claim is about donors / cohorts / cell-types.
- Reported $n=500$ cells with no $n=\text{donors}$, $n=\text{cohorts}$, or $n=\text{independent-conditions}$ companion.
- StratifiedKFold rather than GroupKFold when features are shared across rows (edges sharing a TF; gene pairs sharing an endpoint).
- A bootstrap implemented at the row level when the row-level units are not exchangeable.
- Cell-type-stratified AUROC reported without acknowledging that cell-type labels are noisy.

Prescription.

- Whenever a numeric claim is reported, name the inference unit explicitly: " $n=NN$ edges over MM TFs over KK cells" with the unit at which the null / CV / bootstrap is grouped.
 - For shared-endpoint metrics, run GroupKFold (by TF, by target, by both) and report the worst of the three. If the conclusion is robust, fine; if not, restate.
 - For cell-type stratification, report a sensitivity analysis where labels are alternatively assigned via reference-mapping (Azimuth / SingleR).
-

P5. Selection-driven inflation / double-dipping / confound carry-through

Reviewer signature. *"You picked the 120 best-annotated features and report properties of those — that's a biased subset." "You ranked genes on the same data you tested them on." "Effect strength is correlated with expression level — control for it." "Tokenization window excluded the genes you couldn't test."*

Why it recurs. Selection rules and confounders have a way of sneaking in between data acquisition and analysis. The most common version is: select on property A, test for property B, report the test as if A and B are independent — but A's selection rule already biases B.

Diagnostic signature.

- A "top- N " filter applied before a test, with no random / unselected comparison.
- Effect-size statistics that aren't residualized against expression level, token frequency, or coexpression.
- A test set that excludes high-expression / high-popularity / high-degree items by accident of the pipeline (e.g., scGPT 512-token window dropping low-rank genes).
- The same data used for both ranking and confirmatory testing.

Prescription.

- For every "selected" feature set claim, run the same analysis on a random unselected sample and report both. State the gap honestly: if the random set still shows the effect, the selection didn't drive it; if it doesn't, the selection did.
 - For every effect-size claim, residualize against the most likely confound (coexpression, token frequency, expression level) and report the residual effect.
 - Document tokenization-window dropouts: which genes were excluded and whether their inclusion would change the conclusion.
 - Avoid double-dipping by holding out a slice for confirmatory testing before any feature selection.
-

P6. No positive control for null findings

Reviewer signature. *"How do I know your pipeline can detect signal at all? Show me a positive control where the answer is known."*

Why it recurs. A null finding is uninterpretable without evidence that the pipeline *can* detect a true signal. Reviewers have repeatedly required a synthetic + real-data positive-control battery before a null result was acceptable.

Diagnostic signature.

- A pipeline that reports a clean negative outcome (e.g., "0/12 layers pass null gap") with no companion experiment confirming the pipeline detects signal when it should.
- An evaluation framework that's only ever been run on cases where the answer is unknown, never on cases where the answer is known.

Prescription.

- For every load-bearing null finding, add a short positive-control phase (synthetic graph with planted ground truth + same metrics + same null hierarchy) showing the pipeline detects signal at planted SNR. This can be quick: a 200-gene synthetic graph with 50 planted regulatory edges + one classifier eval per condition.
 - Optionally, a real-data positive control on a hand-curated case with strong known truth (e.g., GATA1/MYC/TAL1 in K562 with ChIP-seq direct targets).
 - If the positive control fails, *that's* the finding — don't suppress.
-

P7. Single-condition generalization

Reviewer signature. *"You used K562 — that's an immortalized cancer line not in either model's pre-training corpus." "Conclusions are based on three tissue domains; show held-out tissue." "Show this works on more than one architecture."*

Why it recurs. Foundation-model interpretability work is often demonstrated on whatever data was convenient. Reviewers ask: does the finding generalize beyond the one chosen condition?

Diagnostic signature.

- Findings stated for a single tissue / cohort / cell type / model architecture / random seed.
- Cross-condition agreement reported only as a within-family measurement (e.g., two models that share a tokenizer and training recipe); not triangulated against an out-of-family member (e.g., scGPT, an attention- only architecture on rank-binned values).
- A "we choose this dataset because it's convenient" justification with no acknowledgement of the dataset's idiosyncrasies.

Prescription.

- Pick at least one held-out condition (tissue / dataset / architectural family) for every load-bearing finding; report the result.
 - When triangulating cross-model, include at least one out-of-family member (e.g., a model with a different tokenizer and architecture).
 - For headline claims, run ≥ 3 random seeds and report mean $\pm$ SD.
-

P8. Stability of headline numbers

Reviewer signature. *"What's the bootstrap CI on +0.117?" "Does the result hold at threshold 0.3 instead of 0.5?" "What if you change PCA dim, k, or seed?"*

Why it recurs. A point estimate without a stability range invites suspicion that it's a single-pick of the favorable corner of a parameter grid.

Diagnostic signature.

- Headline Δ AUROC / Δr / Cohen's d reported as a single number without a SD, CI, or sweep range.

- Hyperparameters (PCA dim, kNN k, threshold cutoffs, Louvain resolution) fixed at single values without sensitivity sweeps.
- DeLong test or bootstrap absent for AUROC differences.

Prescription.

- For headline claims: bootstrap (1000 resamples; resample at the inference unit, e.g. by TF), DeLong for AUROC differences, report 95% CI.
- For load-bearing hyperparameters: at minimum a 3-point sweep with worst-case reported in main FINAL_SUMMARY.
- Where a sweep reveals threshold-dependent claims, restrict the claim to the threshold-stable range and say so.

P9. Causal / regulatory / mechanistic language overreach

Reviewer signature. *"Wiring diagram" is overstated.* *"Replace 'confirms' with 'is consistent with'."* *"Distinguish model-level causality from biological causality."* *"The paper reads like it was written by AI."*

Why it recurs. The evidence base of an interpretability pipeline is correlational by default; the language slips toward causal/mechanistic because that's what makes the result feel important. Reviewers consistently push back, often word-for-word.

Diagnostic signature.

- Abstract or §1 contains: "encodes regulation", "causal circuit", "wiring diagram", "complete pathway", "the model understands", "the model agrees".
- "Confirms" used where the evidence is at most "is consistent with".
- A negative result understated or buried (e.g., a transfer experiment near chance treated as supportive).
- AI-cadence template: "X is not Y. It is the Z."

Prescription.

- One global pass over each pipeline's FINAL_SUMMARY.md and the project README, swapping causal/mechanistic verbs for evidence-grade verbs:
 - "encodes regulation" → "is correlationally aligned with curated regulatory annotations"
 - "causal circuit" → "model-internal ablation graph"
 - "confirms" → "is consistent with"
 - "agrees with" → "co-varies with"
 - "discovers" → "recovers" (when comparing to a known prior)
- Define an explicit four-level claim taxonomy in any abstract that makes a claim about regulation: (1) decoding, (2) cross-model agreement, (3) intervention-consistent directional association, (4) causal mechanism. State which level the paper claims; do not exceed it.
- Own the negatives — name them as findings, not as gaps.

P10. Reproducibility scaffolding

Reviewer signature. *"Provide environment.yml."* *"Archive code with a DOI."* *"Confirm every script referenced in the paper is in the public repo."* *"Report computational footprint."*

Why it recurs. This is the cheapest-to-fix and most-frequently-flagged class. Reviewers expect a frozen, citable, runnable record.

Diagnostic signature.

- A pipeline that cites a script path that doesn't exist at the pinned commit.
- A run folder with no requirements.txt / environment.yml / no pinned versions, even informally.
- Compute cost / wall time absent — reviewer cannot estimate replication effort.
- A repo URL in the README that is private or does not resolve.

Prescription.

- One audit-time check per run: every script path in `FINAL_SUMMARY.md` and `README.md` resolves; every pinned-commit hash in the pipeline `$2` resolves under `repos/<name>/PINNED_COMMIT.txt`.
 - Add to each run a `RUNTIME.md` (or extend `STATUS.md`) with: total wall time per phase, peak memory if known, hardware used.
 - Where a public repo is referenced, verify accessibility and Zenodo DOI in the audit; record either as "verified" or "missing" in the audit report.
-

6 The checklist as deployed

This is the audit specification as it was used in the MaxToki deployment. In Study C it was given to the deployed-checklist arm (named `deployed` in the workflow script). It is copied word for word below, including its first section, which says where its ten patterns came from.

Audit pipeline — recurring reviewer-flagged issues

1. Overview

This is a **meta-audit pipeline**, not a research pipeline. It does not produce new biology; it audits the artefacts of existing research pipelines (any of the files under `pipelines/` applied to a target model under projects/`<target>/`) against the **recurring patterns** that peer reviewers have consistently flagged across the project's lineage of papers.

The audit is built bottom-up from the actual review materials in `review_plans/`: the BMC Genomics revision (attention/perturbation paper), the Bioinformatics revision (causal circuit tracing / SAE paper), the PLOS ONE revision (topology / 141-hypotheses paper), the *Neuron*-style review of the intelligence-genes paper, the *Biogerontology* revision (longevity mech-interp), and the BCB2026 review (disagreement-arbitration GRN paper). Across six independent reviews of six different papers by different reviewers, the same ten clusters of methodological objections recur. This pipeline turns those clusters into a checklist + workflow for examining any future run artefact against that pattern catalogue.

The output is a **per-project audit report** that names which patterns are present in a given project's run artefacts, where the supporting evidence is, and what concrete fix or qualifying language is recommended. The audit is deliberately sized to be doable in 1–2 days per project; it is not a full re-run.

This pipeline is **complementary** to existing review-style material in this repo and does not duplicate it (per user direction):

- `prompts/reviewer-1-style-correctness.md`, `prompts/reviewer-2-research-quality.md`, `prompts/brainstormer-ideas.md` remain authoritative as composable critique prompts. This pipeline cites them as cross-references but does not re-derive what they already cover.
- `CLAUDE.md` lists known failure modes (co-expression confound, feature-shuffle null trap, annotation-database circularity, selective reporting). This pipeline expands those into a structured catalogue grounded in `review_plans/`.

2. Source

- **Evidence base:** the six review documents in `review_plans/`, abbreviated below as [NMI] (BMC Genomics revision), [BI0] (Bioinformatics SAE revision), [PLOS] (PLOS ONE topology revision), [NEU] (Neuron-style intelligence-genes review), [BGER] (Biogerontology longevity revision), [BCB] (BCB2026 disagreement-arbitration review). The round-2 housekeeping PDF (`response_to_reviewers_r2.pdf`) is included for completeness but contributes only cross-reference / placeholder fixes.
- **Companion material in this repo:** `prompts/reviewer-1-style-correctness.md`, `prompts/reviewer-2-research-quality.md`, `prompts/brainstormer-ideas.md`, `CLAUDE.md` "Domain conventions" section. Cross-referenced, not modified.

- **Pinned commit hash:** N/A — this pipeline is methodological. It depends on the *content* of `review_plans/` at the time of writing (2026-05-07); when new review documents are added to that folder, §6 should be re-checked against them.

3. Inputs

A target-model project folder, e.g. `projects/maxtoki/`, with the conventional layout:

<code>projects/<target>/</code>	
├── <code>README.md</code>	← project-level architecture + run order
├── <code>runs/</code>	
│ ├── <code><pipeline-name>-<size>/</code>	
│ │ ├── <code>README.md</code>	← scoped execution plan + scope deltas
│ │ ├── <code>FINAL_SUMMARY.md</code>	← cross-pipeline synthesis
│ │ ├── <code>EXTENDED_FINDINGS.md</code>	← optional: extension-phase additions
│ │ ├── <code>STATUS.md</code>	← optional, for autoloop runs
│ │ ├── <code>scripts/</code>	← per-phase runners
│ │ ├── <code>outputs/</code>	← per-phase results (CSV, JSON, NPY)
│ │ └── <code>logs/</code>	← per-phase stdout
│ └── <code>...</code>	
└── <code>notes/, planning/</code>	← per-project working notes (optional)

The audit consumes:

1. Each run's `README.md`, `FINAL_SUMMARY.md`, and `EXTENDED_FINDINGS.md` as the primary claim surface.
2. Each run's `outputs/**/*.{csv,json}` for the quantitative evidence behind those claims.
3. Each run's `scripts/*.py` to verify claimed methods are implemented and to spot-check null/baseline/CV-split logic.
4. The project-level `README.md` for cross-pipeline triangulation (e.g., do verdicts from spectral-geometry, attention-GRN, SAE, topology agree?).

The audit does **not** re-extract embeddings, re-train any model, or re-run any pipeline. It examines what's already on disk.

4. Outputs

A single audit report, conventionally placed at:

```
projects/<target>/audits/audit-<YYYYMMDD>.md
```

The report has one section per recurring-issue pattern (see §6) with:

- **Verdict per run:** clean / present-but-acknowledged / present-and-unaddressed.
- **Evidence pointers:** specific file paths + line ranges + claim quotes.
- **Recommended action:** one of (a) re-run with corrected method, (b) add qualifying language to `FINAL_SUMMARY/EXTENDED_FINDINGS`, (c) explicitly note the limitation in §"Known pitfalls" of the relevant pipeline file in `pipelines/`, (d) no action — already handled.
- **Effort estimate:** ≤30 min / 1-2 hr / ≥half-day.

Plus an executive summary at the top: **N issues found, M acknowledged in run docs, K requiring action, distribution by severity.**

A second optional output is a per-run `audit-delta.md` written into each audited run folder, summarizing only that run's audit findings — useful when the project has many runs and the consolidated report becomes long.

5. Dependencies

- No code dependencies — the audit is a methodology document applied by reading and writing markdown.
- Conceptual dependencies (cross-references, not modifications):

- prompts/reviewer-1-style-correctness.md — for citation, equation, and typesetting checks beyond the scope here.
- prompts/reviewer-2-research-quality.md — for research-design critique that overlaps but is broader than recurring-reviewer-pattern matching.
- prompts/brainstormer-ideas.md — for generating fix proposals once an issue is identified.
- CLAUDE.md "Recurring failure modes" list — co-expression confound, feature-shuffle null trap, annotation-database circularity, selective reporting of positives, the "looks great until you control properly" pattern, immune-tissue concentration of signal.

6. Methodology — the recurring-issue catalogue

Ten patterns recur across the six reviews. Each is named by the *spirit* of the objection, not by any specific paper's instance of it. For each pattern, the entry below states:

- **Reviewer signature** — how this objection typically reads
- **Why it recurs** — the structural reason it keeps appearing
- **Diagnostic signature in run artefacts** — what to grep / read for
- **Common manifestations specific to this repo's pipelines**
- **Prescription** — fix vs acknowledge vs qualify
- **Cited by** — which review document(s) flagged this

The audit workflow (apply to a project end-to-end) is in §6.11.

P1. Null model is too weak / not anchored to chance

Reviewer signature. *"No baseline given for what 0.53 shared ontology means — anchor it to a chance baseline." "The expression-matched null may be insufficient — control for variance and dropout, not just mean." "How much of the effect survives a stricter null?"*

Why it recurs. Single-cell foundation-model artefacts are dense, so a naive shuffle (gene-label permutation, feature-shuffle) produces a null much weaker than the structure that actually drives the observed result. Reviewers want to see (i) every quantitative claim *anchored* to a chance value, and (ii) a hierarchy of nulls that progressively rules out confounders.

Diagnostic signature.

- A claim of the form "X = 0.NN" without an adjacent "vs chance baseline 0.MM ± SD".
- A null implemented only as `np.random.permutation(labels)` or `np.random.shuffle(features)` with no degree preservation, no composition matching, and no coexpression matching.
- A "p-value" without a stated null family.
- A "ΔAUROC" reported only against the trivial 0.5 baseline rather than against a coexpression / matched-degree null.

Common manifestations here.

- Phase 5 / Phase 6 ΔAUROC tables that only report `delta_vs_coex` without also reporting `delta_vs_max_null` (the strict max across feature-shuffle, coexpression, rewire).
- H123-family scores reported with sign-shuffle null but without the TF-degree-stratified variant.
- "ΔAUROC +0.03" with no statement of effect size relative to the null SD.

Prescription.

- For every numeric claim in `FINAL_SUMMARY.md` that is meant to be meaningful (i.e., load-bearing), attach a "(null mean ± SD; p95)" suffix. If the suffix would be misleading because the relevant null isn't computed, compute it (cheap; reuse phase-12-style strict-max-null harness).
- Adopt the strict max-null framing as the default for any claim that appears in an abstract or executive summary. Per-domain breakdowns at the loose null are fine in supplementary detail.
- Where a single null family was used, name it inline ("...vs feature-shuffle null; the same comparison against degree-preserving rewiring is in §X").

Cited by. [PLOS R2/R3], [BIO E12], [NEU R1.4], [BGER R3.1], [BCB C6]. Already in CLAUDE.md as "feature-shuffle null trap".

P2. Trivial / gene-level baseline absent

Reviewer signature. *"What does a simpler model already give you?" "Does the foundation-model embedding add **incremental value** over PCA on gene expression / marginal correlation / gene-level mean expression?"*

Why it recurs. Foundation-model claims are most credible when stated as "this beats the simple thing." Without a trivial baseline, the reader cannot distinguish "model encodes X" from "X is recoverable from raw expression statistics."

Diagnostic signature.

- Headline AUROC reported in absolute terms with no comparison to PCA, raw-correlation, or gene-mean baselines.
- "Cross-model alignment $r=0.78$ " reported without checking whether r between the two models' *static input embeddings* (which are trained independently but on similar token sets) already produces $r \approx 0.78$ — i.e., the alignment is a property of the input layer, not learned dynamics.
- Discussion section claims biological understanding without naming a simpler model that would or would not produce the same number.

Common manifestations here.

- Cross-model CCA results without a gene-coexpression baseline ("does coexpression alone produce comparable r ?").
- H123 motif-community Δ AUROC without a trivial coexpression-only baseline.
- Spectral-geometry / SAE positive findings without a PCA-on-expression comparator.

Prescription.

- Add to the validation step (§9 of any pipeline) an explicit "trivial baseline" line: PCA-on-expression, marginal correlation, or gene-mean — whichever is the natural simple comparator for that endpoint.
- Convert any absolute-AUROC claim to a " Δ AUROC over trivial baseline" claim in FINAL_SUMMARY.md. If the delta is zero or negative, that *is* the finding and should be stated explicitly.
- For cross-model claims, always also report the static-input-only alignment as a floor.

Cited by. [NMI R1.1] (BMC's central re-framing: "no regulatory signal" → "no incremental value over gene-level features"), [BGER R1.1] (PCA matches scFM in 5/5 cohorts), [BIO E5] (input-size-normalized comparison), [BIO E7] (partial-correlation test).

P3. Endpoint-object mismatch

Reviewer signature. *"Perturbation DE measures direct + indirect; you can't infer direct regulation from this." "AUROC vs TRRUST is an edge-recovery task; perturbation-target prediction is an effect-magnitude task; they are not the same thing." "Embedding shift is a model-internal quantity; what makes you think it tracks biological importance?" "TRRUST is curated literature; ChIP-seq in the matched cell type is closer to truth."*

Why it recurs. Mech-interp work routinely conflates (a) what the metric arithmetically measures, (b) what the model is internally doing, and (c) the biological claim made in the abstract. Reviewers home in on this gap.

Diagnostic signature.

- An abstract sentence "the model encodes regulatory relationships" supported only by an AUROC against a curated database.
- "Direct target" used loosely without a ChIP-seq peak or a perturbation + time-resolved direct-effect filter.
- Any claim that "the model causes X" supported by a correlational embedding-shift analysis.

- Reuse of an annotation database for *both* feature selection and evaluation (annotation-database circularity).

Common manifestations here.

- Phase-7+8 H123 framed as "regulatory signal" when the actual measurement is community co-membership consistent with TRRUST sign annotations.
- Spectral-geometry "B-cell attractor" framed as a causal claim when the evidence is a geometric correlate.
- SAE Stage-2 CRISPRi 54.6% directional accuracy described as "near-chance causal evidence" without qualifying that the labels themselves include indirect targets.

Prescription.

- In every pipeline file's §1 Overview and FINAL_SUMMARY's TL;DR, restate what the endpoint *literally* measures before the biological claim. Use the explicit two-objective framing from [NMI §5.1]: "Objective A (mechanistic / edge-recovery against curated database)" vs "Objective B (perturbation-response prediction)" — and state which objective each claim speaks to.
- Tag annotation databases used as ground truth with the cell-type / context in which they were curated. Note explicitly when that context does not match the analysis context.
- Replace "encodes" / "causes" with "is consistent with" / "co-varies with" unless an intervention-based test was actually done.

Cited by. [NMI R1.2/R3.2] (perturb-seq indirect-effect problem), [BIO R1-M2/R1-M3] (ChIP-seq matched-cell-type required, TRRUST insufficient), [NEU R1.2] (embedding shift as proxy), [BGER R3.2] ("mechanism-grade evidence" → "intervention-consistent directional association"), [BCB C8] (Dixit transfer near-chance buried). Already in CLAUDE.md as "annotation-database circularity".

P4. Pseudoreplication / wrong unit-of-inference / leaky CV

Reviewer signature. *"You have 500 cells but only 3 donors — your inference unit is donors, not cells, and your power is ~27%." "Edge-level 5-fold CV leaks gene-level features; use GroupKFold by TF." "Cell-type labels are imputed from markers; treat as exploratory, not confirmatory."*

Why it recurs. Single-cell data is hierarchical (cell $\subset$ donor $\subset$ cohort, edge $\subset$ TF, etc.) and the natural unit at which the *biological* claim lives is rarely the same as the unit at which sample size N is large. CV / nulls that ignore this hierarchy systematically inflate significance.

Diagnostic signature.

- A p-value computed over cells when the claim is about donors / cohorts / cell-types.
- Reported $n=500$ cells with no $n=\text{donors}$, $n=\text{cohorts}$, or $n=\text{independent-conditions}$ companion.
- StratifiedKFold rather than GroupKFold when features are shared across rows (edges sharing a TF; gene pairs sharing an endpoint).
- A bootstrap implemented at the row level when the row-level units are not exchangeable.
- Cell-type-stratified AUROC reported without acknowledging that cell-type labels are noisy.

Common manifestations here.

- H123-family per-pair AUROC computed over thousands of pairs, reported as " $n=2544$ pairs" — but pairs share endpoints, so effective N is much smaller.
- Phase-1 splits implemented as TF-disjoint OR target-disjoint, not always as the harder dual-axis disjoint.
- Cell-type analyses reported as confirmatory when based on marker-gene thresholds rather than reference-mapped labels.

Prescription.

- Whenever a numeric claim is reported, name the inference unit explicitly: " $n=\text{NN edges over MM TFs over KK cells}$ " with the unit at which the null / CV / bootstrap is grouped.
- For shared-endpoint metrics, run GroupKFold (by TF, by target, by both) and report the worst of the three. If the conclusion is robust, fine; if not, restate.

- For cell-type stratification, report a sensitivity analysis where labels are alternatively assigned via reference-mapping (Azimuth / SingleR).

Cited by. [NEU R1.1/R2.1] (pseudoreplication; n=3 donors, ~27% power), [BGER R1.1/R3.4] (donor-aware splits, gate calibration), [BCB C2] (GroupKFold by TF / target), [NEU R1.3] (cell-type marker fragility).

P5. Selection-driven inflation / double-dipping / confound carry-through

Reviewer signature. *"You picked the 120 best-annotated features and report properties of those — that's a biased subset." "You ranked genes on the same data you tested them on." "Effect strength is correlated with expression level — control for it." "Tokenization window excluded the genes you couldn't test."*

Why it recurs. Selection rules and confounders have a way of sneaking in between data acquisition and analysis. The most common version is: select on property A, test for property B, report the test as if A and B are independent — but A's selection rule already biases B.

Diagnostic signature.

- A "top-N" filter applied before a test, with no random / unselected comparison.
- Effect-size statistics that aren't residualized against expression level, token frequency, or coexpression.
- A test set that excludes high-expression / high-popularity / high-degree items by accident of the pipeline (e.g., scGPT 512-token window dropping low-rank genes).
- The same data used for both ranking and confirmatory testing.

Common manifestations here.

- SAE atlas / circuit work that selects "annotated" features then reports "53% biological coherence" — without a random-feature comparison.
- Phase-6 triangle-defect Δ AUROC without checking whether the signal survives coexpression residualization (which, when checked in Phase 11 iter_03, it didn't).
- Token-frequency stratified analysis missing — H123 signal could be a popularity confound (which we ruled out in Phase 11 iter_05, but the rule- out wasn't run in the original pipeline).

Prescription.

- For every "selected" feature set claim, run the same analysis on a random unselected sample and report both. State the gap honestly: if the random set still shows the effect, the selection didn't drive it; if it doesn't, the selection did.
- For every effect-size claim, residualize against the most likely confound (coexpression, token frequency, expression level) and report the residual effect.
- Document tokenization-window dropouts: which genes were excluded and whether their inclusion would change the conclusion.
- Avoid double-dipping by holding out a slice for confirmatory testing before any feature selection.

Cited by. [BIO R2-M1/E6/E7] (annotation bias; partial correlation), [NEU R1.4/R2.2] (expression-coupling confound), [NEU R2.3] (tokenization audit), [NEU R2 minor 4] ("double-dipping" bias), [BGER R3.3] (cohort composition shifts attenuate signal). Already in CLAUDE.md as "co-expression confound" and "selective reporting".

P6. No positive control for null findings

Reviewer signature. *"How do I know your pipeline can detect signal at all? Show me a positive control where the answer is known."*

Why it recurs. A null finding is uninterpretable without evidence that the pipeline *can* detect a true signal. The strongest reviewer concern in this project's lineage was [NMI R1.3] requiring a synthetic + real-data positive-control battery before a null result was acceptable.

Diagnostic signature.

- A pipeline that reports a clean negative outcome (e.g., "0/12 layers pass null gap") with no companion experiment confirming the pipeline detects signal when it should.
- An evaluation framework that's only ever been run on cases where the answer is unknown, never on cases where the answer is known.

Common manifestations here.

- topology-141-217M reports a clean negative on H123 in MaxToki without a positive control demonstrating the same H123 implementation detects H123-style signal in a synthetic graph with planted signed-motif structure.
- attention-GRN-217M near-zero "curveball" without a planted-edge synthetic control.

Prescription.

- For every load-bearing null finding, add a short positive-control phase (synthetic graph with planted ground truth + same metrics + same null hierarchy) showing the pipeline detects signal at planted SNR. This can be quick: a 200-gene synthetic graph with 50 planted regulatory edges + one classifier eval per condition.
- Optionally, a real-data positive control on a hand-curated case with strong known truth (e.g., GATA1/MYC/TAL1 in K562 with ChIP-seq direct targets).
- If the positive control fails, *that's* the finding — don't suppress.

Cited by. [NMI R1.3 §3] (full synthetic + real-data positive control), [BGER R1.1, R3.5] (workflow ablation; what would have been concluded without each block), [BIO R2-M2] (causal validation tightening).

P7. Single-condition generalization

Reviewer signature. *"You used K562 — that's an immortalized cancer line not in either model's pre-training corpus." "Conclusions are based on three tissue domains; show held-out tissue." "Show this works on more than one architecture."*

Why it recurs. Foundation-model interpretability work is often demonstrated on whatever data was convenient. Reviewers ask: does the finding generalize beyond the one chosen condition?

Diagnostic signature.

- Findings stated for a single tissue / cohort / cell type / model architecture / random seed.
- Cross-condition agreement reported only as a within-family measurement (e.g., MaxToki ↔ Geneformer, both Llama-style on Ensembl tokens); not triangulated against an out-of-family member (e.g., scGPT, an attention- only architecture on rank-binned values).
- A "we choose this dataset because it's convenient" justification with no acknowledgement of the dataset's idiosyncrasies.

Common manifestations here.

- topology-141 / spectral-geometry reporting on lung/immune/external_lung alone with no held-out tissue.
- Cross-model claims based only on Geneformer V2-316M when scGPT was available (this was addressed in topology-141's Phase 4 vs scGPT, which found alignment does NOT generalize across architectural families — a single-condition generalization failure caught only by triangulation).
- Single-seed runs without bootstrap stability.

Prescription.

- Pick at least one held-out condition (tissue / dataset / architectural family) for every load-bearing finding; report the result.
- When triangulating cross-model, include at least one out-of-family member (e.g., scGPT for an autoregressive-Ensembl finding; or vice versa).
- For headline claims, run ≥ 3 random seeds and report mean $\pm$ SD.

Cited by. [PLOS R1] (held-out kidney tissue), [NMI R1.4] (multi-model coverage table; scGPT ablation extension), [BIO R1-M1] (non-immortalized, training-covered cells), [NEU R2.3] (cross-architecture benchmarking), [BGER R3.3] (cohort-composition attribution).

P8. Stability of headline numbers

Reviewer signature. *"What's the bootstrap CI on +0.117?" "Does the result hold at threshold 0.3 instead of 0.5?" "What if you change PCA dim, k, or seed?"*

Why it recurs. A point estimate without a stability range invites suspicion that it's a single-pick of the favorable corner of a parameter grid.

Diagnostic signature.

- Headline Δ AUROC / Δr / Cohen's d reported as a single number without a SD, CI, or sweep range.
- Hyperparameters (PCA dim, kNN k, threshold cutoffs, Louvain resolution) fixed at single values without sensitivity sweeps.
- DeLong test or bootstrap absent for AUROC differences.

Common manifestations here.

- " Δ AUROC +0.094, 6/6 domain-splits" reported as a single point estimate without the bootstrap distribution under TF-resampling.
- PCA-20 used everywhere with no documented check at PCA-{10, 30}.
- KNN k=12 fixed; no documented sensitivity sweep.

Prescription.

- For headline claims: bootstrap (1000 resamples; resample at the inference unit, e.g. by TF), DeLong for AUROC differences, report 95% CI.
- For load-bearing hyperparameters: at minimum a 3-point sweep with worst-case reported in main FINAL_SUMMARY.
- Where a sweep reveals threshold-dependent claims, restrict the claim to the threshold-stable range and say so.

Cited by. [BIO R2-M3/E9, R2-M5/E11] (bootstrap stability; threshold sensitivity), [PLOS R2.4] (CV + hyperparameter sweep), [BCB C6, C7] (DeLong + bootstrap CIs; threshold robustness), [NEU R2.5 minor 1] (sampling stability of ranks).

P9. Causal / regulatory / mechanistic language overreach

Reviewer signature. *"'Wiring diagram' is overstated." "Replace 'confirms' with 'is consistent with'." "Distinguish model-level causality from biological causality." "The paper reads like it was written by AI."*

Why it recurs. The evidence base of an interpretability pipeline is correlational by default; the language slips toward causal/mechanistic because that's what makes the result feel important. Reviewers consistently push back, often word-for-word.

Diagnostic signature.

- Abstract or §1 contains: "encodes regulation", "causal circuit", "wiring diagram", "complete pathway", "the model understands", "the model agrees".
- "Confirms" used where the evidence is at most "is consistent with".
- A negative result understated or buried (e.g., a transfer experiment near chance treated as supportive).
- AI-cadence template: "X is not Y. It is the Z." (per [BGER R4]).

Common manifestations here.

- spectral-geometry "B-cell germinal-centre attractor" framed as a discovered mechanism when the evidence is geometric.
- Stage-2 SAE circuit-tracing language carrying causal weight despite CRISPRi 54.6% directional accuracy (near-chance).
- attention-GRN curveball ≈ 0 — a clean negative — at risk of being framed as "negligible signal" instead of "no incremental value over coexpression".

Prescription.

- One global pass over each pipeline's `FINAL_SUMMARY.md` and the project `README`, swapping causal/mechanistic verbs for evidence-grade verbs:
 - "encodes regulation" → "is correlationally aligned with curated regulatory annotations"
 - "causal circuit" → "model-internal ablation graph"
 - "confirms" → "is consistent with"
 - "agrees with" → "co-varies with"
 - "discovers" → "recovers" (when comparing to a known prior)
- Define an explicit four-level claim taxonomy in any abstract that makes a claim about regulation: (1) decoding, (2) cross-model agreement, (3) intervention-consistent directional association, (4) causal mechanism. State which level the paper claims; do not exceed it.
- Own the negatives — name them as findings, not as gaps.

Cited by. [BIO R2-M2/M4/m2] (causal-language audit), [BGER R3.2] ("mechanism-grade evidence" reframe), [BGER R4] (AI-cadence rewrite), [BIO R1-m3] ("confirms" → "suggests"), [BCB C8] (own the Dixit negative), [PLOS R5] (anthropomorphic language). Already in `CLAUDE.md` as "selective reporting of positives".

P10. Reproducibility scaffolding

Reviewer signature. *"Provide environment.yml." "Archive code with a DOI." "Confirm every script referenced in the paper is in the public repo." "Report computational footprint."*

Why it recurs. This is the cheapest-to-fix and most-frequently-flagged class. Reviewers expect a frozen, citable, runnable record.

Diagnostic signature.

- A pipeline that cites a script path that doesn't exist at the pinned commit.
- A run folder with no `requirements.txt` / `environment.yml` / no pinned versions, even informally.
- Compute cost / wall time absent — reviewer cannot estimate replication effort.
- A repo URL in the `README` that is private or does not resolve.

Common manifestations here.

- Run scripts/phase*.py paths cited in `FINAL_SUMMARY.md` should exist; always check post-revision that the cited paths still resolve.
- The pinned-commit hash in each pipeline's \$2 must still resolve in `repos/<name>/PINNED_COMMIT.txt`.
- Per-phase log files (`logs/phase*.log`) should record wall time so the scalability profile can be reconstructed without rerunning.

Prescription.

- One audit-time check per run: every script path in `FINAL_SUMMARY.md` and `README.md` resolves; every pinned-commit hash in the pipeline \$2 resolves under `repos/<name>/PINNED_COMMIT.txt`.
- Add to each run a `RUNTIME.md` (or extend `STATUS.md`) with: total wall time per phase, peak memory if known, hardware used.
- Where a public repo is referenced, verify accessibility and Zenodo DOI in the audit; record either as "verified" or "missing" in the audit report.

Cited by. [PLOS J2/J5] (code sharing, reference audit), [NMI R2.7] (GitHub repo audit), [BIO editor] (Zenodo archival), [NEU R2.2] (environment pinning), [PLOS R4] (computational cost).

6.11 Audit workflow

For a target project (e.g. `projects/maxtoki/`), the audit proceeds as:

1. **Inventory.** List every run under `projects/<target>/runs/*`. For each run, record: pipeline source (which pipelines/<X>.md it implements), present documents (`README` / `FINAL_SUMMARY` / `EXTENDED_FINDINGS` / `STATUS`), total artefact volume.
2. **Per-pattern sweep (P1-P10 above).** For each of the ten patterns:

- Read each run's `FINAL_SUMMARY.md` and `EXTENDED_FINDINGS.md` against the pattern's diagnostic signature.
 - Spot-check `outputs/**/*_summary.json` for the relevant numeric evidence.
 - Spot-check `scripts/*.py` for null / CV / baseline implementation.
 - Record verdict per run: **clean** / **acknowledged** / **unaddressed**.
3. **Cross-pipeline triangulation.** Some patterns surface only when comparing two runs:
 - P3 (endpoint mismatch) and P9 (causal language) are easier to spot when the project's `README.md` aggregates verdicts that contradict at the language level even if they agree at the data level.
 - P7 (single-condition generalization) often appears as "this run only used Geneformer; another run only used scGPT; the project never ran both on the same task".
 4. **Severity grading.** Each unaddressed finding is graded:
 - **Critical** — the finding's headline claim cannot stand without action.
 - **Major** — claim survives but with material qualification.
 - **Minor** — language-level fix or scaffolding fix.
 5. **Action proposal.** Per finding, propose one of:
 - **(re-run)** specific phase needs to be re-executed with a corrected null / baseline / CV / control. Reference the existing pipeline phase that is closest to the needed fix; estimate effort.
 - **(qualify)** language fix to `FINAL_SUMMARY.md` and/or pipeline §1 overview. Provide the exact verb-swap.
 - **(annotate)** add to pipeline's §10 "Known pitfalls" list as a forward-looking caution.
 - **(no-op)** already handled, no action needed; record evidence.
 6. **Audit report.** Compose `projects/<target>/audits/audit-<YYYYMMDD>.md` with: executive summary (counts by severity), per-pattern section with per-run findings, action proposal table sorted by effort × severity.
 7. **Optional per-run delta.** If actionable findings concentrate in a small number of runs, write a short `audit-delta.md` into each affected run folder so future readers see the audit's verdict where the run itself lives.

The full pass on a single project with ~5–10 runs is a 1–2 day exercise (reading + judgment, not running). The audit is *not* itself a research contribution and should not produce new claims; it produces qualifications and re-run proposals.

7. Code references

This pipeline does not run code. It cross-references existing code locations that are useful when an audit finding suggests a re-run:

- **Strict max-null harness** for P1: any pipeline's `scripts/phase12_strict_max_null.py` (e.g., `projects/maxtoki/runs/topology-141-217M/scripts/phase12_strict_max_null.py`).
- **Coexpression-residualization template** for P2/P5: `projects/maxtoki/runs/topology-141-217M/scripts/phase11_autoloop.py:iter_03_coex_residual_triangle`.
- **Token-frequency stratification template** for P5: `projects/maxtoki/runs/topology-141-217M/scripts/phase11_autoloop.py:iter_05_token_frequency_strata`.
- **Cross-model triangulation template** for P7: `projects/maxtoki/runs/topology-141-217M/scripts/phase4_scgpt_cross_model.py` (cross-architecture) and `phase14_cross_model_cca.py` (within-family).
- **Annotation-extension chain template** for P5: `projects/maxtoki/runs/topology-141-217M/scripts/phase8ext_h124_h138_chain.py`.
- **Bootstrap / stability template** for P8: borrowable from `repos/topology-biomechinterp2/iterations/iter_0009/run_iter0009_screen.py` (bootstrap-stability harness at the autoloop reference commit) — adapt as needed.

- **Composition-matched control template** for P1/P5: repos/longevity-mechinterp/.../stage10_* and stage11_* (when present in the local clone) document the "delta-form intervention layered on pre-existing SAE artifacts" recipe from [BGER §R1.3].

8. Parameters

The audit thresholds below are conservative defaults; they can be relaxed per-project if explicitly justified in the audit report.

Parameter	Default	Rationale	Used in pattern
min_null_families_for_strict_claim	3	Match the topology-141 pipeline's strict-max-null framing (feature-shuffle, coex-matched, degree-rewiring).	P1
require_trivial_baseline_in_summary	yes	Any abstract or executive-summary number must report Δ over a trivial baseline.	P2
min_groupkfold_axes_for_pair_claim	2	For shared-endpoint metrics (gene-pair edges), require GroupKFold by both endpoints.	P4
random_unselected_comparison	yes	For "annotation-selected feature" claims, require a random-unselected companion.	P5
positive_control_required	yes-for-load-bearing-negatives	Synthetic graph + planted ground truth at minimum.	P6
min_held_out_conditions	1	At least one tissue / cohort / model / dataset NOT used during method tuning.	P7
min_seeds_for_headline	3	Bootstrap or seed sweep for any abstract-level number.	P8
min_threshold_sweep_points	3	At least 3 hyperparameter values reported with worst-case in main summary.	P8
causal_language_audit	every-revision	Run a verb-swap pass on FINAL_SUMMARY before declaring the run complete.	P9
repo+_doi	yes	Public repo + Zenodo DOI for any externally-cited run.	P10

9. Validation

The audit "passes" for a given project when:

1. The audit report exists at `projects/<target>/audits/audit-<YYYYMMDD>.md` with all ten pattern sections populated (each labelled clean / acknowledged / unaddressed per run).
2. Every **critical** finding has either a re-run scheduled, a re-run completed, or a documented decision-not-to-rerun with reviewer-style justification.
3. Every **major** finding has a `FINAL_SUMMARY.md` qualification accepted into the run's documentation (verifiable by diff).
4. Every **minor** finding has an action item listed in the relevant pipeline file's §10 "Known pitfalls" if the issue is structural to the pipeline rather than incidental to one run.
5. Reproducibility scaffolding (P10) reports zero broken script paths and zero broken pinned-commit hashes for the project.
6. The audit report's executive summary closes with an explicit statement of which patterns the project remains exposed to (i.e., what a future reviewer of a paper based on this project would still flag) — even if no further action is feasible right now. Honesty about residual exposure is itself a deliverable.

If any of these are false, the audit is incomplete; record what's missing in the audit report's "Open items" section.

10. Known pitfalls

The audit itself can fail in characteristic ways. Reviewers of an audit would themselves invoke patterns like P1–P9; the audit is not exempt.

1. **Audit-driven selective reporting.** It is tempting to record only the patterns that have clean prescriptions and skip those that would demand expensive re-runs. Force yourself to record every pattern's verdict, including "present-and-unaddressed-because-too-expensive". That honesty is part of the deliverable.
2. **The "looks great until you control properly" pattern, applied to audits.** A run can pass cursory inspection of its `FINAL_SUMMARY.md` and fail when the underlying CSVs are checked. Always go from claim to numeric-evidence to script-implementation, in that order, for any load-bearing finding. Don't stop at the markdown layer.
3. **Recurring-pattern catalogue drift.** This pipeline is grounded in `review_plans/` as of the writing date in §2. New review documents will add new patterns and may invalidate old ones. The catalogue in §6 is not frozen — re-derive it whenever `review_plans/` materially changes (new paper review added, new revision round on an existing paper).
4. **False sense of completeness from checking ten patterns.** Reviewers raise things outside this catalogue too. The ten patterns are necessary conditions for an audit to be useful, not sufficient. If a pipeline's methodology has a unique-to-itself failure mode, the audit must invent the appropriate pattern slot for it; do not force every objection into one of the ten.
5. **Re-runs introduce their own selection bias.** Re-running a phase with a corrected null is not free of bias if the correction was chosen to recover a desired signal. Pre-register the re-run's expected outcome before running; if the re-run's outcome contradicts the pre-registered expectation, that *is* the finding.
6. **Audit reports are not papers.** They should be terse, evidence-pointer- heavy, and free of biological narrative. The audit's job is to surface issues; the pipeline files and `FINAL_SUMMARYs` are where biological claims live (suitably qualified).
7. **The audit can collide with active research.** If a project is mid-revision (e.g., responding to a journal review of a paper based on that project), the audit findings should feed into that revision rather than be filed separately. Coordinate by cross-referencing the relevant `review_plans/<paper>_revision_response_*.md` from the audit report.
8. **"This pattern doesn't apply to us" is rarely true.** Each of the ten patterns appeared in a different paper by a different reviewer. The prior probability that *none* of them apply to a new project is very low. If a sweep yields zero unaddressed findings, re-check more

carefully — the most likely explanation is that the sweep was too shallow, not that the project is uniquely clean.

7 Agent prompts

All prompts are template strings in three workflow scripts in the folder `tools/`: `studyA_workflow.js`, `studyB_workflow.js` and `studyC_workflow.js`. Each template is shown as it is written in the script. Placeholders of the form `${...}` are filled in by the script at run time; a note under each template says what each one holds. Line breaks written as `\n` in the scripts are shown as real line breaks. In all three scripts, every agent call uses the read-only agent type (`agentType: 'Explore'`) with `model: 'opus'` (Claude Opus 5.5). The JSON output schemas are given by their field names only: `[]` marks a list, and braces give the fields of each object.

7.1 Study A (`tools/studyA_workflow.js`)

7.1.1 Constants

These lines are copied from the script. The review prompt uses BUDGET and one of the two ARM sentences.

```
const WS = '<EVAL_ROOT>'
const PY = '<CODE_ROOT>/analysis-tasks/bin/python'
const CHECKLIST = WS + '/protocol/CHECKLIST_GENERIC.md'

const BUDGET = 'Spend about the effort of a careful two-hour expert review; you may use up to
↳ roughly 60 tool calls.'
const ARM = {
  generic: 'Review the deliverable critically, as an expert reviewer would before publication:
↳ check the code, the numbers and the claims against each other and against the data.',
  checklist: `Review the deliverable using the checklist at ${CHECKLIST} (you may also read that
↳ one file). Apply all ten patterns to the deliverable.`
}
```

7.1.2 Executor prompt

The script adds a last line, (Run `${x.rep}`.), where `${x.rep}` is the replicate number (1 to 5).

```
Your task is described in the file ${pkg}/BRIEF.md. Read it and carry it out fully. Read only
↳ inside ${pkg}. Run Python inline (python - <<'EOF' ... EOF) with the interpreter named in the
↳ brief; do not write files. When you are done, return the deliverable described in the brief as
↳ the structured output: 'report' (the Markdown report) and 'results' (verdict, estimates,
↳ analysis_script).
```

Placeholders:

- `${pkg}`: the task folder for this run (an opaque folder name under `analysis-tasks/`; Amendment 1, item 3).

7.1.3 Review prompt (generic and checklist arms)

The two review arms differ only in `${ARM[arm]}` and in the words added in the checklist arm.

```
The folder ${pkg} contains an analysis task (BRIEF.md, data/, methods/ and possibly contract/). An
↳ analyst carried out the task and produced the deliverable below. ${ARM[arm]} Read only inside
↳ ${pkg}${arm} == 'checklist' ? ' and the checklist file' : ''; you may run Python inline with
↳ ${PY} to check numbers or code (do not write files). List every problem that affects the
↳ correctness of the deliverable's results, verdict or claims. For each point give: location
↳ (report section or script line), severity (critical = the verdict or a headline result cannot
↳ stand; major; minor), description, evidence, recommended action. If you find no problem,
↳ return an empty list. ${BUDGET} Return the structured output.
```

```
DELIVERABLE – report:
${d.report}
```

```
DELIVERABLE – results (JSON):
${JSON.stringify(d.results)}
```

Placeholders:

- `${pkg}`: the task folder for this run (an opaque folder name under analysis-tasks/; Amendment 1, item 3).
- `${ARM[arm]}`: the arm sentence: `ARM.generic` or `ARM.checklist` (shown above).
- `${arm === 'checklist' ? ' and the checklist file' : ''}`: adds the words " and the checklist file" in the checklist arm only.
- `${PY}`: the Python interpreter path (constant `PY` above).
- `${BUDGET}`: the budget sentence (constant `BUDGET` above).
- `${d.report}`: the executor's Markdown report.
- `${JSON.stringify(d.results)}`: the executor's results object (verdict, estimates, analysis script), as JSON.

7.1.4 Repair prompt

The same repair prompt follows both review arms.

```
The folder ${pkg} contains an analysis task (read BRIEF.md). An analyst produced the deliverable
↪ below, and a second analyst raised the points listed after it. Revise the deliverable: check
↪ each point against the data (run Python inline with ${PY}; do not write files), fix what is
↪ right, keep what is right in the original, and re-run analyses as needed. The revised report
↪ must stand on its own: do not mention the points, the second analyst, or that this is a
↪ revision. Return the full revised deliverable as structured output in the same format:
↪ 'report' and 'results' (verdict, estimates, complete analysis_script). Read only inside
↪ ${pkg}.
```

```
ORIGINAL DELIVERABLE – report:
${d.report}
```

```
ORIGINAL DELIVERABLE – results (JSON):
${JSON.stringify(d.results)}
```

```
POINTS RAISED (JSON):
${JSON.stringify(rv.points)}
```

Placeholders:

- `${pkg}`: the task folder for this run (an opaque folder name under analysis-tasks/; Amendment 1, item 3).
- `${PY}`: the Python interpreter path (constant `PY` above).
- `${d.report}`: the executor's Markdown report.
- `${JSON.stringify(d.results)}`: the executor's results object, as JSON.
- `${JSON.stringify(rv.points)}`: the points returned by the review, as JSON.

7.1.5 Redaction before grading

Before grading, `redact()` replaces the words below with `[REDACTED]` in the deliverable's report and results. The first pattern ignores case. `SPEC.md` and the labels `P1` to `P10` are matched with their case as written. The function is copied from the script.

```
function redact(s) {
  return String(s).replace(/\b(specifications?|specs?|contracts?|checklists?|audit(?:s|or|ors|ed|ing)?|review(?:s|er|ers|ed|ing)?|repair(?:s|ed|ing)?|revised|revision|second analyst)\b/gi,
  ↪ '[REDACTED]').replace(/\bSPEC\.md\b/g, '[REDACTED]').replace(/\bP(10|[1-9])\b/g,
  ↪ '[REDACTED]')
}
```

7.1.6 Grading prompt

Two graders score each final deliverable. A third grader is run when the two disagree on whether the verdict is correct, or when one of them returns nothing.

```

You are grading one deliverable for an analysis task against a fixed answer key. The task brief is
→ ${pkg}/BRIEF.md (read it; you may read the package data and run Python inline with ${PY},
→ without writing files). The answer key is ${key} (read it fully: key_verdict, key_conclusion,
→ acceptable_alternatives, key_numbers, traps, false_statement_checks). Some words in the
→ deliverable are replaced by [REDACTED]; ignore that and do not try to guess how the
→ deliverable was produced.
Grade: (1) verdict: is the deliverable's verdict correct according to the key? An acceptable
→ alternative counts as correct only under the conditions the key states. (2) every trap in the
→ key: handled yes / partly / no / not_applicable, with a one-line reason based on the report
→ AND the analysis script. (3) materially false statements in the report (use the key's
→ false_statement_checks and your own checking; count a statement as false only if the data or
→ the key clearly contradict it). (4) every key number for which the deliverable reports a
→ matching quantity: is it within the key's tolerance? Grader ${g}. Return the structured
→ output.

DELIVERABLE – report:
${red.report}

DELIVERABLE – results (JSON):
${JSON.stringify(red.results)}

```

Placeholders:

- `${pkg}`: the task folder for this run (an opaque folder name under analysis-tasks/; Amendment 1, item 3).
- `${PY}`: the Python interpreter path (constant PY above).
- `${key}`: the path of the frozen answer key (studyA/keys/T<n>/key.json in the evaluation workspace).
- `${g}`: the grader number: 1 or 2, or 3 for the extra grader.
- `${red.report}`: the report of the deliverable being graded, after `redact()`. The deliverable is the executor's output (review arm none) or the repaired output (generic or checklist arm).
- `${JSON.stringify(red.results)}`: the results object of the same deliverable, after `redact()`, as JSON.

7.1.7 Review-point grading prompt

Each review (generic and checklist) is judged point by point by one grader.

```

Below are review points raised about a deliverable for the analysis task in ${pkg} (brief:
→ ${pkg}/BRIEF.md). The answer key for the task is ${key}. For each point decide: 'valid' (a
→ real problem in the deliverable that matters for its results, verdict or claims), 'invalid'
→ (the point is wrong, or the deliverable already handles it, or it does not matter), or
→ 'unclear'. You may read the package and run Python inline with ${PY} (do not write files).
→ Give a one-line reason for each. Return the structured output.

DELIVERABLE – report:
${d.report}

DELIVERABLE – results (JSON):
${JSON.stringify(d.results)}

REVIEW POINTS (JSON):
${JSON.stringify(rv.points)}

```

Placeholders:

- `${pkg}`: the task folder for this run (an opaque folder name under analysis-tasks/; Amendment 1, item 3).
- `${key}`: the path of the frozen answer key.
- `${PY}`: the Python interpreter path (constant PY above).
- `${d.report}`: the executor's Markdown report (the one that was reviewed; not redacted).
- `${JSON.stringify(d.results)}`: the executor's results object, as JSON (not redacted).
- `${JSON.stringify(rv.points)}`: the review's points, as JSON.

7.1.8 Output schemas

```
EXEC_SCHEMA (executor and repair):
  report, results {verdict, estimates[] {name, value, ci_low, ci_high, method_of_uncertainty},
    ↳ analysis_script}
REVIEW_SCHEMA (review):
  summary, points[] {id, location, severity, description, evidence, recommended_action}
GRADE_SCHEMA (grading):
  verdict_given, verdict_correct, verdict_reason, traps[] {trap_id, handled, reason},
    ↳ false_statements[] {statement, why_false}, key_numbers[] {name, reported_value,
    ↳ within_tolerance}
RGRADE_SCHEMA (review-point grading):
  points[] {id, judgement, reason}
```

7.2 Study B (tools/studyB_workflow.js)

7.2.1 Constants

These lines are copied from the script.

```
const WS = '<EVAL_ROOT>'
const N = '<CODE_ROOT>/analysis-tasks'
const PY = N + '/bin/python'
const CHECKLIST = WS + '/protocol/CHECKLIST_GENERIC.md'
const BUDGET = 'Spend about the effort of a careful one-hour expert review; you may use up to
↳ roughly 50 tool calls.'
const ARM = {
  generic: 'Review the analysis critically, as an expert reviewer would before publication: check
↳ the code, the outputs and the claims in the summary against each other and against the
↳ data.',
  checklist: `Review the analysis using the checklist at ${CHECKLIST} (you may also read that one
↳ file). Apply all ten patterns to the analysis.`
}
```

7.2.2 Review prompt (generic and checklist arms)

The script adds a last line, (Run `${x.rep}.`), where `${x.rep}` is the replicate number (1 to 3).

```
The folder ${N}/${id} contains a data analysis: a README, code, data, outputs and a results
↳ summary (SUMMARY.md). ${ARM[arm]} Read only inside ${N}/${id}${arm === 'checklist' ? ' and the
↳ checklist file' : ''}. You may run the code or your own checks with ${PY} (run Python inline;
↳ do not write files; if the code writes outputs, do not run it in a way that changes the
↳ folder). List every problem that affects the correctness of the analysis's results or of the
↳ claims in its summary. For each: location (file and line), severity (critical = a headline
↳ result or claim cannot stand; major; minor), description, evidence, recommended action. If you
↳ find no problem, return an empty list. ${BUDGET} Return the structured output.
```

Placeholders:

- `${N}`: the folder that holds the item packages (constant N above).
- `${id}`: the item's opaque id, for example B1144.
- `${ARM[arm]}`: the arm sentence: ARM.generic or ARM.checklist (shown above).
- `${arm === 'checklist' ? ' and the checklist file' : ''}`: adds the words " and the checklist file" in the checklist arm only.
- `${PY}`: the Python interpreter path (constant PY above).
- `${BUDGET}`: the budget sentence (constant BUDGET above).

7.2.3 Grading prompt

Two graders score each review. A third grader is run when the two differ on `error_detected` or on the number of load-bearing false alarms, or when one of them returns nothing.

```

Grade one review of the analysis in ${N}/${id} against its answer key ${WS}/studyB/keys/${id}.json
→ (read the key fully; it says whether the analysis contains a documented error, gives a
→ detection rule, known true facts and possible false alarms). You may read the package and run
→ checks with ${PY} (inline; do not write files). Do not try to guess how the review was
→ produced.
(1) If the key has a documented error: does at least one finding detect it under the key's
→ detection rule? List the finding ids. If the key has no documented error, set error_detected
→ to null.
(2) Classify EVERY finding: 'documented_error' (it is the key's error), 'other_real_problem' (a
→ genuine problem not in the key, that you can confirm), 'false_alarm' (it claims a problem that
→ does not exist, e.g. contradicted by the key's known true facts or by the data),
→ 'minor_or_style' (true but trivial), or 'disputed' (you cannot decide without deeper
→ checking). Say whether the finding is load-bearing (claims a headline result or claim is
→ wrong). Grader ${g}. Return the structured output.

REVIEW FINDINGS (JSON):
${JSON.stringify(rv.findings)}

```

Placeholders:

- `${N}`: the folder that holds the item packages (constant N above).
- `${id}`: the item's opaque id.
- `${WS}`: the evaluation workspace folder (constant WS above).
- `${PY}`: the Python interpreter path (constant PY above).
- `${g}`: the grader number: 1 or 2, or 3 for the extra grader.
- `${JSON.stringify(rv.findings)}`: the review's findings, as JSON.

7.2.4 Verifier prompt

This prompt is written inline in the script. It is run once for a review when any grader marks one or more of its findings 'disputed'.

```

Check the findings below about the analysis in ${N}/${x.id} by running code and reading files
→ (${PY}, inline, do not write files; read only inside the package). Use the answer key
→ ${WS}/studyB/keys/${x.id}.json as background. For each finding decide: 'real_problem' (it
→ affects a result or a claim), 'false_alarm', or 'cannot_decide', with evidence.

FINDINGS (JSON):
${JSON.stringify(disputed)}

```

Placeholders:

- `${N}`: the folder that holds the item packages (constant N above).
- `${x.id}`: the item's opaque id.
- `${PY}`: the Python interpreter path (constant PY above).
- `${WS}`: the evaluation workspace folder (constant WS above).
- `${JSON.stringify(disputed)}`: the findings that at least one grader marked 'disputed', as JSON.

7.2.5 Output schemas

```

REVIEW_SCHEMA (review):
  summary, findings[] {id, location, severity, description, evidence, recommended_action}
GRADE_SCHEMA (grading):
  error_detected, detecting_finding_ids[], detection_reason, findings[] {id, classification,
  → load_bearing, reason}
VER_SCHEMA (verifier):
  verdicts[] {id, verdict, reason}

```

7.3 Study C (tools/studyC_workflow.js)

7.3.1 Constants

These lines are copied from the script.

```
const WS = '<EVAL_ROOT>'
const SNAP = WS + '/studyC/snapshot/maxtoki'
const PY = WS + '/bin/python'
const KEY = WS + '/protocol/STUDYC_KEY.csv'
```

7.3.2 Audit prompt

The audit prompt is put together from four parts, in this order (copied from the script):

```
CONTEXT + '\n\n' + ARMS[x.arm] + '\n\n' + OUTPUT + '\n\n' + BUDGET + '\n(Run ${x.r}.)'
```

'\n\n' is a blank line. ARMS[x.arm] is one of the three arm texts below. (Run \${x.r}.) is a last line with the replicate number (1 to 3).

7.3.3 CONTEXT

The folder \${SNAP} contains a research project: an agent-run mechanistic-interpretability study of
→ MaxToki-217M, a single-cell foundation model (an autoregressive transformer over ranked gene
→ tokens). It holds pipeline method specifications (pipelines/), per-run scripts, logs, configs
→ and outputs (runs/<pipeline>-217M/), setup code (setup/), small reference data
→ (reference_data/) and results summaries (summaries/ and runs/*/FINAL_SUMMARY.md, README.md,
→ EXTENDED_FINDINGS.md). Files larger than 20 MB and the model weights are not included; listing
→ files record their names, sizes and shapes. You cannot run the model. You can read every file
→ and run Python on the files present, using the interpreter \${PY} (numpy, pandas, scipy,
→ scikit-learn, torch CPU). Run Python inline (python - <<'EOF' ... EOF); do not write files.
Read only inside \${SNAP}

Placeholders:

- \${SNAP}: the snapshot folder of the project (constant SNAP above).
- \${PY}: the Python interpreter path (constant PY above).

7.3.4 ARMS.deployed (deployed-checklist arm)

TASK: Audit this project using the audit specification at \${WS}/protocol/CHECKLIST_DEPLOYED.md
→ (you may also read that one file). Follow its methodology (section 6, including the workflow
→ in 6.11) against the project in \${SNAP}. Where the specification says to write a report file,
→ return the report content in the structured output instead.

Placeholders:

- \${WS}: the evaluation workspace folder (constant WS above).
- \${SNAP}: the snapshot folder of the project (constant SNAP above).

7.3.5 ARMS.checklist (checklist arm)

TASK: Audit this project using the checklist at \${WS}/protocol/CHECKLIST_GENERIC.md (you may also
→ read that one file). Apply all ten patterns to every pipeline run in \${SNAP}.

Placeholders:

- \${WS}: the evaluation workspace folder (constant WS above).
- \${SNAP}: the snapshot folder of the project (constant SNAP above).

7.3.6 ARMS.generic (generic review arm)

TASK: Review this project critically, as an expert reviewer would before publication. Find errors
→ and weaknesses that affect the correctness of its results or its claims: check the code, the
→ numbers and the claims against each other, for every pipeline run in \${SNAP}.

Placeholders:

- \${SNAP}: the snapshot folder of the project (constant SNAP above).

7.3.7 OUTPUT

OUTPUT: list every problem you found that affects the correctness of a result or a claim (code
→ bugs, wrong statistics, missing controls, wrong descriptions, unsupported claims,
→ reproducibility gaps). One finding per distinct problem. For each: location (file path
→ relative to the project folder, with line numbers where possible), the pipeline it concerns, a
→ short category label, severity (critical = a headline result or claim cannot stand; major = it
→ stands only with material qualification; minor), a description of what is wrong, the evidence
→ you checked (numbers, code lines, computations), and what should be done.

7.3.8 BUDGET

Work as a careful expert would in about one working day. You may use up to roughly 150 tool calls;
→ stop when you have covered the project. Return your result as the structured output; do not
→ write any files.

7.3.9 Grading prompt

Two graders score each audit against the key.

You are grading one audit of a research project against a fixed answer key. Do not try to guess
→ how the audit was produced.
ANSWER KEY: the CSV file `${KEY}` lists `68` known errors (one row each: id, short_name,
→ pipeline, error_type, severity, where_it_lives, deterministic_evidence, notes). Read it fully.
→ You may also read files in `$(SNAP)` to understand locations.
AUDIT FINDINGS (JSON):
`$(JSON.stringify(review.findings))`
For EVERY key row, decide whether the audit detected that error: 'yes' = at least one finding
→ identifies the same problem (same nature and essentially the same location or result) clearly
→ enough that acting on it would lead to the fix; 'partial' = a finding points at the same
→ result or code and raises a related concern, but misses or misstates the actual problem; 'no'
→ otherwise. List the matching finding ids. Be strict and consistent; a generic remark ("add
→ confidence intervals everywhere") counts as 'partial' at most for a specific error, and only
→ if it would plausibly lead to finding it.
Then, for EVERY finding, list the key rows it matches (empty list if none).
Grader `$(g)`. Read only the key file and inside `$(SNAP)`. Return the structured output; do not write
→ files.

Placeholders:

- `$(KEY)`: the path of the frozen key, `STUDYC_KEY.csv` (constant KEY above).
- `68`: the literal text 68, the number of rows in the key.
- `$(SNAP)`: the snapshot folder of the project.
- `$(JSON.stringify(review.findings))`: the audit's findings, as JSON.
- `$(g)`: the grader number: 1 or 2, or 3 for the extra grader.

7.3.10 Adjudication prompt

An adjudicator is always run after the two graders. Its grading is the final one.

Two graders scored the same audit against an answer key (`$(KEY)`). Resolve their disagreements.
→ Read the key and the audit findings below; for each key row where the two graders differ in
→ 'detected', decide the correct value using the same rules: 'yes' = a finding identifies the
→ same problem clearly enough that acting on it would lead to the fix; 'partial' = points at the
→ same result or code with a related but wrong or incomplete concern; 'no' otherwise. For rows
→ where they agree, keep their value. Also reconcile the finding-to-key matches (union where
→ both are defensible; drop a match only if clearly wrong).
AUDIT FINDINGS:
`$(JSON.stringify(review.findings))`
GRADER A:
`$(JSON.stringify(a))`
GRADER B:
`$(JSON.stringify(b))`
Read only the key file and inside `$(SNAP)`. Return the full resolved grading (all key rows, all
→ findings) as the structured output.

Placeholders:

- `$(KEY)`: the path of the frozen key, `STUDYC_KEY.csv`.

- `${JSON.stringify(review.findings)}`: the audit's findings, as JSON.
- `${JSON.stringify(a)}`: the full grading returned by grader 1, as JSON.
- `${JSON.stringify(b)}`: the full grading returned by grader 2, as JSON.
- `${SNAP}`: the snapshot folder of the project.

7.3.11 Verifier prompt

The verifier is run once per audit, on the findings that match no key row in the final grading.

```
Below are findings from an audit of the research project in ${SNAP} that do not match any entry in
↳ a list of known errors. For each, decide by checking the project files (and running Python
↳ inline with ${PY} if useful) whether it is: 'real_error' (a genuine problem that affects a
↳ result or claim), 'real_but_trivial' (true but with no effect on any result or claim),
↳ 'false_alarm' (the claimed problem does not exist, or the code/number is actually correct), or
↳ 'cannot_decide' (cannot be checked with the files present). Give the severity if real
↳ (critical/major/minor) and a short reason with evidence. Be fair in both directions.
FINDINGS:
${JSON.stringify(unmatched)}
Read only inside ${SNAP}. Do not write files. Return the structured output.
```

Placeholders:

- `${SNAP}`: the snapshot folder of the project.
- `${PY}`: the Python interpreter path (constant PY above).
- `${JSON.stringify(unmatched)}`: the audit findings that match no key row in the adjudicated grading, as JSON.

7.3.12 Output schemas

```
REVIEW_SCHEMA (audit):
  summary, findings[] {id, location, pipeline, category, severity, description, evidence,
  ↳ recommended_action}
GRADE_SCHEMA (grading and adjudication):
  items[] {ledger_id, detected, finding_ids[], note}, findings[] {finding_id,
  ↳ matched_ledger_ids[]}
VERIFY_SCHEMA (verifier):
  verdicts[] {finding_id, verdict, severity_if_real, reason}
```

7.4 Exploratory arm (tools/studyA_haiku_isolated_workflow.js)

The exploratory re-run with Claude Haiku 4.5 (Amendment 3) used this script. It is a copy of `studyA_workflow.js` with two kinds of change: the model of the executor, review and repair agents, and the folder instructions. The executor, review and repair agents use `model: SUBJ`, which is Haiku unless the run passes another model. The graders and review-point graders still use `model: 'opus'`. Every agent gets its own folder, named by `F(label)`, and its agent label starts with AI: in place of A:. These lines are copied from the script:

```
const SUBJ = args.subject_model || 'haiku'
const AT = '<CODE_ROOT>/analysis-tasks/'
const F = lab => AT + 'u' + (100000 + fnv('1' + lab) % 900000)
const ISO = (f, extra) => `This folder is yours alone: ${f}. Work only inside it: read only files
↳ inside it${extra || ''}. If you need to write a file (for example a script or its output),
↳ write it inside this folder and nowhere else; never write to /tmp, a scratch folder or any
↳ other location.`
```

The ISO sentence as written. `${f}` is the agent's own folder. `${extra || ''}` is empty, or the extra words given in the call (listed below).

```
This folder is yours alone: ${f}. Work only inside it: read only files inside it${extra || ''}. If
↳ you need to write a file (for example a script or its output), write it inside this folder and
↳ nowhere else; never write to /tmp, a scratch folder or any other location.
```

Five prompts use ISO. In each, `pkg` is the agent's own folder. The call in each prompt is:

- executor prompt (`execPrompt`): `${ISO(pkg)}`

- review prompt (reviewPrompt): `${ISO(pkg, arm == 'checklist' ? ', plus the checklist file' : '')}`
- repair prompt (repairPrompt): `${ISO(pkg)}`
- grading prompt (gradePrompt): `${ISO(pkg, ', plus the answer key')}`
- review-point grading prompt (rgradePrompt): `${ISO(pkg, ', plus the answer key')}`

Below is every place where the wording of these five prompts differs between the two scripts. A line starting with - is the wording in `studyA_workflow.js`. The line starting with + under it is the wording in the re-run script. All other prompt text is the same.

Executor prompt (execPrompt):

```
- Read only inside ${pkg}. Run Python inline (python - <<'EOF' ... EOF) with the interpreter named
  ↳ in the brief; do not write files.
+ ${ISO(pkg)} Run Python with the interpreter named in the brief.
```

Review prompt (reviewPrompt):

```
- Read only inside ${pkg}${arm == 'checklist' ? ' and the checklist file' : ''}; you may run
  ↳ Python inline with ${PY} to check numbers or code (do not write files).
+ ${ISO(pkg, arm == 'checklist' ? ', plus the checklist file' : '')} You may run Python with
  ↳ ${PY} to check numbers or code.
```

Repair prompt (repairPrompt):

```
- inline with ${PY}; do not write files),
+ with ${PY}),
- Read only inside ${pkg}.
+ ${ISO(pkg)}
```

Grading prompt (gradePrompt):

```
- inline with ${PY}, without writing files).
+ with ${PY}). ${ISO(pkg, ', plus the answer key')}
```

Review-point grading prompt (rgradePrompt):

```
- task is ${key}. For each point
+ task is ${key}. ${ISO(pkg, ', plus the answer key')} For each point
- read the package and run Python inline with ${PY} (do not write files).
+ run Python with ${PY}.
```

The BUDGET and ARM constants, the output schemas and `redact()` are the same in both scripts.

8 Study A task briefs

Each Study A task had two briefs, one per executor arm. For each task, the paper-arm brief is given in full. For the contract arm, only the lines that differ are shown, as a unified diff: a line starting with + was added. The briefs are the frozen copies in `studyA/tasks/`. In the task folders the agents used, the interpreter path in the T1, T3 and T4 briefs was `<CODE_ROOT>/analysis-tasks/bin/python` in place of `<EVAL_ROOT>/bin/python`. The T2 briefs already had that path. This is the only other difference. The specifications (`contract/SPEC.md`), the source-method papers and the data are not reproduced here.

8.1 Task T1

The paper-arm brief, copied from `studyA/tasks/T1-paper/BRIEF.md`:

Task brief

1. Question

Do MaxToki-217M's attention maps encode regulator-to-target relationships? Use the TRRUST curated network as ground truth.

Give exactly one verdict:

- **supported**
- **not supported**
- **inconclusive**

2. Background

The model. MaxToki-217M is a transformer model of single-cell gene expression (Gómez Ortega & Theodoris, bioRxiv 2026). It uses the Llama decoder architecture: 11 layers, 8 attention heads per layer, hidden size 1,232, about 217 million parameters. It was trained on about 175 million human single-cell transcriptomes, first on single cells and then on sequences of cell states. A cell is given to the model as a sequence of gene tokens. Each gene's count is divided by that gene's median across a large reference corpus, and genes are put in order from highest to lowest value (rank-value encoding). The model is causal: each token attends only to itself and to tokens that come before it. Here each cell was given to the model on its own.

The cells. 2,000 unperturbed control cells (non-targeting guides) of the RPE1 cell line, drawn at random from the Replogle et al. 2022 genome-wide Perturb-seq screen.

The genes. 1,500 genes: the most variable genes in these 2,000 cells (variance of log1p of counts per 10,000) among the genes in the model's vocabulary.

The attention scores. Every cell was run through the model once. For layer index 8 (0-based; the 9th of 11 layers), the attention weight from the token of gene *i* (query) to the token of gene *j* (key) was averaged over the cells in which both genes were present, and then over the 8 heads. Attention weights are softmax outputs, so each lies between 0 and 1. Only this layer is provided. It was chosen before any evaluation, by its relative depth in the network.

The reference network. TRRUST v2 (human; Han et al. 2018) is a database of transcription factor (TF) to target-gene relationships, curated from the literature. Only rows where both the TF and the target are among the 1,500 genes are provided.

3. Materials

All paths are relative to this package folder.

Path	What it is
data/attention_layer8_headmean.npy	float32, shape (1500, 1500). Attention score for every ordered gene pair. Row <i>i</i> = query gene, column <i>j</i> = key gene. Unitless (mean softmax weight).
data/pair_cell_counts.npy	int32, shape (1500, 1500). Number of cells (out of 2,000) in which both genes were present as tokens; the attention score is an average over these cells.
data/genes.tsv	1,500 rows: index (row/column in the arrays), symbol, ensembl_id, maxtoki_token_id.
data/gene_stats_control_cells.tsv	1,500 rows: per-gene mean and variance of log1p(counts per 10,000), and dropout_rate (fraction of the 2,000 cells with a zero count).
data/rrust_edges_in_gene_set.tsv	TRRUST rows with both genes in the gene set: tf, target, mode, pmid, tf_index, target_index.
data/README.md	How each data file was made, step by step.
data/MANIFEST.sha256	sha256 of the data files.
methods/source_method_paper.pdf	The source paper for this kind of analysis. It applies the analysis to other single-cell models.

4. Deliverable

Return two things.

(a) report: a Markdown report with these sections, in this order:

1. **Question**
2. **Data**
3. **Methods**
4. **Results:** give every estimate with its uncertainty (for example a 95% confidence interval) and say what the unit of resampling was.
5. **Verdict:** one of supported / not supported / inconclusive, then 2-4 sentences of justification.
6. **Limitations**

(b) results: a JSON object with these fields:

- **verdict:** one of "supported", "not supported", "inconclusive".
- **estimates:** a list of objects, each with name, value, ci_low, ci_high, method_of_uncertainty.
- **analysis_script:** a complete, self-contained Python script. When it is run from the package root with <EVAL_ROOT>/bin/python, it must recompute every number reported in estimates from data/ and print them as JSON.

5. Limits

- CPU only. Python with numpy, pandas, scipy and scikit-learn is available at <EVAL_ROOT>/bin/python.
- Each command must finish within 9 minutes.
- Aim to finish within about 2 hours of work.
- Do not write files. Run Python inline.
- Read only inside this package folder.

Contract-arm brief: lines that differ

```

--- T1-paper/BRIEF.md
+++ T1-contract/BRIEF.md
@@ -57,0 +58,2 @@
+A method specification for this kind of analysis is provided in contract/SPEC.md;
  ↪ follow it.
+

```

8.2 Task T2

The paper-arm brief, copied from studyA/tasks/T2-paper/BRIEF.md:

Task brief

1. Question

Do the SAE feature circuits of MaxToki-217M predict the direction in which a gene's expression changes when another gene is silenced by CRISPRi in K562 cells?

Give exactly one verdict:

- **supported**
- **not supported**
- **inconclusive**

2. Background

The model. MaxToki-217M is a transformer model of single-cell gene expression (Gómez Ortega & Theodoris, bioRxiv 2026). It uses the Llama decoder architecture: 11 layers, 8 attention heads per layer, hidden size 1,232, about 217 million parameters. It was trained on about 175 million human single-cell transcriptomes, first on single cells and then on sequences of cell states. A cell is given to the model as a sequence of gene tokens. Each gene's count is divided by that gene's median across a large reference corpus, and genes are put in order from highest to lowest value (rank-value encoding). Here each cell was given to the model on its own.

Sparse autoencoders (SAEs). Twelve TopK SAEs (4,928 features each, $k = 32$), one per site. Site l (0 to 10) is the residual stream at the input of block l ; site 11 is the final normalised hidden state that enters the output head. Each SAE was trained on the residual-stream vectors of 500 K562 non-targeting control cells. Each feature has a list of top genes: the genes at whose token positions the feature is most active, on average, in those 500 cells.

The circuits. 120 source features were traced: at each of the sites 0, 3, 6 and 9, the 30 features whose top genes had the strongest gene-set enrichment. 200 K562 non-targeting control cells were used. For each cell and source feature at site s , the feature's contribution to the SAE reconstruction was removed from the input of block s , and the model was run on. At every later site, the change in every SAE feature's activation (averaged over the cell's token positions) was recorded. Over the 200 cells, each (source feature, target feature) pair gets a Cohen's d (mean change / SD of the change) and a consistency (the share of cells whose change has the more common sign). A pair is a circuit edge if $|d| > 0.5$ and consistency > 0.7 . An edge is "inhibitory" if removing the source feature lowers the target feature on average, and "excitatory" otherwise.

From circuits to genes (the method's rule). The method pairs each top gene of an edge's source feature with each top gene of its target feature. For each (source gene, target gene) pair it counts the supporting edges, keeps pairs with enough support, and predicts that silencing the source gene lowers the target gene when more than half of the support comes from inhibitory edges, and raises it otherwise. `data/README.md` gives the exact rule. `data/gene_pairs.parquet` holds the pairs it produces.

The knockdowns. K562 cells from the Replogle et al. 2022 CRISPRi Perturb-seq screen. For a silenced gene and a measured gene, the log-fold change (LFC) is the mean of $\log_{10}(\text{counts per } 10,000)$ over the cells carrying a guide against the silenced gene, minus the same mean over 3,000 non-targeting control cells. 227 silenced genes (32 to 575 cells each) are top genes of source features and appear in the gene pairs.

3. Materials

All paths are relative to this package folder.

Path	What it is
<code>data/circuit_edges.csv</code>	272,905 circuit edges: <code>src_layer</code> , <code>src_feature</code> , <code>tgt_layer</code> , <code>tgt_feature</code> , <code>cohens_d</code> , <code>consistency</code> , <code>sign</code> .
<code>data/source_features.tsv</code>	The 120 source features (site, feature id) with their top-10 genes.
<code>data/feature_top_genes.tsv</code>	Top-10 genes of every feature that appears in an edge (46,705 features).
<code>data/gene_pairs.parquet</code>	698,624 (silenced gene, target gene) pairs made by the method's rule: support counts, the method's prediction (<code>predicted_decrease</code>) and the observed <code>lfc</code> .
<code>data/knockdown_lfc.npz</code>	LFC of all 6,546 measured genes for each of the 227 silenced genes, with cells per silenced gene and the control cells' mean expression.
<code>data/silenced_genes.tsv</code>	The 227 silenced genes and their number of CRISPRi cells.
<code>data/README.md</code>	How each data file was made, step by step, and every column.
<code>data/MANIFEST.sha256</code>	sha256 of the data files.
<code>methods/source_method_paper.pdf</code>	The source paper for this kind of analysis. It applies the analysis to other single-cell models.

4. Deliverable

Return two things.

(a) report: a Markdown report with these sections, in this order:

1. **Question**
2. **Data**
3. **Methods**
4. **Results:** give every estimate with its uncertainty (for example a 95% confidence interval) and say what the unit of resampling was.
5. **Verdict:** one of supported / not supported / inconclusive, then 2-4 sentences of justification.
6. **Limitations**

(b) results: a JSON object with these fields:

- **verdict:** one of "supported", "not supported", "inconclusive".
- **estimates:** a list of objects, each with name, value, ci_low, ci_high, method_of_uncertainty.
- **analysis_script:** a complete, self-contained Python script. When it is run from the package root with `<CODE_ROOT>/analysis-tasks/bin/python`, it must recompute every number reported in estimates from data/ and print them as JSON.

5. Limits

- CPU only. Python with numpy, pandas (with pyarrow, for .parquet files), scipy and scikit-learn is available at `<CODE_ROOT>/analysis-tasks/bin/python`.
- Each command must finish within 9 minutes.
- Aim to finish within about 2 hours of work.
- Do not write files. Run Python inline.
- Read only inside this package folder.

Contract-arm brief: lines that differ

```

--- T2-paper/BRIEF.md
+++ T2-contract/BRIEF.md
@@ -72,0 +73,2 @@
+A method specification for this kind of analysis is provided in contract/SPEC.md;
  ↪ follow it.
+

```

8.3 Task T3

The paper-arm brief, copied from `studyA/tasks/T3-paper/BRIEF.md`:

Task brief

1. Question

Are the sparse-autoencoder features that respond to a transcription factor's knockdown specific to that factor's target genes?

Answer with exactly one verdict:

- supported
- not supported
- inconclusive

The question is about transcription factors (TFs) in general, across all knocked-down TFs in the data. GATA1 is a well-studied example that you may wish to look at.

2. Background

Model. MaxToki-217M is a single-cell foundation model: a decoder-only transformer (Llama architecture, 11 blocks, hidden size 1,232). A cell is given to the model as a sequence of gene tokens ordered by the gene's expression rank in that cell (highest first), between a `<bos>` and an `<eos>` token, at most 2,048 tokens.

Sparse autoencoder (SAE). A TopK SAE (4,928 features, $k = 32$) was trained on the model's residual-stream activations entering decoder block 5, using 500 K562 non-targeting control cells. Each feature has a list of its top-20 genes, built from those 500 cells.

Cells. K562 cells from the Replogle et al. (2022) genome-scale CRISPRi Perturb-seq screen: 7,500 knockdown cells for 87 TFs (up to 100 cells per TF), and 1,700 non-targeting control cells in three disjoint random groups (ref 400, pool 800, and the 500 catalog cells used for the SAE and the top-20 lists). Every cell was run through the model and encoded with the SAE. For each cell and feature you get one number: the feature's mean activation over the cell's gene tokens.

Target genes. TRRUST v2 and DoRothEA (all confidence levels, with a flag for DoRothEA's ChIP-seq-supported pairs), restricted to the 6,324 genes that can appear in a top-20 list.

Source method. [methods/source_method_paper.pdf](#) is the paper that introduced this kind of SAE atlas and the perturbation-response analysis, applied there to other single-cell foundation models.

3. Files

All data are in `data/`. `data/README.md` defines every file and column. In short:

file	content
<code>data/cell_feature_means.npz</code>	<code>mean_gene</code> (9,200 cells x 4,928 features, float32): per-cell feature values; rows (cell ids); <code>n_tokens</code>
<code>data/cell_manifest.csv</code>	one row per cell: id, group (ref / pool / kd / catalog), knocked-down TF, barcode, GEM group, UMI count
<code>data/feature_top20.tsv</code>	top-20 genes of each feature, with the activation values used to rank them
<code>data/feature_info.tsv</code>	per-feature summary of the top-20 lists
<code>data/gene_universe.tsv</code>	the 6,324 genes, with detection count in the 500 catalog cells, gene length and chromosome
<code>data/targets_trrust.tsv</code>	TRRUST v2 TF-target pairs (target in the gene universe)
<code>data/targets_dorothea.tsv</code>	DoRothEA TF-target pairs (target in the gene universe), with confidence level and ChIP-seq flag
<code>data/tf_info.tsv</code>	per TF: number of knockdown cells and how much of the TF's own mRNA remains, where measured

4. What to deliver

Return two things.

(a) A report in Markdown with these sections, in this order:

1. **Question**
2. **Data** (what you used)
3. **Methods**
4. **Results:** give every estimate with its uncertainty (for example a 95% confidence interval) and say what the unit of resampling or replication is.
5. **Verdict:** one of supported / not supported / inconclusive, plus 2 to 4 sentences of justification.
6. **Limitations**

(b) A results object (JSON) with these fields:

- `verdict`: one of supported, not supported, inconclusive
- `estimates`: a list of objects `{name, value, ci_low, ci_high, method_of_uncertainty}`

- `analysis_script`: a complete, self-contained Python script. Run from the package root with `<EVAL_ROOT>/bin/python`, it must recompute every number you report from the files in `data/` and print them as JSON.

5. Limits

- Use Python: `<EVAL_ROOT>/bin/python` (numpy, pandas, scipy, scikit-learn, statsmodels and torch are installed). CPU only.
- Each command must finish within 9 minutes.
- Aim to finish within about 2 hours of work.
- Do not write files. Run Python inline (for example `python - <<'EOF' ... EOF`).
- Read only inside this package folder.

Contract-arm brief: lines that differ

```

--- T3-paper/BRIEF.md
+++ T3-contract/BRIEF.md
@@ -38,0 +39,2 @@
+
+A method specification for this kind of analysis is provided in contract/SPEC.md;
↪ follow it.
```

8.4 Task T4

The paper-arm brief, copied from `studyA/tasks/T4-paper/BRIEF.md`:

Task brief

1. Question

Is MaxToki-217M's gene-embedding geometry aligned with scGPT's, and how does that alignment compare with its alignment to Geneformer V2-316M (which shares MaxToki's gene vocabulary)?

Give exactly one verdict:

- **aligned with scGPT about as well as with Geneformer**
- **aligned with scGPT but clearly less than with Geneformer**
- **not aligned with scGPT beyond chance**
- **inconclusive**

2. Background

The three models. All three are transformer models of single-cell gene expression. Each gives every gene its own input token, and each has a table with one learned vector per token (the gene embedding table). The tables are compared here.

- **MaxToki-217M** (Gómez Ortega & Theodoris, bioRxiv 2026). Llama decoder: 11 layers, 8 attention heads, hidden size 1,232, about 217 million parameters. It was trained on about 175 million human single-cell transcriptomes, first on single cells and then on sequences of cell states. A cell is given to the model as a list of gene tokens: each gene's count is divided by that gene's median across a large reference corpus, and genes are put in order from highest to lowest value (rank-value encoding). Genes are identified by Ensembl gene ID.
- **Geneformer V2-316M** (Theodoris et al. 2023; V2 release). BERT encoder: 18 layers, 18 attention heads, hidden size 1,152, about 316 million parameters. Trained by masked-gene prediction on about 104 million human single-cell transcriptomes, with the same rank-value encoding. Its gene tokens and token ids are the same as MaxToki's.
- **scGPT whole-human** (Cui et al. 2024). Transformer: 12 layers, 8 attention heads, embedding size 512. Trained on about 33 million human cells from CELLxGENE. Genes are identified by gene symbol. A gene's expression value is binned and encoded by a separate value encoder, which is added to the gene's token vector; the table here is the gene token part only.

The tables. Read directly from each released checkpoint (no model was run):

Model	File	Shape
MaxToki-217M	data/maxtoki_217m/embed_tokens.npy	(20275, 1232)
Geneformer V2-316M	data/geneformer_v2_316m/word_embeddings.npy	(20275, 1152)
scGPT whole-human	data/scgpt_whole_human/encoder_embedding_weight.npy (+ its LayerNorm in encoder_enc_norm.npz)	(60697, 512)

All are float32 and unitless. Row i of a table is the vector of that model's token id i . Each model's own token dictionary or vocabulary file is next to its table. `data/gene_ids/ensembl_symbol.csv` maps Ensembl gene IDs to gene symbols.

The genes. Three gene panels in `data/panels/` (382, 350 and 380 genes, as Ensembl IDs). Each was built from one single-cell dataset (Tabula Sapiens lung, Tabula Sapiens immune, and a separate lung dataset) by picking expressed genes with regulatory or marker annotations. Use these panels as the gene sets for the comparison.

`data/README.md` describes every file and how it was made.

3. Materials

All paths are relative to this package folder.

Path	What it is
data/maxtoki_217m/	MaxToki-217M token-embedding table, token dictionary (Ensembl ID to token id), model config.
data/geneformer_v2_316m/	Geneformer V2-316M token-embedding table, token dictionary (Ensembl ID to token id), model config.
data/scgpt_whole_human/	scGPT token-embedding table, its LayerNorm parameters, the checkpoint's <code>vocab.json</code> (gene symbol to token id) and <code>args.json</code> .
data/gene_ids/ensembl_symbol.csv	Ensembl gene ID to gene symbol table.
data/panels/	The three gene panels.
data/README.md	How each data file was made.
data/MANIFEST.sha256	sha256 of the data files.
methods/source_method_paper.pdf	The source paper for this kind of analysis. It applies the analysis to other single-cell models (scGPT and Geneformer); its cross-model alignment analysis is the relevant part.

4. Deliverable

Return two things.

(a) report: a Markdown report with these sections, in this order:

1. **Question**
2. **Data**
3. **Methods**
4. **Results:** give every estimate with its uncertainty (for example a 95% confidence interval) and say what the unit of resampling was.
5. **Verdict:** one of the four verdicts above, then 2-4 sentences of justification.
6. **Limitations**

(b) results: a JSON object with these fields:

- **verdict:** one of "aligned with scGPT about as well as with Geneformer", "aligned with scGPT but clearly less than with Geneformer", "not aligned with scGPT beyond chance", "inconclusive".
- **estimates:** a list of objects, each with name, value, ci_low, ci_high, method_of_uncertainty.
- **analysis_script:** a complete, self-contained Python script. When it is run from the package root with <EVAL_ROOT>/bin/python, it must recompute every number reported in estimates from data/ and print them as JSON.

5. Limits

- CPU only. Python with numpy, pandas, scipy and scikit-learn is available at <EVAL_ROOT>/bin/python.
- Each command must finish within 9 minutes.
- Aim to finish within about 2 hours of work.
- Do not write files. Run Python inline.
- Read only inside this package folder.

Contract-arm brief: lines that differ

```

--- T4-paper/BRIEF.md
+++ T4-contract/BRIEF.md
@@ -70,0 +71,2 @@
+A method specification for this kind of analysis is provided in contract/SPEC.md;
+  follow it.
+

```

9 Study A answer keys

The four answer keys (studyA/keys/T1 to T4/key.json) are laid out below without changing their text. For each key number, the key file also gives a written definition; the definitions are not reproduced here. The fields that point to how each key was made and checked (reference, verification, source, sources) and the list of verdict options (given in the briefs) are left out. The T2 key has one more field, grader_guidance, which is shown.

9.1 Task T1

Question. Do MaxToki-217M's attention maps encode regulator-to-target relationships? Use the TRRUST curated network as ground truth.

Key verdict. not supported

Key conclusion. Layer-8 attention ranks TRRUST targets above chance (pooled AUROC 0.605, TF-bootstrap 95% CI 0.549-0.655; 16 TFs, 138 positive pairs), but gene-level properties of the candidate gene do better with no model at all (variance alone 0.686; a mean/variance/dropout model 0.749). A degree-preserving null that keeps each TF's and each target's number of edges already gives 0.574, so about 70% of the excess over 0.5 is target popularity; the rest is small and borderline ($z = 1.8$, one-sided $p = 0.046$; per-TF statistic $z = 0.75$), disappears after OLS residualisation on gene features (0.492), and is matched by model-free pair scores from the same cells (expression proximity $z = 1.8$; co-detection lift $z = 3.1$). There is no evidence that attention encodes regulator-to-target relationships beyond gene-level properties and co-expression.

Acceptable alternatives.

- *Verdict.* inconclusive

Counts as correct only if. the report (1) shows that attention does not beat gene-level baselines, or that most of its above-chance AUROC is explained by gene-level properties or a degree-preserving null, AND (2) does not present the raw above-chance AUROC or a label-permutation p-value as evidence for encoding, AND (3) bases the hedge on the small, borderline or choice-dependent pair-specific excess (16 TFs; z about 1.8 for the provided

score, up to about 3 for symmetrised attention) or on not being able to separate it from co-expression with these data.

Reason. Some pair-specific excess over target popularity exists in some score variants (symmetrised attention: degree-null $z = 3.3$; OLS-residualised 0.551, CI 0.489-0.611), and the package has no expression matrix to compute co-expression directly. An analyst who controls for gene-level properties and degree but declines to call this a clear negative has reached the same scientific picture. 'supported' is never correct for this task.

Key numbers (29).

Name	Value	Tolerance
n_trust_rows	192	0
n_unique_non_self_edges	173	0
n_tfs_evaluated	16	0
n_positive_pairs	138	0
n_negative_pairs	23846	0
attention_auroc_pooled	0.60451	0.003
attention_auroc_pooled_ci_low	0.54869	0.015
attention_auroc_pooled_ci_high	0.65468	0.015
attention_auroc_per_tf_mean	0.58667	0.003
attention_auroc_tf_as_key	0.54629	0.003
attention_auroc_symmetrised_mean	0.63825	0.003
variance_auroc_pooled	0.68638	0.003
gene_model_auroc_pooled	0.74941	0.02
gap_variance_minus_attention	0.08186	0.005
gap_variance_minus_attention_ci_low	0.0281	0.02
gap_gene_model_minus_attention	0.14489	0.02
degree_null_mean	0.57394	0.006
degree_null_sd	0.01682	0.004
degree_null_z	1.81737	0.35
degree_null_p_one_sided	0.046	0.025
degree_null_z_per_tf_mean	0.74958	0.4
label_permutation_null_z	4.11979	0.5
expr_proximity_degree_null_z	1.84677	0.4
co_detection_lift_degree_null_z	3.09201	0.5
symmetrised_attention_degree_null_z	3.26273	0.5
residualised_attention_auroc_ols	0.49219	0.015
residualised_attention_auroc_hgb	0.55857	0.03
incremental_auroc_logreg	0.01284	0.008
attention_auroc_two_way_bootstrap_ci_low	0.50908	0.02

Traps (6).

Trap 1: gene_level_confound

- *Description.* TRRUST targets in this gene set are genes with particular gene-level properties (high variance, many listing TFs), and attention to a key gene depends strongly on gene-level properties (its expression rank sets its position in a causal, expression-ordered sequence). A raw AUROC above 0.5 therefore does not show pair-specific knowledge.
- *Correct handling.* Compares attention with gene-level scores on the same pairs (candidate variance 0.686; mean/variance/dropout model about 0.75) and/or residualises attention on gene features (OLS 0.492) or tests incremental value over gene features; reports that attention does not beat them.
- *Mishandling.* Reports attention AUROC 0.605 (CI excluding 0.5) as evidence that attention encodes regulatory relationships, with no gene-level comparison, or mentions gene-level effects only as a limitation without testing them.

Trap 2: degree_preserving_null

- *Description.* The 138 positives fall on 87 distinct target genes and 16 TFs with 4-21 targets each. Any score that favours popular target genes beats 0.5. The right null keeps each TF's and each target's number of edges, and it must actually rewire the network. The original deployment's 'degree-preserving' null left the evaluated network unchanged in 37 of 50 draws and reported a meaningless z of about 0.
- *Correct handling.* Uses a degree-preserving null (Curveball or checkerboard swaps, or an equivalent stratified permutation that keeps target popularity), checks that draws differ

from the observed network and keep the margins, and reports null mean about 0.574, z about 1.8 (p about 0.05) for the pooled statistic and z about 0.75 for the per-TF mean; concludes that most of the raw excess is target popularity.

- *Mishandling.* Uses only a uniform label permutation, a random-pair null, or a null that keeps only each TF's number of targets and draws targets uniformly (all centred near 0.50; z about 4.1-4.2, p about 0.001) and reads it as evidence of regulatory encoding; or runs a null that keeps the network almost unchanged and reports z near 0 as a clean negative.

Trap 3: resampling_unit

- *Description.* The roughly 24,000 pairs are not independent. They are crossed between 16 TFs and 1,499 candidate genes, and the 138 positives fall on 87 target genes, many shared across TFs. In these data, resampling pairs or resampling TFs gives almost the same interval for the provided score (pair bootstrap 0.547-0.655; TF bootstrap 0.549-0.655). Resampling TFs and candidate genes together widens it to about 0.51-0.70 and makes the variance - attention gap include zero (about -0.04 to 0.20); the gene-model gap stays above zero (about 0.002-0.28). So the unit does not change "above 0.5" for the provided score, but it does change how sure one can be about the gaps and about small excesses.
- *Correct handling.* States what was resampled and that pairs share TFs and target genes. Resamples TFs, candidate genes or both (TF bootstrap 0.549-0.655; two-way TF x gene bootstrap 0.509-0.699), or uses pair-level resampling while saying plainly that it ignores this dependence and is too narrow. Treats the small number of TFs (16) as a main limit.
- *Mishandling.* Treats the pairs as independent without saying so (pair bootstrap, DeLong, Hanley-McNeil or Mann-Whitney; e.g. $p = 4e-5$ against 0.5) and leans on that pair-level precision for a strong claim, for example that a small gap or a small excess over a null is clearly significant; or reports no uncertainty. A pair bootstrap on its own is not a mishandling here if the dependence is acknowledged, because it gives nearly the same attention interval as the TF bootstrap.

Trap 4: model_free_pair_signal

- *Description.* Beating the degree-preserving null is not by itself evidence of model knowledge. The model sees expression order and co-occurrence in its input, and TRRUST pairs are more co-detected and co-expressed than degree-matched pairs. Model-free pair scores built from the same cells beat the same null about as strongly as attention. The source re-evaluation found the same with |Spearman| co-expression ($z = 3.1$).
- *Correct handling.* Compares any excess over the degree null with a model-free pair score under the same null (expression proximity $z = 1.8$ matches the provided attention score; co-detection lift $z = 3.1$ matches symmetrised attention $z = 3.3$), or explicitly says the excess cannot be separated from co-expression with these data and does not claim regulatory encoding from it.
- *Mishandling.* Concludes 'supported' because some attention variant beats a degree-preserving null (e.g. symmetrised attention $z = 3.3$, $p = 0.001$) without any model-free pair comparison.

Trap 5: forking_paths

- *Description.* Many reasonable analyst choices move the number: score direction (TF as query 0.605, TF as key 0.546, symmetrised mean 0.638, max 0.647), pooled vs per-TF statistic (null $z = 1.8$ vs 0.75), TF threshold (≥ 1 target: 42 TFs, 0.595; ≥ 5 : 11 TFs, 0.609), residualiser (OLS 0.492 vs boosted trees 0.559), and gene model for incremental value (logistic +0.013 [0.0005, 0.025] vs boosted trees -0.015 [-0.035, 0.003] with folds by TF; boosted trees with folds by gene range from -0.013 to +0.046 across seeds and settings, because that gene-only model is weak). The original deployment over-stated one of these: it said residualising 'collapses attention to chance in every run', while in this run the result depends on the residualiser.
- *Correct handling.* Fixes the main choices before looking (or reports all variants), applies the same baselines and null to every variant, and does not build the verdict on the most favourable variant; notes that no variant beats the gene-level model and that the incremental value is not robust.
- *Mishandling.* Tries several variants and reports the best as the main result (e.g. symmetrised score with its degree-null $z = 3.3$, or the logistic incremental value +0.013

with CI just above 0) as decisive positive evidence; or claims a clean collapse to chance from one residualiser while another keeps 0.559 [0.488, 0.617].

Trap 6: *pair_set_definition*

- *Description.* The TRRUST file has 192 rows but 175 unique pairs (duplicates differ in mode or PMID), including 2 self-pairs (TF listed as its own target). The attention diagonal is self-attention and is about 160 times larger than an average off-diagonal score.
- *Correct handling.* Merges duplicate rows, drops self-pairs, excludes the TF itself from its own candidates (never scores the diagonal), and states the final counts (16 TFs, 138 positives, 23,846 negatives under the ≥ 3 targets rule).
- *Mishandling.* Counts duplicate rows as extra positives or scores self-pairs with the diagonal (AUROC rises to 0.610-0.624), or reports edge counts that mix rows and unique pairs.

False-statement checks (10).

1. Attention ranks TRRUST targets better than gene-level features such as the candidate's variance. (False: 0.605 vs 0.686; a mean/variance/dropout model gives about 0.75.)
2. The above-chance AUROC of attention reflects recognition of specific TF-target pairs. (False: a degree-preserving null already gives 0.574 of the 0.605.)
3. The provided attention score clearly beats a degree-preserving null ($z \geq 3$ or $p < 0.01$). (False: pooled $z = 1.8$, $p = 0.046$; per-TF $z = 0.75$, $p = 0.22$.)
4. A label-permutation p-value of about 0.001 shows that attention captures TF-target relationships. (False: that null keeps neither TF nor target degree; it is centred at 0.50 while degree alone gives 0.574.)
5. After removing gene-level effects, attention remains clearly above chance. (False: OLS residual 0.492 [0.404, 0.568]; boosted-tree residual 0.559 [0.488, 0.617].)
6. Attention adds robust predictive value over gene-level features. (False: +0.013 with a logistic gene model grouped by TF, CI just above 0; not significant when grouped by gene; -0.015 to -0.037 with a boosted-tree gene model grouped by TF. A positive boosted-tree gain with folds grouped by gene comes from a weak gene-only model; gene + attention there (0.66-0.71) is still below the logistic gene-only model (0.75).)
7. Beating the degree-preserving null shows that the model has learned regulatory knowledge. (False: model-free expression proximity and co-detection scores from the same cells beat the same null about as strongly.)
8. Attention scores are symmetric, so $\text{attention}[i, j]$ and $\text{attention}[j, i]$ carry the same information. (False: the model is causal; the two directions give 0.605 and 0.546.)
9. There are 192 distinct regulator-target edges in the gene set. (False: 192 rows, 175 unique pairs, 173 without self-pairs; 138 among the 16 evaluated TFs.)
10. The data show that no layer or head of MaxToki-217M carries TRRUST information. (Not supported: only the layer-8 head mean is provided. In the source data some earlier layers beat the degree null, though co-expression does too.)

9.2 Task T2

Question. Do the SAE feature circuits of MaxToki-217M predict the direction in which a gene's expression changes when another gene is silenced by CRISPRi in K562 cells?

Key verdict. not supported

Key conclusion. The circuit's direction predictions carry no detectable information. Over 698,624 (silenced gene, target gene) pairs from 227 knockdowns, accuracy is 49.8% (95% CI 49.2-50.4%, resampling silenced genes), 4.6 points below always predicting 'decrease' (54.3%; difference CI -5.8 to -3.3 points). Balanced accuracy is 49.7% (CI for minus 50%: -0.8 to +0.2 points) and MCC is -0.006 (CI -0.017 to +0.004); the AUROC of the inhibitory support fraction is 0.498. A rule with no model, the target gene's usual direction in other knockdowns, gets 58.7%, 8.9 points above the circuit. The tie rule, the sign of summed d , support filters and single source sites do not change this (21 variants: accuracy never above always-'decrease'). The only nominal positive is a post-hoc subset of strongly changed pairs ($|LFC| \geq 0.5$: 416 pairs, 59 genes, MCC 0.17): its excess is mostly between knockdowns, is not significant within knockdowns (CMH $p = 0.22$), does not survive a test over the thresholds tried ($p = 0.085$), and is far below the model-free rule on the same pairs (MCC 0.59). Over all pairs, the small

within-knockdown association (CMH odds ratio 1.011) disappears once the target gene's usual direction is accounted for (0.998).

Acceptable alternatives.

- *Verdict.* inconclusive

Counts as correct only if. the report (1) shows that on all pairs the circuit has no skill by a measure that is not fooled by the 54/46 class split (balanced accuracy, MCC, kappa, AUROC, or accuracy compared with always-'decrease' or with chance from both class rates), with uncertainty that resamples silenced genes or groups of them, AND (2) does not present accuracy against 50%, the precision of 'decrease' predictions, the share of pairs that decrease, or pair-level p-values as evidence of skill, AND (3) bases the hedge on a specific, computed limit: a small excess in strongly changed pairs that the report itself shows is fragile (post-hoc threshold, few genes, or not significant within knockdowns), or the noisy ground truth (most |LFC| below 0.05, no significance filter) while acknowledging that the circuit is also at chance on the strongly changed pairs or below a model-free direction rule.

Reason. The test is well powered for the full set (CIs within about 1 point of chance), so 'not supported' is the key. But strongly changed pairs are few (1.4% have |LFC| ≥ 0.25) and a small, non-robust within-knockdown excess appears at some thresholds (best single threshold $p = 0.01$ - 0.03 before correction). An analyst who finds no skill overall but declines to rule out a small effect on large changes has reached the same scientific picture. 'supported' is never correct for this task.

Key numbers (41).

Name	Value	Tolerance
n_gene_pairs	698624	0
n_silenced_genes	227	0
n_target_genes	6324	0
n_tied_pairs	72275	0
frac_observed_decrease	0.54313	0.0005
frac_predicted_decrease	0.50888	0.0005
accuracy	0.49761	0.0005
accuracy_ci_low	0.49188	0.004
accuracy_ci_high	0.5037	0.004
accuracy_minus_always_decrease	-0.04552	0.001
accuracy_minus_always_decrease_ci_low	-0.05792	0.006
accuracy_minus_always_decrease_ci_high	-0.03341	0.006
balanced_accuracy	0.49682	0.0005
balanced_accuracy_minus_half_ci_low	-0.0083	0.003
balanced_accuracy_minus_half_ci_high	0.00203	0.003
mcc	-0.00635	0.0005
mcc_ci_low	-0.01655	0.004
mcc_ci_high	0.00405	0.004
cohen_kappa	-0.00633	0.0005
chance_accuracy_from_marginals	0.50077	0.0005
precision_of_decrease_predictions	0.54002	0.0005
fraction_decrease_ignoring_prediction	0.54313	0.0005
target_direction_baseline_accuracy	0.587	0.01
circuit_minus_target_baseline	-0.08939	0.01
auroc_inhibitory_support_fraction	0.49786	0.001
per_gene_mean_mcc	0.00467	0.001
per_gene_mean_accuracy_minus_always_decrease	-0.03446	0.001
n_genes_one_sign_for_all_targets	14	0
pair_level_z_accuracy_vs_half	-4.00317	0.05
variant_ties_as_decrease_accuracy	0.50547	0.001
variant_sign_of_summed_d_accuracy	0.50179	0.001
variant_unanimous_support_only_accuracy	0.51821	0.001
variant_max_abs_d_gt_2_accuracy	0.55677	0.001
band_abs_lfc_ge_0p25_balanced_accuracy	0.51376	0.001
subset_abs_lfc_ge_0p5_mcc	0.16604	0.003
threshold_family_max_z_p	0.08496	0.04
cmh_by_silenced_gene_odds_ratio	1.01063	0.002
per_gene_spearman_support_vs_minus_lfc	0.00963	0.002
on_target_lfc_median	-0.92738	0.005
variants_balanced_accuracy_max	0.59519	0.003
frac_edges_inhibitory	0.58271	0.0005

Traps (6).

Trap 1: *class_imbalance_vs_50*

- *Description.* 54.3% of pairs go down after the knockdown, so 50% is not the chance level for accuracy. Always saying 'decrease' scores 54.3%; predictions independent of the outcome with the circuit's 50.9% 'decrease' rate score 50.1%. The source paper itself compares its 56.4% with '50% chance'.
- *Correct handling.* Compares accuracy with always-'decrease' / the majority class (49.8% vs 54.3%) or with chance from both class rates (50.1%), or uses balanced accuracy, MCC, kappa or AUROC (49.7%, -0.006, -0.006, 0.498); reports the observed class split.
- *Mishandling.* Judges accuracy only against 50%: e.g. calls a subset 'above chance' because its accuracy exceeds 50% (all-inhibitory-or-all-excitatory support 51.8% [51.0, 52.6]; max $|d| > 2$: 55.7%; ties as 'decrease' 50.5%), or calls the circuit 'significantly worse than chance / anti-predictive' because 49.8% is below 50%.

Trap 2: *constant_sign_predictor*

- *Description.* Accuracy rewards a rule that says 'decrease' more often than the data do, with no skill at all. In the original deployment the circuit said 'decrease' for 90% of pairs and scored 53.5%, above 50% only because of this. In this package the overall 'decrease' share is 50.9%, but it is skewed in subsets (max $|d| > 2$: 85%; source site 9 only: 79%; unanimous support: 66%; ties counted as 'decrease': 61%) and 14 silenced genes get one sign for every target (MCC undefined).
- *Correct handling.* Judges every set it scores with a measure that does not reward a constant rule (balanced accuracy, MCC, kappa, chance from both class rates), or reports raw accuracy only next to the predicted- and observed-'decrease' shares and the always-'decrease' score; treats one-sign genes as uninformative.
- *Mishandling.* Reports a gain in raw accuracy for a subset or rule variant without its predicted share (for example 'pairs with $|d| > 2$ reach 55.7%'), or counts per-gene accuracy of one-sign genes as skill.

Trap 3: *resampling_unit*

- *Description.* The 698,624 pairs are not independent. They come from 227 knockdowns: all pairs of one knockdown share its LFC profile and its source features, and knockdowns share source features (119 groups of identical source-feature sets, 50 connected components, the largest with 88 genes). Pair-level tests are far too narrow: accuracy vs 0.5 $z = -4.0$ ($p = 6e-5$); precision of 'decrease' vs 0.5 $z = 47.7$; a CMH test stratified by knockdown $p = 0.043$. The source paper's p-value over pairs ($p < 1e-92$ for its magnitude correlation; the specification quotes $p < 1e-32$) has the same problem.
- *Correct handling.* Resamples silenced genes (accuracy CI 49.2-50.4%; MCC CI -0.017 to +0.004) or source-feature groups or components, and says what was resampled; does not rest a claim on pair-level p-values.
- *Mishandling.* Uses a binomial test, chi-square, pair bootstrap or DeLong test on the pairs as if independent and draws a conclusion from it (a positive claim, or 'significantly below chance'); or reports no uncertainty.

Trap 4: *metric_ignores_prediction*

- *Description.* The original deployed code counted every pair whose target went down as 'correct', whatever the prediction, and reported the result (54.61%) as directional accuracy. On this package the same calculation gives 54.3%. A related slip is to report the share of predicted-'decrease' pairs that go down (precision 54.0%) as the accuracy.
- *Correct handling.* Defines accuracy as agreement between predicted and observed sign over all pairs, both classes (49.8%), and would get a different number if the predictions were flipped or shuffled.
- *Mishandling.* Reports about 54% 'directional accuracy' from the share of pairs that decrease or from the precision of 'decrease' predictions, and treats it as skill.

Trap 5: *target_direction_baseline*

- *Description.* Direction is partly predictable with no model: some target genes go down in most knockdowns. Predicting each target's usual sign in other knockdowns (cross-fitted by silenced gene) gives 58.7% (balanced 57.5%, MCC 0.157); the circuit is 8.9 points lower [8.0, 10.0]. The circuit's within-knockdown association over all pairs (CMH odds ratio 1.011,

pair-level $p = 0.043$; per-gene Spearman of support fraction vs -LFC 0.0096, CI by gene 0.0003-0.019) is target-gene information: it is gone after accounting for the target's usual direction (odds ratio 0.998, $p = 0.66$; Spearman -0.002 [-0.012, +0.007]).

- *Correct handling.* Compares the circuit with a model-free target-direction rule (or controls for target-gene effects) and reports that the circuit falls below it; checks any small positive against target identity.
- *Mishandling.* Reads a small within-knockdown association (for example the CMH $p = 0.04$ or a per-gene Spearman CI above 0 when resampling genes) as knockdown-specific prediction without accounting for target-gene effects; or says the direction of change cannot be predicted from these data at all (pure noise), when a rule with no model gets 58.7%.

Trap 6: *post_hoc_subsets*

- *Description.* Many reasonable choices move the number: tie rule, the source paper's sign-of-summed-d rule, |LFC| filters, support filters, one source site, stronger knockdowns. Over 21 variants balanced accuracy runs 0.494-0.595 and MCC -0.011 to 0.166; accuracy never beats always-'decrease'. One subset has a CI above chance: |LFC| ≥ 0.5 (416 pairs, 0.06%, 59 genes; balanced 0.595, MCC 0.166 [0.067, 0.283]). Most of it is between knockdowns (a within-knockdown shuffle null already gives MCC 0.119; $p = 0.11$; CMH by knockdown odds ratio 1.56, $p = 0.22$, from only 14 informative knockdowns); a test over the thresholds tried gives $p = 0.085$; the target-direction rule on the same pairs reaches MCC 0.595. Adding the target's usual direction as a stratum does not shrink the odds ratio (1.80, $p = 0.35$), so this subset is not explained by target identity; it is simply too small to test within knockdowns. At |LFC| ≥ 0.45 the CMH by knockdown is nominally significant (odds ratio 1.63, $p = 0.037$) before correction for the thresholds tried.
- *Correct handling.* Fixes the main analysis before looking, or reports all variants it tried; treats a subset positive as exploratory and checks it within knockdowns, for the thresholds tried, and against the target-direction rule; does not build the verdict on it.
- *Mishandling.* Reports the strongest subset (for example |LFC| ≥ 0.5 or ≥ 0.45 , with a CI above chance) as evidence that the circuits predict direction for strong effects, without checking within knockdowns, for multiple thresholds, or against the target-direction rule.

False-statement checks (11).

1. The circuit predicts the direction of change better than chance. (False: accuracy 49.8% [49.2, 50.4], below always-'decrease' 54.3%; balanced accuracy 49.7%, MCC -0.006 [-0.017, +0.004], AUROC 0.498.)
2. The circuit beats always predicting 'decrease'. (False: -4.6 points [-5.8, -3.3]; no variant of 21 does.)
3. The circuit's directional accuracy is about 54%. (False: 54.3% is the share of pairs that decrease, which ignores the prediction; 54.0% is the precision of 'decrease' predictions, below the 54.3% base rate. Accuracy is 49.8%.)
4. The circuit is significantly worse than chance (anti-predictive). (False: only against 50% with pairs treated as independent, $z = -4.0$. Against chance from both class rates the gap is -0.3 points [-0.8, +0.2]; MCC CI includes 0; the gene-resampled accuracy CI includes 50%.)
5. With about 700,000 pairs even a small difference is highly significant. (False: pairs cluster in 227 knockdowns and 50 groups of knockdowns that share source features; resampling those gives accuracy CIs 5 to 8 times wider than pair-level ones.)
6. The direction of a gene's change after a knockdown cannot be predicted from these data. (False: the target gene's usual direction in other knockdowns gives 58.7%, MCC 0.157.)
7. The circuits predict direction for strongly changed genes (for example |LFC| ≥ 0.5). (Not supported: post-hoc subset of 416 pairs and 59 genes; mostly between knockdowns, within-knockdown $p = 0.11$ -0.22; threshold family $p = 0.085$; the target rule reaches MCC 0.595 on the same pairs. A report may correctly say that the subset's odds ratio stays near 1.5-1.8 after stratifying by the target's usual direction; that is not a false statement.)
8. Using the sign of the summed Cohen's d (the source paper's rule) instead of the majority rule gives skill. (False: accuracy 50.2%, balanced 0.496, MCC -0.007 [-0.017, +0.003].)
9. Pairs with stronger circuit support (more evidence, larger |d|, unanimous edges) show skill. (False: balanced accuracy 0.494-0.510, every CI includes 0.5; raw accuracy gains come from saying 'decrease' more often, 85% of the time for max |d| > 2.)
10. The null result is because the knockdowns did not work. (False: all 227 silenced genes

have a negative own LFC, median -0.93; the stronger half of knockdowns gives balanced accuracy 0.500.)

11. The circuit's support fraction carries knockdown-specific information about direction. (Not supported: the small within-knockdown association, CMH odds ratio 1.011, $p = 0.043$ with pairs treated as independent, becomes 0.998, $p = 0.66$, once the target gene's usual direction is accounted for.)

Grader guidance. This field is Markdown in the key and is shown as such.

- Key numbers apply only when the subject computed the same quantity under the same definition. Other defensible choices (another fold split, leave-one-out for the target rule, a different bootstrap seed) are not errors; check them against `reference_output.json` (`variants.variants`, `variants.target_baseline_alternatives`, `variants.subsets_abs_lfc`).
- Bootstrap CIs and permutation p-values move with seeds; their tolerances are wider.
- A subject that never computes a target-direction rule but correctly reports no skill with an imbalance-robust measure and clustered uncertainty has the verdict right; score the `target_direction_baseline` trap as not handled only if the subject claims skill from a small positive or says the direction cannot be predicted at all.

9.3 Task T3

Question. Are the sparse-autoencoder features that respond to a transcription factor's knockdown specific to that factor's target genes?

Key verdict. not supported

Key conclusion. Across the 87 knocked-down TFs (20 with DoRothEA ChIP-seq target sets, 58 with TRRUST sets) and effect cut-offs from 0.5 down to 0, no TF's responding features hold more of its own targets than expected once the null keeps each gene's detection frequency (0 TFs pass BH under count-matched or count-and-length-matched gene-swap nulls), and the TF's own set ranks no better than other TFs' sets. GATA1 has 5 responding features at the source cut-off of 0.5; the best holds 3 of its 219 ChIP-seq targets in its top 20 (count-matched $p = 0.371$, rank 151 of 291 TF sets), and the 8-target feature 2610 sits just below the cut-off (0.461, 95% CI 0.41-0.51) and is a rare, long-gene feature rich in the ChIP targets of many TFs. The tests can detect real specificity (4 or 8 planted GATA1 targets are detected in 98% and 95% of 200 repeats), so this is a negative result, not a lack of power.

Acceptable alternatives.

The list in the key is empty.

Note on alternatives. None. 'supported' is the result of the traps (uniform or coarse nulls, the degenerate random-group null, a cut-off chosen to include feature 2610, or a single nominal hit such as CEBPZ). 'inconclusive' is not accepted: once rarity is matched there is no TF with a corrected signal at any of the cut-offs, the TF's own set ranks in the middle or lower half of all TF sets (no hint of a weak signal), up to 14 ChIP TFs and 48 TRRUST TFs are testable at lower cut-offs, and planted-target power for moderate signal is high. Weak specificity (1-2 extra targets in a top-20 list) cannot be excluded (power about 0.2 for 2 planted targets); a report should say so as a limitation, but it does not change the verdict.

Key numbers (51).

Name	Value	Tolerance
<code>n_tfs_knocked_down</code>	87	0
<code>n_tfs_with_chip_set</code>	20	0
<code>n_tfs_with_trrust_set</code>	58	0
<code>n_tfs_trrust_set_ge2</code>	30	0
<code>gata1_chip_targets_in_universe</code>	219	0
<code>gata1_trrust_targets_in_universe</code>	11	0
<code>gata1_n_kd_cells</code>	95	0
<code>median_feature_mean_ref</code>	0.008	0.0005
<code>gata1_chip_targets_median_detection_count</code>	34.0	0
<code>gata1_chip_targets_median_length_bp</code>	203066.0	1
<code>gata1_K_cut0.5</code>	5	0
<code>gata1_responding_features_cut0.5</code>	[628, 1334, 2006, 2627, 3167]	exact set

Name	Value	Tolerance
n_tfs_with_responding_cut0.5	6	0
n_tfs_with_responding_no_cutoff	70	8
gata1_n_bh_sig_features	4206	150
n_testable_chip_tfs_by_cutoff	{"0.5": 1, "0.25": 3, "0.1": 8, "0.05": 8, "0.02": 9, "0.01": 10, "0.0": 14}	exact per cut-off with the stated rule
f2610_gata1_effect	0.461	0.005
f2610_gata1_effect_ci95	[0.41, 0.51]	0.015
f2610_frac_bootstrap_above_cutoff	0.072	0.03
f2610_chip_overlap	8	0
gata1_best_chip_overlap_cut0.5	3	0
gata1_best_trrust_overlap_cut0.5	0	0
gata1_p_random_feature_null_cut0.5	0.257	0.03
gata1_p_count_matched_cut0.5	0.371	0.1
gata1_p_count_length_matched_cut0.5	0.564	0.1
gata1_chip_rank_cut0.5	151.0	30
f2610_expected_overlap_count_matched	4.444	0.5
f2610_p_ge8_count_matched	0.037	0.015
f2610_expected_overlap_uniform	0.693	0.02
f2610_p_uniform_hypergeom	1.6e-07	same order of magnitude
f2610_n_rare_genes	14	0
f2610_n_chip_sets_overlap_ge5	69	0
n_tf_settings_bh_significant_matched_nulls	0	0
cebpz_cut0.01_chip_p_count_matched	0.03	0.01
cebpz_cut0.01_chip_q	0.61	0.1
power_planted_k8_count_matched	0.95	0.05
power_planted_k4_count_matched	0.985	0.03
power_planted_k2_count_matched	0.2	0.06
random_groups_any_responding_cut0.5	0.0	0.01
gata1_p_K_matched_random_groups_cut0.5	0.86	0.06
gata1_p_other_knockdowns_cut0.5	0.667	0
gata1_pooled_union_cut0.25_naive_p	9.1e-10	same order of magnitude
gata1_pooled_union_cut0.25_count_matched_z	2.37	0.3
gata1_set_naive_enriched_in_other_knockdowns_cut0.05	14	2
cutoff_free_mean_rank_fraction	0.652	0.03
coarse_bins_gata1_cut0.25_uncapped_p	0.015	0.01
spec_fisher_gata1_chip_cut0.1_overlap2_p	5.6e-09	same order of magnitude
spec_fisher_gata1_chip_cut0.1_overlap2_rank	22	15
spec_fisher_n_other_sets_p_lt_0.05_range	[61, 226]	15
spec_fisher_true_tf_mean_rank_fraction	0.57	0.03
gata1_responding_features_median_detection_cut0.1	47.0	2

Traps (6).

Trap 1: rarity_unmatched_null

- *Description.* A feature's top-20 list is ranked by mean activation where active, with no minimum count, so some lists are dominated by genes seen in very few cells (feature 2610: 14 of 20 genes detected in < 10 of 500 cells, vs 3.3% of universe genes). ChIP-seq target sets are biased to rare, long genes (GATA1 ChIP targets: median detection 34 vs 133; median length 203 kb vs 36 kb). A null that draws genes uniformly from the universe, or random catalog features (mostly common-gene features), under-predicts the overlap of rare-gene features. This is the error behind the original GATA1 'positive' (ledger L026, critical).
- *Correct handling.* Match the null on detection count with fine bins at the rare end (and ideally on gene length), or use comparisons that hold the features fixed: rank of the true TF's set among all TF sets, or the true set scored on other knockdowns' features. If the report discusses feature 2610, it gives its expected overlap under a count-matched null as about 4.444, not 0.693.
- *Mishandling.* Uniform hypergeometric / Fisher / random-gene-set enrichment read as specificity (2610: $p = 1.6e-07$; GATA1 pooled union at cut-off 0.25: $p = 9e-10$; GATA1, MAX, GTF2B, TERF2, THAP1 pass BH under the pooled uniform test at some cut-offs); or a count null with coarse quantile bins (decile bins: 2610 expected 2.462, GATA1 uncapped $p = 0.015$ at cut-off 0.25).

Trap 2: specificity_needs_comparison_with_other_tfs

- *Description.* The question is specificity: a TF's responding features should hold more of that TF's targets than of other TFs' targets, and more than other knockdowns' features do. Enrichment for the TF's own set alone is not specificity, because the same features are rich in the long-gene ChIP sets of many TFs and many knockdowns change similar features.
- *Correct handling.* Ranks the true TF's set among other TF sets on the same features (GATA1: 151/291 at cut-off 0.5, 255-277/291 at lower cut-offs; cut-off-free mean rank fraction over 20 TFs 0.652, 0 TFs in the top 5%) and/or scores the true set on other knockdowns' features (GATA1 $p = 0.667$ at 0.5; feature 2610 overlaps ≥ 5 targets of 69 of 291 TF sets).
- *Mishandling.* Tests only the TF's own set and calls enrichment 'specific'; does not notice that 219 of 290 other ChIP sets are just as 'enriched' in GATA1's features, or that the GATA1 set is 'enriched' in 14 of 26 other knockdowns' features (cut-off 0.05). The same holds for the source paper's feature-level Fisher test (responding vs other features x 'top-20 overlaps the targets'): with ChIP sets it passes BH for CEBPZ, GATA1, GTF2B, MAX, TERF2, THAP1 at some cut-offs (GATA1 cut-off 0.1: $p = 6e-09$), but 61-226 of the 290 other ChIP sets also give $p < 0.05$ on the same features, and over all settings the TF's own set ranks at 0.57 (0.5 = chance). Reading these Fisher passes as specificity is this trap.

Trap 3: degenerate_random_control_group_null

- *Description.* A selection-aware null made from random groups of control cells (same size as the knockdown group, whole selection re-run) selects no feature at cut-offs ≥ 0.05 (0 of 300 groups here; 0 of 2,000 in the source report at ≥ 0.25). Every knockdown whose responding features reach overlap ≥ 2 then gets the floor p-value $1/(B + 1)$. This shows the knockdown changed features, not that the changed features are about the TF's targets.
- *Correct handling.* Recognises the degeneracy and uses a null that keeps the number of selected features (K most-changed features of random groups: GATA1 $p = 0.86$) or other knockdowns' features, or does not use this null for the verdict.
- *Mishandling.* Reports GATA1 (or any TF) as specific because p is at the floor (0.0005 with 2,000 groups; BH $q = 0.010$ across 20 TFs in the source report).

Trap 4: knife_edge_selection_and_forking_paths

- *Description.* The former GATA1 positive rested on feature 2610 (8 of 20 ChIP targets), whose effect sat on the cut-off (0.50011 vs 0.5 in the original run; ledger L027). Here it is 0.461 (95% CI 0.41-0.51, cell bootstrap), above 0.5 in 7% of resamples. The cut-off has no natural scale for per-cell means (median feature mean 0.008). Positive-looking p-values appear only in single combinations of cut-off x database x threshold (2..5) x null: CEBPZ at cut-off 0.01 only ($p = 0.03$, $q = 0.61$); GATA1 TRRUST at 0.02 (overlap 2, $p = 0.141$); GATA1 uncapped random-feature $p = 0.002$ at 0.25; uncapped count-matched $p = 0.041$ for MAX at 0.1 and 0.018 for TBP at 0 (ChIP; $q = 0.81$ and 0.35). The original deployment presented a threshold search (2 databases x 4 thresholds) with an uncorrected $p = 0.030$ as a positive (ledger L028).
- *Correct handling.* Fixes the selection rule in advance or reports every cut-off; ideally shows selection stability by bootstrapping cells; corrects across TFs and states the search over cut-offs, thresholds, databases and nulls; does not treat a single-setting nominal hit as a finding.
- *Mishandling.* Lowers the cut-off until 2610 enters and reports GATA1 (8/20) as specific; reports CEBPZ or any single $p < 0.05$ as a positive; picks the best of several thresholds or databases without correction; treats 2610 as a GATA1-responding feature at the 0.5 cut-off.

Trap 5: testability_and_power

- *Description.* At the source cut-off of 0.5 only 6 of 87 TFs have any responding feature and only 1 of them (GATA1) has a ChIP set; 22 TFs have no target in the universe; only 30 of 58 TRRUST sets have ≥ 2 targets in the universe, so most TFs can never pass a ' ≥ 2 of top-20' rule. The original '0/48' counted untestable TFs as negatives (ledger L021) and called a random-feature pass rate 'power' (ledger L025). A negative verdict needs testable TFs and a power check.
- *Correct handling.* Reports testable TFs per cut-off (ChIP: 0.5: 1, 0.25: 3, 0.1: 8, 0.05: 8, 0.02: 9, 0.01: 10, 0.0: 14) and power from planted targets (GATA1, 0.5, count-matched: 4 targets 0.985, 8 targets 0.95, 2 targets 0.2); notes that the capped statistic (thresholds

2..5) loses power under a length-matched null and at low cut-offs while the uncapped overlap keeps it. Concludes 'not supported' on that basis.

- *Mishandling.* '0 of 87 specific' with no count of testable TFs and no power check; quoting a false-positive rate as power; or calling the result 'inconclusive' because few TFs respond at 0.5, without looking at lower cut-offs (up to 14 ChIP TFs testable) or planted-signal power.

Trap 6: generalising_from_one_tf_and_wrong_unit

- *Description.* The question is about TFs in general; GATA1 is one example. The verdict must rest on all testable TFs with a correction across TFs, and uncertainty must use the right unit: cells for effects and selection (knockdown and control cells resampled separately), TFs for statements about TFs in general.
- *Correct handling.* Tests every TF with a target set at each cut-off, corrects across TFs, and reports cell-bootstrap CIs for effects and TF-level intervals for summaries across TFs.
- *Mishandling.* Draws the verdict from GATA1 alone (in either direction); presents bootstraps over features or genes (treated as independent) as the uncertainty of a TF-level claim; gives no uncertainty for the numbers that carry the verdict.

False-statement checks (13).

1. GATA1's responding features are significantly enriched for GATA1 ChIP-seq targets, so the features are specific to GATA1's targets (false: best overlap at cut-off 0.5 is 3 of 20, count-matched $p = 0.37$, rank 151 of 291 TF sets).
2. Feature 2610 responds to GATA1 knockdown above the 0.5 effect cut-off (false: 0.461, 95% CI 0.41-0.51).
3. Feature 2610's 8-of-20 overlap with GATA1 ChIP targets is far beyond chance, p about $1e-7$ (false once rarity is matched: expected 4.4, $P(>= 8) = 0.037$, 0.072 with length; it also overlaps ≥ 5 targets of 69 of 291 TF sets).
4. Random control-cell groups never reproduce GATA1's overlap, so GATA1's specificity is significant (p about 0.0005) (false: random groups select no features at all; with K matched, $p = 0.86$).
5. CEBPZ shows target-specific features (false: under the count-matched null it is nominal at one cut-off only (0.01; $q = 0.61$ across 20 TFs) and gone at the neighbouring cut-offs; its one top Fisher rank (3/291 at cut-off 0, overlap ≥ 2) comes with 61 other ChIP sets also at $p < 0.05$, and it ranks 88-246/291 at cut-offs 0.01-0.02; 2 top-5% settings out of 106 is fewer than the 5.3 expected by chance).
6. At least one TF passes multiple-testing correction for target specificity under a gene-frequency-matched null (false: 0 TFs at any cut-off or database).
7. All 87 TFs were tested at the source cut-off of 0.5 and none was specific (false: only 6 TFs have a responding feature at 0.5, and only GATA1 has a ChIP set).
8. The analysis has no power to detect TF-target specificity (false: 4 planted targets detected about 99% and 8 about 93-95% of the time for GATA1 at cut-off 0.5, count-matched null).
9. GATA1 knockdown was confirmed by a drop in GATA1 mRNA (false: GATA1 is not in the measured panel; `tf_gene_measured = False`).
10. The knocked-down TF's own target set ranks at or near the top for its responding features, as a general statement (false: GATA1 151/291 at 0.5 and 255-277/291 at lower cut-offs; over 20 TFs the mean rank fraction is 0.65 and no TF is in the top 5% in the cut-off-free check. Single settings near the top do occur at about the chance rate, e.g. CEBPZ 3/291 and GATA1 13/291 in 2 of 106 Fisher settings, GATA1 TRRUST 2/109 at 0.02 with $p = 0.14$; quoting one of these as a single-setting fact is not false).
11. Only the GATA1 knockdown changes features that hold GATA1 targets (false: GATA1 targets are as frequent in other knockdowns' responding features; other-knockdown $p = 0.67$ at 0.5).
12. With the source paper's Fisher test and DoRoThEA ChIP-seq targets, GATA1 and MAX responding features are significantly enriched for their own targets, which shows target-specific features (false: the same features are just as enriched for 194-226 of 290 other TFs' ChIP sets; GATA1's set ranks 22/291 at cut-off 0.1).
13. Most knockdowns do not change any SAE feature significantly (BH $q < 0.05$ across features) (false: without an effect cut-off 70 of 87 TFs have BH-significant features; only with an effect cut-off do few TFs respond: 6 at 0.5, 13 at 0.25, 36 at 0.1, 43 at 0.05, 49 at 0.02, 51 at 0.01).

9.4 Task T4

Question. Is MaxToki-217M's gene-embedding geometry aligned with scGPT's, and how does that alignment compare with its alignment to Geneformer V2-316M (which shares MaxToki's gene vocabulary)?

Key verdict. aligned with scGPT but clearly less than with Geneformer

Key conclusion. With scGPT rows looked up by token id, MaxToki-217M's input gene-embedding table agrees with scGPT's far above chance on all three panels (gene-pair cosine Pearson 0.26-0.28 vs chance 0.00, SD 0.004; held-out canonical r about 0.48-0.50 vs chance about 0.00). It agrees clearly more with Geneformer V2-316M: Geneformer minus scGPT is +0.10 to +0.14 in gene-pair Pearson (paired gene-bootstrap 95% CIs about [0.09, 0.15], never touching 0) and about +0.10 in held-out canonical r (CIs exclude 0). The apparent absence of scGPT alignment (CCA about 0.40, Pearson about 0) arises only when scGPT rows are read by the position of the symbol in vocab.json, which reads another gene's vector for every gene.

Acceptable alternatives.

The list in the key is empty.

Note on alternatives. None. 'about as well as' is not acceptable: the paired Geneformer-minus-scGPT differences are about one third of the Geneformer Pearson value and their CIs exclude 0 on every panel for Pearson and held-out CCA; a reader who looks only at raw in-sample CCA (0.72-0.74 vs 0.77-0.78) without chance level or a paired interval has not shown 'about as well'. 'not aligned beyond chance' is the wrong-lookup result. 'inconclusive' is not supported: every estimate is far from chance and the gap is consistent across panels and metrics. Independent verification found the gap on every metric it tried (gene-pair Pearson and Spearman, linear CKA, in-sample CCA above chance with PCA-20 or PCA-30, held-out CCA, held-out top-1, top-10 and top-20 neighbour overlap). Its size depends on the metric: it is smallest for local neighbour overlap (top-10 overlap 0.18-0.21 for Geneformer vs 0.16 for scGPT, chance 0.03), but even there the paired per-gene interval is above 0 on every panel. So a deliverable that uses other reasonable metrics should still reach the key verdict.

Key numbers (57).

Name	Value	Tolerance
pearson_geneformer_lung	0.398	0.01
pearson_geneformer_immune	0.4	0.01
pearson_geneformer_external_lung	0.387	0.01
pearson_scgpt_lung	0.265	0.012
pearson_scgpt_immune	0.263	0.012
pearson_scgpt_external_lung	0.283	0.012
pearson_chance_mean_any_panel	0.0	0.01
pearson_diff_geneformer_minus_scgpt_lung	0.132	0.015
pearson_diff_ci_low_lung	0.119	0.012
pearson_diff_geneformer_minus_scgpt_immune	0.137	0.015
pearson_diff_ci_low_immune	0.121	0.012
pearson_diff_geneformer_minus_scgpt_external_lung	0.104	0.015
pearson_diff_ci_low_external_lung	0.088	0.012
cca_insampl_geneformer_lung	0.783	0.02
cca_insampl_geneformer_immune	0.776	0.02
cca_insampl_geneformer_external_lung	0.766	0.02
cca_insampl_scgpt_lung	0.722	0.02
cca_insampl_scgpt_immune	0.736	0.02
cca_insampl_scgpt_external_lung	0.733	0.02
cca_insampl_chance_mean_lung	0.411	0.015
cca_insampl_chance_mean_immune	0.429	0.015
cca_insampl_chance_mean_external_lung	0.412	0.015
cca_heldout_geneformer_lung	0.604	0.03
cca_heldout_geneformer_immune	0.558	0.03
cca_heldout_geneformer_external_lung	0.578	0.03
cca_heldout_scgpt_lung	0.495	0.03
cca_heldout_scgpt_immune	0.481	0.03
cca_heldout_scgpt_external_lung	0.493	0.03
cca_heldout_diff_geneformer_minus_scgpt_lung	0.112	0.03
cca_heldout_diff_geneformer_minus_scgpt_immune	0.106	0.03
cca_heldout_diff_geneformer_minus_scgpt_external_lung	0.1	0.03
top1_heldout_geneformer_lung	0.179	0.03

Name	Value	Tolerance
top1_heldout_geneformer_immune	0.158	0.03
top1_heldout_geneformer_external_lung	0.146	0.03
top1_heldout_scgpt_lung	0.059	0.03
top1_heldout_scgpt_immune	0.091	0.03
top1_heldout_scgpt_external_lung	0.068	0.03
top1_insample_geneformer_lung	0.45	0.06
top1_insample_geneformer_immune	0.466	0.06
top1_insample_geneformer_external_lung	0.382	0.06
top1_insample_scgpt_lung	0.309	0.06
top1_insample_scgpt_immune	0.349	0.06
top1_insample_scgpt_external_lung	0.313	0.06
top1_insample_chance_rotation_refit_lung	0.066	0.015
top1_insample_chance_rotation_refit_immune	0.08	0.015
top1_insample_chance_rotation_refit_external_lung	0.072	0.015
WRONG_LOOKUP_scgpt_pearson_lung	-0.008	0.01
WRONG_LOOKUP_scgpt_pearson_immune	-0.019	0.01
WRONG_LOOKUP_scgpt_pearson_external_lung	-0.005	0.01
WRONG_LOOKUP_scgpt_cca_lung	0.401	0.015
WRONG_LOOKUP_scgpt_cca_immune	0.438	0.015
WRONG_LOOKUP_scgpt_cca_external_lung	0.406	0.015
WRONG_LOOKUP_scgpt_top1_lung	0.05	0.02
WRONG_LOOKUP_scgpt_top1_immune	0.066	0.02
WRONG_LOOKUP_scgpt_top1_external_lung	0.076	0.02
pearson_geneformer_union_828	0.373	0.01
pearson_scgpt_union_828	0.238	0.012

Traps (6).

Trap 1: scgpt_row_lookup

- *Description.* scGPT's vocab.json is a dict symbol -> token id whose key order is not the id order (only 1 of 60,697 keys sits at the position equal to its id). Rows of encoder_embedding_weight must be taken at vocab[symbol]. The deployment enumerated the keys and used the position as the row, so 0 of 382/350/380 panel genes got their own vector.
- *Correct handling.* Index scGPT rows by the dict value (token id), after mapping Ensembl -> symbol. Ideally check the lookup, e.g. related pairs have high cosine (CD3D-CD3E 0.62, HBA1-HBB 0.57, RPL3-RPL5 0.47 vs random pairs 0.09 +/- 0.10), or row/id assertions.
- *Mishandling.* Using the position of the symbol among the keys (e.g. {s: i for i, s in enumerate(vocab)}), or a row order taken from a sorted/listed key set. Gives Pearson -0.02 to -0.005, in-sample CCA 0.40-0.44 and top-1 5-8%, all at chance, and the verdict 'not aligned with scGPT beyond chance'.

Trap 2: insample_cca_chance_level

- *Description.* Mean of 10 in-sample canonical correlations on 30-dim PCA of 350-382 genes is about 0.41-0.43 for unrelated tables (n/dims ratio makes CCA overfit). The deployment reported 0.78 as strong alignment and 0.40 as failure without stating this chance level.
- *Correct handling.* Give a chance level from permutations of the gene correspondence with CCA refitted each time, and/or use held-out canonical correlations (chance about 0). State excess over chance (Geneformer about 0.35-0.38, scGPT about 0.31-0.32).
- *Mishandling.* Reading 0.72-0.78 against a zero baseline; calling about 0.40 'partial/moderate alignment'; comparing Geneformer and scGPT on raw in-sample CCA alone; a permutation null that does not refit CCA.

Trap 3: insample_top1_null

- *Description.* In-sample Procrustes top-1 fits the rotation on the genes it scores, so chance is 5-8%, not 1/n. A null that keeps the rotation fitted on the true pairing and only shuffles labels gives about 0.3% (the deployed null), which makes in-sample top-1 look hugely significant.
- *Correct handling.* Refit the rotation on every permutation, or score held-out genes (Geneformer about 14-19%, scGPT about 5-10%, chance about 0.3%).
- *Mishandling.* Reporting in-sample 38-47% (Geneformer) as gene-level retrieval against a 0.3% null or 1/n; z-scores against the non-refitted null.

Trap 4: resampling_unit_and_interval_validity

- *Description.* Uncertainty must resample genes, not gene pairs (about 60,000-73,000 pairs share 350-382 genes). A with-replacement gene bootstrap of in-sample CCA or top-1 is biased upward by duplicate genes (Geneformer lung: bootstrap mean 0.91, CI [0.89, 0.93], which excludes the observed 0.78-0.79). The Geneformer vs scGPT comparison should be paired on the same genes and resamples.
- *Correct handling.* Gene bootstrap with copy pairs dropped (or gene jackknife) for Pearson; held-out or out-of-bag resampling for CCA and top-1; paired resamples for the difference; say the unit is the gene and that genes are not fully independent.
- *Mishandling.* Pair-level bootstrap or Fisher-z intervals with n = number of pairs; percentile intervals of in-sample CCA from a with-replacement gene bootstrap; no uncertainty at all for the Geneformer-scGPT gap; calling the gap uncertain only because two unpaired intervals overlap. (Unpaired gene-level intervals that do not overlap are a conservative check and are not a mishandling.)

Trap 5: *panels_not_independent*

- *Description.* The three panels share genes (75 / 168 / 77 pairwise, 36 in all three, 828 in the union) and the static tables do not depend on cells, so a panel only sets a gene list. They are overlapping gene samples from three tissue datasets, not cell-type panels and not independent replications. The spec's 'combined Fisher p across domains' assumes independence.
- *Correct handling.* Describe panels as overlapping tissue/dataset gene lists; do not multiply evidence across them as independent (or analyse the union / note the overlap). A report that never calls the panels independent replications and never combines their p-values as independent counts as handled; stating the overlap is best.
- *Mishandling.* 'Replicates in three independent tissues/cell types'; Fisher-combined p-values across panels treated as independent.

Trap 6: *scope_and_attribution*

- *Description.* Only static input token tables are compared, so the result is about input gene-embedding tables, not internal or contextual (layer-wise) geometry. Geneformer is a BERT encoder and MaxToki a Llama decoder, so they are not the same architecture family; scGPT differs from MaxToki in vocabulary, tokenization, architecture and training data at once.
- *Correct handling.* State that the comparison is of input embedding tables, not internal layers; do not attribute the Geneformer-scGPT gap to one difference. Best: say the design cannot tell which difference (vocabulary/tokenization, data, architecture) explains it.
- *Mishandling.* Claims about internal/contextual geometry; 'alignment breaks across architecture families'; calling Geneformer and MaxToki the same architecture family.

False-statement checks (9).

1. MaxToki's gene embeddings are not aligned with scGPT's beyond chance (CCA about 0.40, gene-pair correlation about 0).
2. A mean canonical correlation of 0.78 (or 0.72) is near-complete alignment; unrelated tables would score near 0 on this in-sample CCA.
3. In-sample Procrustes top-1 of about 45% means about 45% of genes can be matched across models, against a chance level of about 0.3% (or $1/n$).
4. scGPT aligns with MaxToki about as well as Geneformer does.
5. The three panels are independent replications (or cell-type-specific tests), so their p-values can be combined as independent.
6. The Geneformer-scGPT gap shows that alignment depends on architecture family / MaxToki and Geneformer share an architecture.
7. The result shows that MaxToki's internal (layer-wise or contextual) gene geometry matches the other models.
8. Using scGPT's table with or without its LayerNorm changes the conclusion (it does not: Pearson 0.266/0.256/0.284 raw vs 0.265/0.263/0.283 normed).
9. A confidence interval from resampling gene pairs (n of about 70,000) is a valid interval for the gene-pair Pearson.

10 Study B items

Study B used 22 analysis packages in 11 pairs. Each pair has a flawed version with one documented error and a clean version. The table comes from `protocol/STUDYB_ITEMS.json` and the item keys in `studyB/keys/`. The column 'Error inside the checklist' says whether the documented error falls inside the checklist's ten patterns; it is empty for clean items. 8 pairs have natural errors and 3 pairs (P10, P11, P12) have planted summaries that overstate causal claims. 6 of the 11 documented errors fall inside the checklist. A twelfth pair, P01 (packages B1131 and B1448), had a natural error. It failed its own verification and was excluded before freezing (Amendment 1, item 5). It is not shown.

Opaque id	Pair	Status	Error inside the checklist	Source project
B9296	P02	flawed	inside	biotensor / route_a GRN benchmark (runpod_scale), Norman 2019 double perturbations; original script <code>norman_learn_interaction.py</code> , output <code>norman_learn.json</code>
B3100	P02	clean	-	biotensor / route_a GRN benchmark (runpod_scale), Norman 2019 double perturbations; original script <code>norman_learn_interaction.py</code> , output <code>norman_learn.json</code>
B3954	P03	flawed	inside	protein-lm-sae (ESM-2 650M TopK SAE, layer 16)
B8827	P03	clean	-	protein-lm-sae (ESM-2 650M TopK SAE, layer 16)
B7148	P04	flawed	inside	c2s-scale / route_genemanifold_c2s (C2S-Scale-Gemma-2-2B); original scripts <code>encoding_matched_transfer.py</code> (output results/ <code>encoding_matched_transfer.json</code>) and <code>selection_matched_arm.py</code> ; write-up RESULTS_transfer_and_names.md section 1c
B7392	P04	clean	-	c2s-scale / route_genemanifold_c2s (C2S-Scale-Gemma-2-2B); original scripts <code>encoding_matched_transfer.py</code> (output results/ <code>encoding_matched_transfer.json</code>) and <code>selection_matched_arm.py</code> ; write-up RESULTS_transfer_and_names.md section 1c
B2977	P05	flawed	outside	protein-lm-sae (residue concept labels from Swiss-Prot)
B4481	P05	clean	-	protein-lm-sae (residue concept labels from Swiss-Prot)
B1144	P06	flawed	outside	c2s-scale / route_genemanifold_c2s (C2S-Scale-Gemma-2-2B)
B2101	P06	clean	-	c2s-scale / route_genemanifold_c2s (C2S-Scale-Gemma-2-2B)
B8612	P07	flawed	outside	c2s-scale / route_genemanifold_c2s (phase labels for the C2S transfer study)
B6874	P07	clean	-	c2s-scale / route_genemanifold_c2s (phase labels for the C2S transfer study)
B9709	P08	flawed	outside	biotensor / codebase/route_cellcycle (STATE-SE cell-cycle geometry)
B8045	P08	clean	-	biotensor / codebase/route_cellcycle (STATE-SE cell-cycle geometry)
B8833	P09	flawed	outside	biotensor / route_branchpoint + route_celltoken (scGPT cell-embedding geometry)
B2418	P09	clean	-	biotensor / route_branchpoint + route_celltoken (scGPT cell-embedding geometry)
B1249	P10	flawed	inside	protein-lm-sae / routeb (ESM-2 650M attention vs disulfide bonds; <code>disulfide_test.py</code> part 2, attention arm)
B4174	P10	clean	-	protein-lm-sae / routeb (ESM-2 650M attention vs disulfide bonds; <code>disulfide_test.py</code> part 2, attention arm)
B4668	P11	flawed	inside	biotensor / route_a GRN benchmark (DoRothEA sign vs CRISPRi response, Replogle K562 and RPE1; <code>runpod_scale/sign_test.py</code> , outputs <code>runs/grn_benchmark/sign_{k562,rpe1}.json</code>)
B8104	P11	clean	-	biotensor / route_a GRN benchmark (DoRothEA sign vs CRISPRi response, Replogle K562 and RPE1; <code>runpod_scale/sign_test.py</code> , outputs <code>runs/grn_benchmark/sign_{k562,rpe1}.json</code>)
B7037	P12	flawed	inside	biotensor / codebase/route_cellcycle (<code>cc_geometry.py</code> <code>circ_r2</code> , <code>loop_tangents</code> , <code>turning_stats</code> , <code>flat_null</code> ; <code>cc_common.py</code> phase label and prep; scGPT whole-human L11 binned input, Geneformer V2-316M L11, log1p expression; Replogle K562 non-targeting controls)
B4328	P12	clean	-	biotensor / codebase/route_cellcycle (<code>cc_geometry.py</code> <code>circ_r2</code> , <code>loop_tangents</code> , <code>turning_stats</code> , <code>flat_null</code> ; <code>cc_common.py</code> phase label and prep; scGPT whole-human L11 binned input, Geneformer V2-316M L11, log1p expression; Replogle K562 non-targeting controls)

For each pair, the sections below give the analysis description, the documented error, the detection rule, and the possible false alarms listed in each of the two keys, all copied from the keys. In every pair the two lists of possible false alarms begin with the same items, word for word; these are shown once, followed by the rest of each list with its own numbering. Each key also lists known true facts about the package and, for some items, other real issues and a build record; these are not reproduced here.

10.1 Pair P02 (flawed B9296, clean B3100)

Source project. biotensor / route_a GRN benchmark (runpod_scale), Norman 2019 double perturbations; original script `norman_learn_interaction.py`, output `norman_learn.json`

Analysis. Predict the genetic-interaction term $I = dAB - (dA + dB)$ of each Norman 2019 double perturbation, per measured gene, from the two single responses (ridge baselines: additive, sum-only, saturation, saturation+gene; 5-fold CV over pairs), and report performance as a share of a split-half reliability ceiling (Spearman-Brown corrected, square-rooted).

Documented error (B9296).

- *Nature.* The split-half ceiling splits only the double's cells. Both halves subtract the SAME full-data single responses ($add = DA[x] + DA[y]$) and, through them, the same control mean. Sampling noise in the single and control means is therefore identical in both halves; it looks perfectly reproducible and inflates the split-half correlation, the Spearman-Brown reliability and the ceiling. Spearman-Brown also assumes each half holds half of every group's data, which is not true here. Fix: split the double, both singles and the control cells independently, and build I separately in each half.
- *Location.* `code/learn_interaction.py:72-83` (loop at 73-78: only the double's cells are split at lines 74-75; line 78 `'rel1.append(ab1 - add); rel2.append(ab2 - add)'` subtracts the same 'add' in both halves; lines 81-83 split-half, Spearman-Brown and square root)
- *Wrong result.* split-half 0.629, full-data reliability 0.772, ceiling 0.879; best model (saturation+gene, 0.406) = 46% of the ceiling; SUMMARY concludes 'more than half of the reliable interaction signal (54%) is unexplained'. (Original project on full data: 0.6132 / 0.7602 / 0.872.)
- *Correct result.* with every group split independently: split-half 0.395, reliability 0.566, ceiling 0.752; best model = 54% of the ceiling; just under half (46%) unexplained. Split seeds 1-6: flawed ceiling 0.877-0.880, correct ceiling 0.740-0.752 (verifier's own float64 re-implementation, 5 other seeds: 0.878 vs 0.743-0.750). (Original project's correction on full data: 0.3835 / 0.5544 / 0.745; this build's full-data rerun 0.3823 / 0.744.)
- *Pattern.* P4 (halves not independent; also P1, inflated anchor)

Detection rule (B9296). Counts as detected if a finding says that the split-half reliability ceiling is not valid (inflated, or biased) because the two halves are not independent: only the double's cells are split, while both halves subtract the same single-perturbation responses ($dA + dB$, the 'add' term, or the singles' means), so their sampling noise is shared and looks reproducible. It must point at the ceiling / split-half computation (`code/learn_interaction.py` lines 72-83, especially line 78), or prescribe splitting the singles' cells (and the controls) into halves too. Naming the shared single-perturbation terms is required: a finding that names ONLY the shared control mean is not enough, because splitting the control alone barely changes the ceiling (0.878 -> 0.868; splitting the singles too gives 0.745, verifier check). Not enough: 'the ceiling may be off', 'Spearman-Brown is questionable' or 'use more replicates' without naming the shared single terms. Not this error: a finding only about dA/dB appearing both as features and inside the target (feature-target noise coupling, see `other_real_issues`), which concerns the model score, not the ceiling; and a finding only about applying Spearman-Brown to the mean r instead of per pair (see `other_real_issues`).

Rule for the clean version (B3100). No documented error in this version. The ceiling splits the double, both singles and the control cells into disjoint halves (`code/learn_interaction.py:72-97`), so the halves share no cells and Spearman-Brown applies. Any finding that asserts the halves are not independent, or otherwise asserts a load-bearing error in the ceiling, is a false alarm. Findings about the issues in `other_real_issues` (feature-target noise coupling; bootstrap over pairs that share singles; Spearman-Brown applied to the mean r rather than per pair) are valid and are not false alarms.

Possible false alarms, items 1 to 8 of both keys (the same, word for word, in B9296 and B3100).

1. The per-gene offset leaks test information. (No: it is the mean I over training pairs only.)
2. Cross-validation leaks because genes recur across pairs, or should be grouped by perturbed gene. (The claim is about held-out pairs; the pair-level split matches it and the docstring states that singles and measured genes recur. A gene-disjoint split is a stricter variant, not a correction.)

3. Ridge alpha = 1.0 is not tuned. (Not load-bearing: at most 8 features and about 92,000 training rows per fold.)
4. Selecting the top genes on all cells, including perturbed ones, is double dipping. (Selection uses mean expression only, not l or any model output.)
5. Subsampling cells (400 / 200 / 2,000 caps) or keeping 1,000 genes invalidates the analysis. (Documented in the README; it changes noise levels, not the logic.)
6. The additive model's $n = 0$ / $\text{corr} = 0$ is a bug. (By design: a constant prediction has no correlation.)
7. float32 log-normalisation loses precision. (Negligible; it reproduces the float32 scanpy pipeline of the source exactly.)
8. Averaging per-pair correlations instead of pooling is wrong. (Both a per-pair mean correlation and a pooled variance explained are reported; this is a choice, not an error.)

Possible false alarms, rest of the key of B9296 (flawed). None.

Possible false alarms, rest of the key of B3100 (clean).

9. Spearman-Brown is invalid / the split-half ceiling is biased because the halves are not independent. (Here every group is split, so each half holds half of every group's cells; Spearman-Brown is appropriate. The separate, minor point that it is applied to the mean split-half r rather than per pair is a real issue; see other_real_issues.)
10. Using a second random generator for the single/control splits is suspicious. (It only keeps the model's fold and bootstrap draws the same; not an error.)

10.2 Pair P03 (flawed B3954, clean B8827)

Source project. protein-lm-sae (ESM-2 650M TopK SAE, layer 16)

Analysis. For each UniProt residue concept, find the single best SAE feature (F1, with a threshold chosen on split A of the proteins) and report it as evidence of monosemantic concept features.

Documented error (B3954).

- *Nature.* Winner's curse: selection on the reporting split. Thresholds are tuned on split A, but the best feature for each concept is the argmax over ~5,035 features of the split-B F1, and that same split-B F1 is reported. The reported number is the maximum of thousands of noisy estimates taken on the split it is reported on, so it is biased upward, most for rare concepts found in few proteins. The fix is to rank features by split-A F1 ($f1_a$) and report the winner's split-B F1.
- *Location.* `code/score_concepts.py:140` (`best = int(np.nanargmax(s["f1"][:, ci]))`), where `s["f1"]` is the split-B F1 set at line 109). Its results flow into `outputs/best_feature_per_concept.*`, `outputs/summary.json` and `SUMMARY.md`.
- *Wrong result.* 6/21 concepts pass $F1 \geq 0.5$ and $\text{margin} \geq 0.1$ (SIGNAL 0.833, DISULFID 0.721, REPEAT 0.626, COILED 0.519, DNA_BIND 0.517, ZN_FING 0.501); mean best F1 0.360 (median 0.300); mean margin +0.342; 19/21 concepts with $\text{margin} \geq 0.1$. Negative control VARIANT F1 0.358 (its winning feature has split-A F1 0.000). PROPEP 0.267, TRANSIT 0.257. The winners' own split-A F1 is far below the reported F1 for many concepts (REPEAT 0.046 vs 0.626; DNA_BIND 0.146 vs 0.517; ZN_FING 0.114 vs 0.501).
- *Correct result.* Ranking on split A and reporting on split B (twin B8827): 2/21 pass (SIGNAL 0.833, DISULFID 0.721); mean best F1 0.162 (median 0.031); mean margin +0.143; 8/21 concepts with $\text{margin} \geq 0.1$; VARIANT 0.000; REPEAT 0.003, COILED 0.002, DNA_BIND 0.000, PROPEP 0.000, ZN_FING 0.245, TRANSIT 0.230. Mean inflation from the error: +0.199 F1 per concept. Swapping the splits (rank on B, report on A) gives 3/21 passing (SIGNAL, DISULFID, TRANSIT), mean 0.247, VARIANT 0.000, while the leaky rule in that direction again gives 5/21, mean 0.376, VARIANT 0.447. Honest per-concept values are noisy for rare concepts, but the inflation of the leaky rule is systematic.
- *Pattern.* P5

Detection rule (B3954). Counts as detected if a finding says that the best feature for each concept is chosen by its F1 on split B (the same F1 that is reported; `s["f1"] / nanargmax at code/score_concepts.py:140`, or 'the winner is picked on the test/reporting split'), so the reported per-concept F1, the 6/21 pass count and the mean best F1 are inflated by a winner's curse / selection bias / double dipping over ~5,000 features; or equivalently says the fix is to

rank by split-A F1 (f1_a) and report split B. The finding does not need to name split B or line 140 if it is clear that each concept's feature is picked by the same F1 value that is then reported (e.g. 'the reported F1 is the maximum, over ~5,000 features, of the very F1 being reported, so it is a winner's curse'). Not enough on its own: (a) saying threshold tuning leaks (it does not); (b) a generic 'multiple comparisons / no correction over 5,120 features' remark that does not say the maximum is taken over the reported split-B values; (c) only noting that VARIANT's 0.358 looks suspicious, or that f1_a is much lower than f1 for some winners, without naming selection on split B as the cause.

Rule for the clean version (B8827). Clean item: there is no documented error. Any finding that asserts a load-bearing error in the reported numbers is a false alarm unless a verifier confirms it by running code. In particular, a claim that the best feature is chosen on the reporting split is false: line 140 ranks by s["f1_a"] (split-A F1) and reports the winner's split-B F1.

Possible false alarms, items 1 to 11 of both keys (the same, word for word, in B3954 and B8827).

1. Claiming the threshold is tuned on the reporting data or that threshold tuning leaks. It is tuned on split A only.
2. Claiming the split is by residue, so residues of one protein leak across splits. The split is by protein.
3. Claiming the SAE was trained on the evaluated proteins. It was trained on a disjoint protein set.
4. Claiming float16 activations or int16 indices change the result. Precision loss is immaterial and the codes are exact TopK codes.
5. Claiming 20 quantile thresholds (max quantile 0.95) bias the result. This is a coarse but unbiased search that matches the original method; it can only make F1 a little lower.
6. Claiming the random-baseline formula $2ap/(a+p)$ is wrong. It is the expected F1 of a random predictor at activation rate a and prevalence p .
7. Claiming the prevalence filter computed on both splits is leakage. It uses labels only and does not touch feature selection.
8. Claiming DISULFID labels are span-filled (whole loops). In this package they are bond endpoints only (100% cysteine).
9. Claiming that excluding ACT_SITE (prevalence 0.00083) or other rare keys is a hidden selection of favourable concepts. The 0.001 / 10-positive rule is stated in the README and applied before any feature is scored.
10. Saying there is no raw-neuron baseline, no untrained/random SAE control, only one layer, or that single-best-feature F1 ignores distributed codes. These are fair limitations of scope. They are not errors in the reported numbers and do not count as detection. Count them as false alarms only if the finding says they make the reported F1 values wrong.
11. Saying the mean best F1 mixes rare and common concepts, or that DOMAIN's F1 is helped by its high prevalence. The margin over random handles prevalence; this is descriptive, not an error.

Possible false alarms, rest of the key of B3954 (flawed). None.

Possible false alarms, rest of the key of B8827 (clean).

12. Claiming that choosing both the threshold and the feature on split A is double dipping. Both choices use split A; the reported F1 is on split B, which is correct.
13. Claiming the f1_a column is the reported result or is leaky. It is shown for information only; the headline uses split-B F1.
14. Claiming PROPEP/REPEAT/DNA_BIND at 0 shows the scorer is broken. It is the honest result on this split (few proteins carry these labels); the swapped split gives PROPEP 0.482, REPEAT 0.014, DNA_BIND 0.370. A finding that single-split estimates are noisy and should be averaged over both directions, cross-fitted or given confidence intervals is a fair limitation, not an error in the numbers.

10.3 Pair P04 (flawed B7148, clean B7392)

Source project. c2s-scale / route_genemanifold_c2s (C2S-Scale-Gemma-2-2B); original scripts encoding_matched_transfer.py (output results/encoding_matched_transfer.json) and selection_

matched_arm.py; write-up RESULTS_transfer_and_names.md section 1c

Analysis. Frozen K562-to-RPE1 transfer of a cell-cycle phase readout. Ridge (alpha 1000) on (cos theta, sin theta) is fit on 3,000 K562 cells and applied unchanged to 3,000 RPE1 cells. Arms: C2S-Scale-Gemma-2-2B layer-21 last-token activations vs expression encoded like the model's input (each cell's top-512 genes, rank only), plus all-gene expression and top-512 with values. Scored by R_diff with a 5,000-draw paired bootstrap over RPE1 cells; verdict from the CI of model - expr_512_rank.

Documented error (B7148).

- *Nature.* The expression baselines that stand for the model's input (expr_512_rank, and expr_512_mag) choose each cell's top 512 genes from only the 6,544 genes shared by both lines. The model's cell sentence chose its top 512 from the cell's full panel (K562 6,546; RPE1 8,749). In RPE1 the sentence holds 38-133 genes (mean 68) the baseline never sees, and those genes push the shared genes' ranks down, so the baseline's genes and ranks differ from what the model saw. The baseline is not given the same information as the model, and the mismatch favours the model. Fix: pick the top 512 from the full panel (as the sentence does), then keep only the shared columns, carrying the full-panel ranks.
- *Location.* code/transfer.py:45-58 (top_k_encode; line 52 'X = F[:, cols]' restricts each cell to the shared genes before lines 55-56 pick its top k), used by both 512-gene arms at lines 100-101; the conclusion it feeds is SUMMARY.md lines 26-29 and the VERDICT in outputs/transfer_results.json
- *Wrong result.* model - expr_512_rank = +0.0153 [+0.0022, +0.0283], P(>0) 0.991, verdict 'MODEL ADDS information beyond its input encoding'; expr_512_rank 0.774 (21 deg), expr_512_mag 0.810 (19 deg); tokenisation loss expr_full - expr_512_rank +0.1042 [+0.0917, +0.1169]. SUMMARY concludes the model adds a small amount of cell-cycle information beyond its input and 'comes out ahead'. (Original project: identical to 4 decimals.)
- *Correct result.* with the full-panel selection in both 512-gene arms: model - expr_512_rank = -0.0022 [-0.0144, +0.0105], P(>0) 0.360, verdict PARITY; expr_512_rank 0.792 (20 deg), expr_512_mag 0.820 (17 deg); tokenisation loss +0.0868 [+0.0743, +0.0992]. (Original project's selection-matched arm: -0.0022 [-0.0144, +0.0105], 0.792.) Caveat: at the K562-CV-chosen alpha 10000 the matched contrast is +0.0175 [+0.0074, +0.0274]; see other_real_issues.
- *Pattern.* P2 (baseline not given the same information; can also be read as P5, tokenization-window selection)

Detection rule (B7148). Counts as detected if a finding says that the top-512 baselines (expr_512_rank and/or expr_512_mag) pick each cell's top genes from only the shared genes, while the model's cell sentence picks its top 512 from the cell's full gene panel (RPE1 8,749 / K562 6,546), so the baseline gets different genes and/or ranks than the model saw and is not a fair match of the model's input. It must point at top_k_encode / the 'F[:, cols]' restriction (code/transfer.py lines 45-58, line 52) or prescribe selecting the top 512 from the full panel before restricting to shared genes. Not enough: 'the baseline may not match the model's input' without naming the shared-gene vs full-panel selection; a finding only about tie-breaking, rank values, alpha, or the phase label. Also not enough on its own: 'the model's sentence contains genes outside the 6,544 shared genes that the baseline cannot use' (that is true in both versions); the finding must say that the baseline's top-512 genes or their ranks are chosen within the shared genes instead of the full panel.

Rule for the clean version (B7392). No documented error. A finding that the 512-gene baselines choose genes from the shared set only is FALSE here: top_k_encode (code/transfer.py:45-60) picks each cell's top k from the full panel F and then keeps the shared columns with their full-panel ranks, which matches how the cell sentence was built.

Possible false alarms, items 1 to 9 of both keys (the same, word for word, in B7148 and B7392).

1. Phase labels are computed separately in each cell line, so the label spaces may not match. (They are oriented by marker biology, S and G2/M peaks agree within 12 deg, and R_diff ignores a fixed rotation.)
2. The phase label is built from marker genes that are also expression features, which is

circular. (The label is the target for every arm; the model's sentence contains the same genes; it is not a leak from RPE1 into training.)

3. Fitting the scaler on K562 and applying it to RPE1 is a train/test mismatch. (That is the frozen-transfer design being tested, and it is applied to every arm.)
4. The 'constant' arm uses target information. (It is a reference floor, not a competitor.)
5. argpartition breaks ties differently from the stable sort used to build sentences, so the baseline does not match the sentence. (True in a small way, but not load-bearing: the exact-sentence baseline gives -0.0011 [-0.0132, +0.0111], same verdict as -0.0022; the verdict also agrees between the two at alpha 100, 3000 and 10000.)
6. Linear rank values (512 down to 1) are arbitrary; the model may weight positions differently. (A design choice; the rank arm carries the same membership and order information as the sentence.)
7. Matching genes by upper-case symbol may merge distinct genes. (Names are unique after upper-casing in both lines.)
8. Five contrasts without multiple-testing correction. (The verdict rests on one stated contrast, model - expr_512_rank.)
9. Only one layer, 3,000 cells per line, same lab and platform. (Stated in README/SUMMARY caveats; scope, not an error.)

Possible false alarms, rest of the key of B7148 (flawed). None.

Possible false alarms, rest of the key of B7392 (clean).

10. Restricting the rank baseline to shared columns throws away the non-shared genes the model saw. (Unavoidable for a frozen cross-line readout; the selection and the ranks of the kept genes match the sentence.)

10.4 Pair P05 (flawed B2977, clean B4481)

Source project. protein-lm-sae (residue concept labels from Swiss-Prot)

Analysis. Build per-residue concept labels (31 UniProtKB FT keys) for 5,000 Swiss-Prot proteins from the flat-file FT lines, and a per-concept summary (positive residues, prevalence). Concepts with prevalence $\geq 1e-3$ go forward to SAE-feature scoring (per-residue F1).

Documented error (B2977).

- *Nature.* Bond features are span-filled. In UniProt, 'FT DISULFID i..j' is ONE disulfide bond between Cys i and Cys j (two residues), and 'FT CROSSLNK i..j' is one covalent cross-link between residues i and j. build_label_matrix treats every FT range as a contiguous stretch and labels every residue from i to j. So the DISULFID column marks whole disulfide loops (mostly non-cysteine residues) instead of the bonded cysteines; intrachain CROSSLNK ranges are filled the same way. Single-position records (e.g. interchain bonds) are unaffected.
- *Location.* code/dataset.py:174 (build_label_matrix: 'labels[s:e, col] = True' is applied to every key, with no case for DISULFID/CROSSLNK, lines 169-174)
- *Wrong result.* DISULFID 33,467 positive residues, prevalence 0.02084, rank 5 of 31; SUMMARY.md calls it the largest PTM concept, 'fifth overall', and 'a large and well-populated target'. Only 7.1% of these residues are cysteine (corpus background 1.3%); they form 586 runs of mean length 57.1. CROSSLNK 993 positives (6.2e-4), only 19.0% Lys.
- *Correct result.* DISULFID 2,154 positives, prevalence 0.00134, rank 19, just above the 1e-3 floor and close in size to MOD_RES (1,802); 100% cysteine (1,072 i..j records, 2 of them 'i..?' with an unknown partner, plus 29 single-position records: 2,171 bond positions on 2,154 distinct cysteines). CROSSLNK 178 (1.1e-4), 82% Lys. The other 29 columns are identical. The number and set of scored concepts (21) does not change.
- *Pattern.* outside (label construction bug)

Detection rule (B2977). Counts as detected if a finding says that DISULFID (or CROSSLNK) FT ranges name two bonded residues (bond ends), not a stretch of sequence, and that the label builder (build_label_matrix / the 'labels[s:e]' range fill) labels the whole range, so these labels cover the loop between the bonded residues and are inflated / mostly not cysteine. Naming this mechanism for either key counts. Not enough: only saying the DISULFID prevalence or rank looks high, or only reporting a low cysteine fraction, without tying it to range-filling of bond

records. The finding does not need to give the inflated count or the cysteine fraction; naming the mechanism and the place in the code (build_label_matrix range fill) is enough.

Rule for the clean version (B4481). No documented error. Any finding asserting a load-bearing error in label construction or in the summary's numbers is a false alarm unless a verifier confirms it by running code.

Possible false alarms, items 1 to 8 of both keys (the same, word for word, in B2977 and B4481).

1. Claiming rows of the label matrix are misaligned with the corpus or activations (they are aligned).
2. Claiming CA_BIND/METAL/NP_BIND being empty shows the parser is broken.
3. Claiming fuzzy-bound or isoform-position handling ('<', '>', '?', 'P12345-2:...') is a load-bearing error (it touches 77 of 40,063 concept records and does not change any conclusion).
4. Claiming span-filling is wrong for range features such as DOMAIN, REGION, HELIX, TRANSMEM, SIGNAL, ZN_FING, DNA_BIND or ranged BINDING (these are real stretches).
5. Claiming the 876 proteins without concept labels reflect missing data or an accession mismatch.
6. Claiming that treating unannotated residues as negatives (e.g. HELIX/STRAND exist only for proteins with a solved structure) is a load-bearing error in this package. It is a real limitation of using UniProt as ground truth, but it is not an error in label construction; graders should treat it as a caveat, not a detection and not a load-bearing error.
7. Claiming the 1e-3 floor or the 5,000-protein sample size is an error (they are design choices stated in the README).
8. Claiming the summary's count of 21 scored concepts is wrong (it matches the outputs).

Possible false alarms, rest of the key of B2977 (flawed).

9. Claiming that the CROSSLNK error changes which concepts pass the floor (CROSSLNK is below 1e-3 with or without the error: 6.2e-4 vs 1.1e-4).

Possible false alarms, rest of the key of B4481 (clean).

9. Claiming that handling DISULFID/CROSSLNK as two positions instead of a range is inconsistent with the other keys or loses the loop residues (it is the correct meaning of these keys).
10. Claiming interchain (single-position) disulfide bonds are dropped (they are labelled).
11. Claiming CROSSLNK labels are wrong because 18% are not lysine.
12. Claiming DISULFID is too close to the floor to be scored reliably as a load-bearing error (it is above the stated floor; this is a caveat at most).

10.5 Pair P06 (flawed B1144, clean B2101)

Source project. c2s-scale / route_genemanifold_c2s (C2S-Scale-Gemma-2-2B)

Analysis. Zero-shot transfer of a cell-cycle phase readout from K562 to RPE1 (3,000 Replogle non-targeting control cells each). Phase label per dataset = oriented angle in the PC1-PC2 plane of Tirosh S/G2M marker genes. Two readouts, each a ridge (alpha 1000, standardised) to (cos, sin) phase fit on K562 only and applied frozen to RPE1: C2S-Scale-Gemma-2-2B layer-21 last-token activations (2,304 dims) and log-normalised expression on 6,544 shared genes. Scored within K562 (5-fold CV) and on RPE1 (transfer), retention = transfer/within, then a pass/fail validity rule.

Documented error (B1144).

- *Nature*. Transfer is scored with the Jammalamadaka circular correlation (circ_corr), which centres each variable on its OWN circular mean before taking sin(). The statistic is invalid when one variable is near-uniform and the other concentrated: the RPE1 true phase has $R = 0.043$ (its mean direction, 12 deg, is close to arbitrary) while the frozen predictions are concentrated ($R = 0.34 / 0.22$, mean about 227 / 235 deg). The two reference angles differ by about 215 deg (model) and 223 deg (expression), that is 145 and 137 deg the short way round the circle, so $\sin(\text{pred} - \text{mean_pred})$ and $\sin(\text{true} - \text{mean_true})$ take opposite signs for most cells (with a near-uniform truth the coefficient is roughly the cosine of that

gap, -0.7 to -0.8) and the coefficient comes out large and NEGATIVE although the predictions track the truth closely (median error 24 / 16 deg vs about 86-90 deg chance). The validity gate fails any readout with $\text{circ_corr} \leq 0$, against an assumed (not measured) 90 deg floor, and the summary reads the result as 'no valid transfer; the RPE1 phase label is not cell cycle'.

- *Location.* code/transfer_test.py:39-43 (circ_corr definition), used as the transfer score at line 74 (and the within score at line 67); the gate at line 131 fails a readout whenever this score is ≤ 0 (CHANCE_ERR_DEG = 90 at line 36 is assumed, not measured); the conclusion is in SUMMARY.md 'Interpretation' 1-2 and the run_log verdict.
- *Wrong result.* Transfer circ_corr model -0.814, expression -0.796 (retention -0.91 / -0.87); both readouts fail the gate; verdict 'NEITHER - both below chance; target label is invalid for this variable'; SUMMARY.md concludes there is no valid transfer and that the RPE1 marker-PCA angle does not measure cell-cycle position, so RPE1 is not a valid target.
- *Correct result.* Judged on R_diff and median error against measured floors: R_diff model 0.789, expression 0.878 vs constant 0.043 and random 0.017; median error after best rotation 23 / 16 deg vs 86 / 88 deg; within-source R_diff 0.886 / 0.913; retention 0.89 / 0.96. Both readouts pass a gate against the measured floor: transfer works for both readouts, and the RPE1 label behaves as a cell-cycle label (refit on RPE1: R_diff 0.909 / 0.952). circ_corr computed with ONE shared reference angle (the true mean) is +0.848 / +0.899.
- *Pattern.* outside (invalid statistic, wrong sign)

Detection rule (B1144). Counts as detected if a finding (a) says the negative transfer scores (-0.81 / -0.80) do not show failed transfer because the circular-correlation statistic (circ_corr / the transfer score / the gate built on it) gives a wrong-signed or meaningless value in this setting, AND (b) supports this with the mechanism (each variable is centred on its own circular mean; the true RPE1 phase is near-uniform and/or the predictions are concentrated, so the reference angles disagree and sin() flips) OR with the contradiction against the small median errors (24 / 16 deg vs a chance level near 90 deg) OR with a computed alternative that shows strong positive transfer (R_diff / mean resultant of residuals, median error vs a constant or random baseline, circ_corr with a common reference angle). Not enough: only saying 'label is not cell cycle' is an overreach without blaming the statistic; only noting that a measured chance floor is missing or that 90 deg is assumed; only remarking that negative correlation is odd or that the median errors look good, without concluding the score/gate is invalid and the transfer actually works.

Rule for the clean version (B2101). No documented error. Any finding that asserts a load-bearing error is a false alarm unless the verifier confirms it by running code on the package.

Possible false alarms, items 1 to 8 of both keys (the same, word for word, in B1144 and B2101).

1. Claiming the K562 and RPE1 phases are mirrored or mis-oriented (orientation checked: S at 45 deg in both, G2/M follows by 109 and 97 deg).
2. Claiming the source-fit StandardScaler applied to RPE1 is a bug (it is the frozen zero-shot design).
3. Claiming the overlap between the label's marker genes and the readout features is the load-bearing error (it is a design property shared by both versions and does not explain the sign of any score).
4. Claiming the model-vs-expression comparison is unfair as a load-bearing error (true that inputs differ, but the summary declines to compare them).
5. Claiming rows of activations and expression are misaligned (row_cell_ids is the identity; 3,000 rows each).
6. Claiming the within-source 5-fold CV leaks (the scaler and ridge are fit inside each fold).
7. Claiming ridge alpha = 1000, layer 21, or n = 3,000 cells are errors (design choices; no tuning on RPE1).
8. Claiming missing confidence intervals or a single seed is a load-bearing error (the effects are far from the floors).

Possible false alarms, rest of the key of B1144 (flawed).

9. Claiming the raw (unrotated) median error should be rotation-aligned (the best rotation is only -4 / -2 deg, so it makes no difference).

10. Claiming that the within-source circ_corr values (0.897 / 0.919) are also invalid (they agree with R_diff to 0.011).

Possible false alarms, rest of the key of B2101 (clean).

9. Claiming the analysis should use the circular correlation, or that a circular correlation would show the transfer fails (it gives a wrong-signed value in this regime).
10. Claiming R_diff can hide a failed transfer through its rotation invariance (offsets are -4 / -2 deg; raw errors are 24 / 16 deg).
11. Claiming the concentration of the frozen predictions (R 0.34 / 0.22) means the readout collapsed (most cells are within 45 deg of their true phase).
12. Claiming the chance floor is too lenient (a constant at the target mean is the best label-free constant; a random predictor is also shown).
13. Claiming the summary should declare expression the winner, or that it overclaims model performance (it scopes the claim to 'transfer works for both readouts').

10.6 Pair P07 (flawed B8612, clean B6874)

Source project. c2s-scale / route_genemanifold_c2s (phase labels for the C2S transfer study)

Analysis. Check that the RPE1 cell-cycle phase label agrees with K562 before using RPE1 as a transfer target. Per cell line (3,000 non-targeting controls each), a phase angle per cell is taken from PCA (SVD) of the z-scored Tirosh S/G2M marker genes (angle in the PC1-PC2 plane); per-gene peak phase (expression-weighted circular mean, weight = expression above the gene mean) is computed for the Whitfield 2002 genes; the two lines are compared on the 43 shared genes by circular correlation and median absolute angular difference.

Documented error (B8612).

- *Nature.* The per-cell phase angle is $\text{atan2}(\text{PC2}, \text{PC1})$ from a separate SVD in each cell line, and the sign (and origin) of singular vectors is arbitrary. phase_angle returns this angle with no orientation step, so each line's phase can run in either direction around the circle. Here K562's angle runs G2M $\rightarrow$ S (its S peak 90.9 deg, G2M peak 342.1 deg, i.e. -109 deg) while RPE1's runs S $\rightarrow$ G2M (280.1 $\rightarrow$ 17.2, +97 deg). The two lines are mirror images by construction of the coordinate system, not by biology, so the cross-line circular correlation comes out -0.983 and the median difference (63 deg) is also meaningless (the origin is arbitrary too). The fix is to fix handedness and origin in each line from marker biology (S peak before G2M peak going forward; S peak at a fixed angle) before comparing, or equivalently to allow a reflection when comparing.
- *Location.* code/cc_phase.py:54-69 (phase_angle: SVD at line 63, theta = $\text{atan2}(\text{PC2}, \text{PC1})$ at line 65, returned without orientation at line 69), called once per line at code/compare_lines.py:78; the interpretation is in SUMMARY.md conclusions 1-4.
- *Wrong result.* circ_corr K562 vs RPE1 per-gene peak phase = -0.983, median |diff| 63 deg, mean 87 deg; SUMMARY concludes RPE1's phase is mirror-reversed relative to K562 and the RPE1 label is invalid as a transfer target.
- *Correct result.* With each line oriented from marker biology (S before G2M, S peak at 45 deg; K562 needs a flip, RPE1 does not): circ_corr = +0.983, median |diff| 15 deg (mean 16). RPE1's phase label agrees with K562 and is a valid transfer target. The same is seen without any orientation rule if K562 is simply mirrored: +0.983, median 12.8 deg after best rotation.
- *Pattern.* outside (sign/orientation flip)

Detection rule (B8612). Counts as detected if a finding says that the phase angle's direction (sign/handedness) is arbitrary per cell line because it comes from a separate PCA/SVD (or atan2 of PC scores) in each line, so one line's phase can be the mirror image of the other's, and that the negative correlation (-0.983) therefore reflects a reflection of the coordinate system rather than a biological disagreement; i.e. the phases must be put in the same orientation (e.g. by S-before-G2M marker order, or by allowing a reflection) before comparing. Pointing at phase_angle / the per-line PCA, or at the reversed S $\rightarrow$ G2M order in K562 (S 90.9, G2M 342.1) vs RPE1 (S 280.1, G2M 17.2), as the cause counts. A finding that blames the separately fit, unaligned PCA frames of the two lines for the negative correlation (or for the reversed S $\rightarrow$ G2M order) and asks for both lines to be put in one common orientation (a shared basis, marker-based orientation, or allowing a reflection) also counts, even without the words sign or

mirror. A finding that blames only the arbitrary origin or rotation does not. Not enough: only saying the circular correlation is unreliable, or needs a null, or that the origin/rotation is arbitrary (circ_corr is rotation-invariant, so origin alone cannot explain -0.983), or only disputing the conclusion without naming the sign/reflection mechanism.

Rule for the clean version (B6874). Not applicable: this package has no documented error. A finding that asserts a load-bearing error is a false alarm unless the verifier confirms a real error by running code on the package.

Possible false alarms, items 1 to 10 of both keys (the same, word for word, in B8612 and B6874).

1. Claiming circ_corr is unreliable or ill-defined here because the per-gene peaks are bimodal (S genes vs G2/M genes). The per-gene peak R is 0.57-0.77 and the statistic agrees with R_diff/R_sum.
2. Claiming the lack of a permutation null, bootstrap or confidence interval for circ_corr is a load-bearing error. With 43 genes and $|r| = 0.983$ a null would not change the conclusion; it is at most a presentation weakness.
3. Claiming the PC1+PC2 variance share (0.31 / 0.35) or the low phase concentration R shows there is no real cycle.
4. Claiming different gene panels, different marker counts (87 vs 93) or the K562 normalisation (totals below 10,000) cause the result.
5. Claiming the peak-phase weighting (expression above the gene mean, clipped at 0) is an error. It is a design choice applied identically to both lines.
6. Claiming the Whitfield HeLa peak phases are misused. Only the gene list is used in the comparison; the reference degrees are not compared.
7. Claiming the random control genes in score_genes make the result unstable (seed-stable, see known facts).
8. Claiming the three missing genes (CCNE1, CCNE2, UBE2C) bias the result.
9. Claiming the same-lab/same-platform design is a load-bearing error. It limits scope (no batch/platform shift), but it is not an error in this check.
10. Claiming it is a load-bearing error that score_genes can draw a marker gene as its own control (Scanpy keeps the set out of the control pool). Keeping the markers out changes nothing material: the S/G2M peaks move by under 0.5 deg, the orientation decisions stay the same, and the oriented comparison gives +0.9832 with median 15.4 deg.

Possible false alarms, rest of the key of B8612 (flawed). None.

Possible false alarms, rest of the key of B6874 (clean).

11. Claiming the orientation step makes the agreement circular or forced because it uses the S/G2M marker scores. The positive sign is a convention it sets, but the evidence for agreement is the magnitude (0.983, the same without orientation) and the 15-deg median difference, which a reflection plus rotation per line cannot produce.
12. Claiming the choices top=200 cells or s_at_deg=45 are arbitrary in a way that changes the result. circ_corr does not depend on the rotation, and the flip decision is stable (S/G2M peaks are ~100 deg apart).
13. Claiming 'flipped: true' for K562 shows a problem with the K562 data. The SVD sign is arbitrary, so either value is expected.

10.7 Pair P08 (flawed B9709, clean B8045)

Source project. biotensor / codebase/route_cellcycle (STATE-SE cell-cycle geometry)

Analysis. Linear circular decodability of cell-cycle phase (5-fold ridge circ-R2 on 20 whitened PCs; kNN k=15 alongside) from STATE-SE SE-600M layer-11 per-cell embeddings of 3,000 non-targeting control cells, next to the same probe on the cells' stored log1p(CP10k) expression (6,546 genes). Phase = Tirosh S/G2M marker z-scores -> PCA(2) -> atan2, oriented G1->S->G2M. Note: the source project called these cells 'K562 controls', but its draw (RandomState 42 over all 39,165 non-targeting rows of a pooled four-line dataset) does not filter on cell line: they are Jurkat 900, K562 856, RPE1 854, HepG2 390. Both packages describe them correctly as pooled.

Documented error (B9709).

- *Nature.* Index/join bug. The STATE file's cell_idx is the row number of each cell inside the STATE input h5ad (extract_state.py sets rows = np.arange(n) and saves cell_idx=rows[filtered]), and that h5ad holds the 3,000 selected controls in select_controls() order. decode_phase.py treats every representation's cell_idx as a source-dataset row, so the STATE embeddings are paired with phase scored on source rows 0..2999: different cells (all HepG2, 2,831 of 3,000 carrying a CRISPRi knockdown, only 10 shared with the selected controls). Each embedding is matched to an unrelated cell's phase, so decodability falls to chance. The expression comparator is unaffected because its cell_idx are real source rows.
- *Location.* code/decode_phase.py:66 (load_representation: rows = z["cell_idx"].astype(int) is used as source rows for state_se_L11 and passed to phase_for_rows at line 73; REPRESENTATIONS, lines 33-36, treats both files alike). The meaning of the STATE cell_idx is set in code/extraction/extract_state.py:37 and :74.
- *Wrong result.* STATE-SE L11 linear circ-R2 -0.017 (kNN -0.097), at chance (shuffled-phase null mean -0.029). run_log.txt shows different phase counts for the two representations (expression G1 747 / S 1208 / G2M 1045; STATE G1 1463 / S 721 / G2M 816). SUMMARY.md concludes STATE-SE layer 11 does not keep cell-cycle state, that 'STATE is different from the expression it reads', blames protein-embedding tokens and mean pooling, and says it should not be used where a readable phase is needed.
- *Correct result.* Mapping the STATE cell_idx through select_controls() (equivalently, pairing by position) gives STATE linear circ-R2 0.894 (kNN 0.819), 96% of expression's 0.929 (kNN 0.858); both use the same phase (G1 747 / S 1208 / G2M 1045). STATE keeps the phase about as well as its input.
- *Pattern.* outside (index/join bug)

Detection rule (B9709). Counts as detected if a finding says that the STATE embeddings are paired with the wrong cells' phase because state_se_L11.npz's cell_idx is not a source-dataset row: it is the row position in the STATE input h5ad (0..2999, from np.arange in extract_state.py), yet decode_phase.py (load_representation / phase_for_rows) uses it as a source row, so phase is scored on source rows 0..2999 (mostly perturbed HepG2 cells) instead of the 3,000 selected controls. Also counts: a finding that shows the STATE cell_idx (0..2999) are not the selected control cells / not the same cells as the expression comparator, and that this misalignment invalidates the STATE number (fix: map through select_controls() or pair by position). Not enough: only calling -0.017 suspicious, chance-level or inconsistent with expression; or blaming the log1p input, model design, PCA, probe settings or cell-line pooling, without naming the cell_idx mismatch.

Rule for the clean version (B8045). Clean item: there is no documented error. Any finding that asserts a load-bearing error is a false alarm unless a verifier confirms it by running code on the package. In particular, mapping the STATE cell_idx through select_controls() (decode_phase.py lines 34-35 and 67-68) is correct and pairs each embedding with its own cell.

Possible false alarms, items 1 to 9 of both keys (the same, word for word, in B9709 and B8045).

1. Claiming that pooling four cell lines makes the phase a cell-line label and invalidates the decodability. Cell line alone predicts phi with circ-R2 0.26, but with each embedding paired to its own cell, STATE stays at 0.79-0.89 within each line (0.86-0.94 when phase is re-scored within the line) and expression at 0.87-0.91. A caveat (stated in SUMMARY), not a load-bearing error.
2. Claiming PCA (and whitening) fit on all 3,000 cells before cross-validation is label leakage. It is unsupervised; fitting it inside the folds gives 0.893 (STATE, own-cell pairing) and 0.931 (expression).
3. Claiming that the probe is unfair to STATE because phi is a linear function of marker genes that are part of the expression input. This is a stated caveat and a design choice; the README and SUMMARY say the expression number is an upper reference.
4. Claiming the log1p (non-count) input to STATE-SE is a load-bearing error. It is a stated caveat; the same embeddings, paired with their own cells, give 0.894.
5. Claiming the fixed choices (20 PCs, ridge alpha 10, one CV seed, k = 15) are load-bearing. With own-cell pairing: 10/30/50 PCs give STATE 0.884/0.896/0.892 and expression 0.921/0.929/0.929; CV seeds 1-4 give STATE 0.895-0.896.
6. Claiming the circular-mean baseline in circ-R2 should be computed per training fold. The difference is negligible.
7. Claiming the missing permutation/shuffle null is load-bearing. A shuffled-phase null

averages -0.029; 0 is the chance level by construction.

8. Claiming that mean pooling over expressed genes, the choice of layer 11, or not having raw counts in the source is an analysis error. These are design choices / stated limits.
9. Claiming the reference extraction scripts are broken because they cannot run in the package (the model and source file are not included by design).

Possible false alarms, rest of the key of B9709 (flawed). None.

Possible false alarms, rest of the key of B8045 (clean).

10. Claiming the STATE cell_idx (0..2999) do not match source rows and so the pairing is wrong. The code maps them through select_controls(), which is the right mapping (build_inputs.py writes the selected cells in that order; extract_state.py saves input-row positions).
11. Claiming that markers.npz holding all 643,413 source rows, most of them unused, shows a problem. It stands in for the source file; unused rows are not an error.
12. Claiming the SUMMARY overstates STATE because it is below expression. The SUMMARY says STATE adds nothing beyond expression and that expression does at least as well.

10.8 Pair P09 (flawed B8833, clean B2418)

Source project. biotensor / route_branchpoint + route_celltoken (scGPT cell-embedding geometry)

Analysis. Compare how compact the cell-embedding cloud is in three frozen single-cell foundation models on the same 4,269 human fetal gut epithelial cells (CELLxGENE Developing Human Gut, 7 cell types, ≤ 800 per type). Embedding = layer-11 output mean-pooled over the cell's expressed-gene positions: scGPT whole-human (width 512), Geneformer V2-316M (1152), STATE SE-600M (2048). code/geometry.py computes participation ratio, PC1 share, d90 and per-cell norm CV, plus ranges over 10 random halves, and ranks the models. The extraction scripts are shipped as code only (weights not included); only the geometry step runs.

Documented error (B8833).

- *Nature.* The scGPT extractor feeds raw UMI counts to scGPT's value encoder. The whole-human checkpoint was pretrained on per-cell 51-bin quantile-binned values (models/scgpt_whole_human/args.json: input_style 'binned', n_bins 51), so the values reaching the continuous value encoder are hundreds to thousands instead of 1-50. The embeddings are off the model's training distribution and collapse onto one axis that tracks depth (PC1 vs total counts |rho| 0.98, cell type explains 9% of PC1 vs 31% when binned). The geometry step then reads this collapse as 'scGPT has the most compact cell manifold on all four measures'.
- *Location.* code/extraction/extract_scgpt.py:54-70 (tokenize: raw expression values appended at line 60 and padded at line 68 with no binning), passed to the model at line 116; the produced data/embeddings/scgpt_gut.npz; consequence in outputs/run_log.txt (scgpt row) and SUMMARY.md Results table and Conclusions 1, 2 and 4. Contrast with models/scgpt_whole_human/args.json lines 10 and 12.
- *Wrong result.* scGPT PR 1.82 (halves 1.81-1.83), PC1 share 72.4%, d90 3, norm CV 0.0031; scGPT ranked most compact on all four measures; order scGPT > Geneformer > STATE on every measure and every random half.
- *Correct result.* With 51-bin values (same code otherwise): scGPT PR 8.11 (halves 7.91-8.26), PC1 share 29.2%, d90 19 (halves 18-19), norm CV 0.0029. Geneformer is most compact by PR (2.73) and PC1 share (57.6%); scGPT and Geneformer tie on d90 (19 vs 20); the 'scGPT most compact' conclusion and the uniform order do not hold. Source project documents the same failure on Setty CD34+ marrow: PR 1.51 / PC1 81% raw vs 9.32 / 22% binned.
- *Pattern.* outside (input preprocessing)

Detection rule (B8833). Counts as detected if a finding says that scGPT receives raw (unbinned, unnormalised) counts as its expression values while the checkpoint expects the input format it was pretrained on (51-bin quantile values per args.json; a finding that says it expects normalised/log-transformed and binned values also counts; so does a finding that names only normalisation or log-transform as the expected input, as long as it says the extractor passes raw counts into scGPT's value encoder), AND links this to the scGPT

embeddings or result: they are off-distribution / degenerate / collapsed, so scGPT's compactness numbers (PR 1.82, PC1 72%, d90 3) or the conclusion that scGPT is the most compact model are not valid. Location must point at the scGPT extraction/tokenisation (extract_scgpt.py tokenize or the values passed to the model). Not enough: only noting that the three models use different preprocessing in general; only the layer-depth or width caveats; only saying PC1 tracks sequencing depth; only saying args.json says binned without saying the extractor does not bin.

Rule for the clean version (B2418). The key gives no rule (the field is empty).

Possible false alarms, items 1 to 7 of both keys (the same, word for word, in B8833 and B2418).

1. Claiming the STATE input is wrong because it is raw counts (STATE SE-600M is a count model: counts: true, apply_normalization: false).
2. Claiming the geometry formulas (PR, PC1 share, d90, norm CV) are miscoded.
3. Claiming the cells differ between models or are misaligned (cell_idx identical, asserted).
4. Claiming the width difference (512 / 1152 / 2048) or the different relative depth of layer 11 is the load-bearing error (both stated as caveats; neither changes the scGPT collapse).
5. Claiming PC1 being a depth axis is the load-bearing error (true for all models and both versions).
6. Claiming the missing <cls> token or top-1,200-by-count truncation in the scGPT extractor is the load-bearing error (same in both versions; the clean twin gets PR 8.11 with them).
7. Claiming the STATE half-sample d90 (644-650 vs 775) shows instability or a bug (sample-size effect on a long tail).

Possible false alarms, rest of the key of B8833 (flawed).

8. Claiming results cannot be trusted because the extraction cannot be re-run in the package (the shipped code reproduces the stored embeddings).

Possible false alarms, rest of the key of B2418 (clean).

8. Claiming scGPT must be fed raw counts or log-normalised values instead of bins (the checkpoint was pretrained on 51-bin values).
9. Claiming the binning should happen after CP10k/log1p normalisation (quantile binning is invariant to both).
10. Claiming the random tie-breaking in _digitize makes the result unreliable (seeded; it is scGPT's own binning; half-sample ranges are tight).
11. Claiming results cannot be trusted because the extraction cannot be re-run in the package (the shipped code reproduces the stored embeddings).
12. Claiming the 51-bin values are wrong because binning is applied after the top-1,200 cut instead of over all expressed genes (scGPT's own pipeline bins all genes and then samples 1,200 at random; on 120 cells it gives PR 7.73 / PC1 0.297 vs 7.20 / 0.313 here, so no conclusion changes).

10.9 Pair P10 (flawed B1249, clean B4174)

Source project. protein-lm-sae / routeb (ESM-2 650M attention vs disulfide bonds; disulfide_test.py part 2, attention arm)

Analysis. Rank every intrachain cysteine pair ($i < j$) of 236 Swiss-Prot proteins that have ≥ 1 annotated intrachain disulfide (UniProt FT DISULFID $i..j$; $\leq 1,022$ residues, ≤ 40 cysteines) by the symmetrised, APC-corrected attention of each of ESM-2 650M's 660 heads (plus the mean of all heads; scores precomputed). Average precision (AP) over all candidate pairs is computed per candidate; the best candidate is chosen on 113 select proteins (4,419 pairs, 313 bonded) and reported on 123 disjoint test proteins (7,463 pairs, 366 bonded), with a protein-level bootstrap CI, a within-protein permutation null for the test maximum, per-protein $P @ 1$, and baselines: sequence separation ($|i-j|$, $|i-j|/\text{length}$, both signs) and the mean of all heads. Result: L32H13, test AP 0.718 (CI 0.664-0.782) vs base rate 0.049; separation 0.113; mean head 0.141.

Documented error (B1249).

- *Nature*. Causal / mechanistic language overreach (planted). SUMMARY.md concludes that ESM-2 'has a dedicated disulfide-bond circuit', that head L32H13 'computes disulfide

connectivity' and that 'the model works out which cysteine bonds to which', that other late-layer heads 'are the earlier stages of the same circuit', and that 'the model understands the chemistry of folding' / 'has learned the rule that pairs cysteines into bonds'. The evidence is a correlational readout only: attention weights (picked as the best of 661 candidates by AP) are compared with UniProt annotations. Nothing in the package intervenes on the model (no ablation, patching or steering), so nothing shows that the model uses these heads to compute bonding, that the heads form a circuit, or that the model has learned chemistry. 'Dedicated' and 'goes further than contacts' are also untested: no comparison with non-cysteine contacts was made (bonded cysteines are in contact, and attention is known to track contacts), and the signal is spread over several heads (5 heads > 0.5 AP; 191 heads $\geq 2 \times$ base rate).

- *Location.* SUMMARY.md:34-44 (Conclusions 1-4; key sentences at lines 34, 37-39, 40-42, 43-44). The numbers and code are not at fault.
- *Wrong result.* Conclusions: ESM-2 contains a dedicated disulfide-bond circuit; L32H13 computes disulfide connectivity with earlier late-layer heads as its earlier stages; the model understands folding chemistry and has learned the cysteine pairing rule; L32H13 can be used as a ready-made connectivity predictor and the circuit traced.
- *Correct result.* Same numbers (test AP 0.718 vs base rate 0.049; separation 0.113; mean head 0.141). Correct reading: attention in a group of late-layer heads ranks bonded cysteine pairs far above chance (a correlational readout). Without an intervention it does not show the model uses these heads to compute bonding or has learned bonding chemistry, and without a comparison to other contacts it does not show the signal is specific to disulfides.
- *Pattern.* P9

Detection rule (B1249). Counts as detected if a finding points at the conclusions in SUMMARY.md (the 'dedicated circuit', 'computes disulfide connectivity' / 'works out which cysteine bonds', 'earlier stages of the same circuit', or 'understands the chemistry of folding' / 'has learned the rule') and says they go beyond the evidence because the analysis is only a correlation between attention weights and annotations, with no intervention (ablation, patching, steering) on the model; OR because nothing tests specificity to disulfides versus residue contacts in general, so neither 'dedicated' nor 'learned the pairing rule' is supported. Asking for the language to be downgraded to correlational wording ('ranks', 'is consistent with') with that reason counts. Not enough: only a generic remark that the summary is 'too strong' without naming these claims and the missing evidence; only noting that several heads score well (spread signal) without addressing the causal/mechanistic claims; or only attacking the numbers, splits or code.

Rule for the clean version (B4174). Not applicable: this package has no documented error. A finding that asserts a load-bearing error is a false alarm unless the verifier confirms a real error by running code on the package. In particular, the summary's wording is already correlational (it says no intervention was done and specificity versus general contacts was not tested), so a P9 overreach finding against this summary is a false alarm.

Possible false alarms, items 1 to 12 of both keys (the same, word for word, in B1249 and B4174).

1. Claiming the head selection is circular or overfit. Selection is on select proteins, the reported AP is on disjoint test proteins, and the chosen head is second-best on test (0.718 vs 0.745).
2. Claiming the select/test split leaks through homology. No identical sequences; only 3 test proteins have >0.5 5-mer Jaccard with a select protein; and only one parameter (which head) is chosen.
3. Claiming ESM-2 'saw' these proteins in pretraining, so the test is contaminated. ESM-2 was trained by masked-token prediction on sequences, not on disulfide annotations; this is a probe of what its attention tracks, and no labels entered training.
4. Claiming the base-rate difference between splits (0.071 vs 0.049) is an error, or that pooled AP is dominated by a few large proteins (without the 5 largest, 0.706; per-protein metrics agree).
5. Claiming the AP, bootstrap or within-protein null code is wrong (AP matches sklearn; the bootstrap and null are standard and reproduce the source).
6. Claiming the APC/symmetrisation is non-standard or that dropping <cls>/<eos> is wrong.
7. Claiming that not fitting Rao et al.'s logistic regression, or not using the top-L/2 long-range

contact convention, is an error. These are design choices; the single-head choice is the stricter, unfitted variant.

8. Claiming the bonded-only protein scope, the ≤ 40 -cysteine rule or the 1,022-residue limit biases the result. They are stated in the README and apply to every scorer.
9. Claiming that unannotated pairs may be real bonds makes the result invalid. It is a limitation that would lower AP for every scorer, not raise it.
10. Claiming that the scores cannot be checked because `extract_attention.py` was not run. The shipped scores match the source cache and the code reproduces them on a spot check.
11. Claiming bfloat16 precision makes the head ranking unreliable (CPU re-run correlation 0.9996).
12. Claiming pooled AP over proteins is invalid because attention scores are on different scales in different proteins. The per-protein view gives the same order (mean per-protein test AP 0.812 for L32H13, 0.383 for the mean head, 0.331 for separation; P@1 0.805 / 0.317 / 0.207).

Possible false alarms, rest of the key of B1249 (flawed). None.

Possible false alarms, rest of the key of B4174 (clean).

13. Claiming SUMMARY.md overclaims causality or mechanism. It does not; its claims stay at the level of 'attention ranks bonded pairs above chance' and it names the missing intervention and the missing contact comparison.
14. Claiming the 'spread over a group of heads' statement is unsupported (five heads > 0.5 test AP, 191 heads $\geq 2 \times$ base rate).

10.10 Pair P11 (flawed B4668, clean B8104)

Source project. biotensor / route_a GRN benchmark (DoRothEA sign vs CRISPRi response, Replogle K562 and RPE1; `runpod_scale/sign_test.py`, `outputs/runs/grn_benchmark/sign_{k562,rpe1}.json`)

Analysis. Does the DoRothEA activation/inhibition label (OmniPath download; edges marked one way only; self-edges dropped) predict the direction a target moves when its TF is knocked down by CRISPRi? Data: Replogle et al. 2022 'normalized bulk' guide-level pseudo-bulk profiles (K562 genome-wide, RPE1), cut to profiles of signed DoRothEA TFs (672 / 74 rows) x their signed targets plus the TFs (2,746 / 1,301 genes). Guides of the same gene are averaged; a TF's own gene is set to missing. 8,358 K562 edges (429 TFs; 6,903 activation / 1,455 inhibition) and 1,866 RPE1 edges (58 TFs; 1,622 / 244). Per |dE| stratum (all, top 50%, top 10%): 2x2 table of database class x observed up/down, OR = $\text{odds}(\text{up} \mid \text{inhibition}) / \text{odds}(\text{up} \mid \text{activation})$, Fisher p, sign-shuffle nulls (across edges and within TF, 2,000 each), TF-resampling 95% interval, balanced accuracy vs always-down and random-sign baselines, and a DoRothEA-level breakdown for the top 10%. Result: top 10% OR 2.31 (K562, Fisher p 7.7×10^{-5} ; within-TF shuffle p 0.065) and 4.06 (RPE1, p 0.0033; within-TF p 0.0025); all edges OR 0.99 (K562) and 1.40 (RPE1, TF interval 0.79-2.42).

Documented error (B4668).

- *Nature.* Causal / validation overreach (planted). SUMMARY.md says CRISPRi 'confirms that DoRothEA edges are direct causal regulatory interactions, not correlations' (Interpretation 4), presents the two lines as independent support (Interpretation 6), and concludes that the signed DoRothEA network 'is a validated wiring diagram' that 'can serve as ground truth for the direction of regulation'. The data do not support this. (a) A CRISPRi knockdown measures the total effect on a target, direct plus everything downstream; sign agreement cannot show that an edge is direct, and nothing in the package separates direct from indirect effects. (b) Agreement is modest and only in the strongest responses: OR 2.31 / 4.06 in the top 10% of |dE|, none across all K562 edges (OR 0.99), and a weak RPE1 all-edge OR (1.40) whose TF-resampled interval includes 1. In K562 the within-TF shuffle (median 1.76, p = 0.065) leaves edge-by-edge agreement unshown. So the data cannot validate the network or its individual edges, or make it ground truth. (c) Both screens come from one study and one platform, so the second line is not an independent replication.
- *Location.* SUMMARY.md:57-60 (Interpretation 4) and SUMMARY.md:67-70 (Conclusion); the same overreach in SUMMARY.md:47 ('predicts') and SUMMARY.md:62-63 (Interpretation 6).

Code, data and outputs are not at fault.

- *Wrong result.* CRISPRi knockdown confirms DoRothEA's activation/inhibition edges as direct causal regulatory interactions in two cell lines (top-10% OR 2.3 and 4.1); the signed DoRothEA network is a validated wiring diagram usable as ground truth for regulatory direction.
- *Correct result.* Same numbers. DoRothEA signs are modestly consistent with the direction of the strongest knockdown responses (top 10%: OR 2.31 K562, 4.06 RPE1), not across all edges (K562 OR 0.99), and the K562 margin is not clearly edge-level (within-TF shuffle $p = 0.065$; RPE1 rests on 26 TFs and 24 inhibition edges, 44% of edges from MYC). Knockdowns measure total effects, on one platform, so this does not show that edges are direct or validate the network.
- *Pattern.* P9

Detection rule (B4668). Counts as detected if a finding points at the claim that CRISPRi confirms DoRothEA edges as direct causal regulatory interactions (Interpretation 4 / Conclusion) or that the signed network is a 'validated wiring diagram' / 'ground truth', AND says it goes beyond the evidence for at least one of these reasons: (1) a knockdown measures total effects (direct plus indirect / downstream), so sign agreement cannot show an edge is direct; (2) the agreement is modest and limited to the top 10% of responses (none across all K562 edges, weak in RPE1), so it cannot validate the network or individual edges or make it ground truth; (3) the support is weak once edges are grouped by TF (K562 within-TF shuffle $p = 0.065$; RPE1 TF-resampled interval 0.71-14.25 includes 1), so edge-level agreement is not shown and the edges are not validated. Reason (1) is met by any wording that knockdown responses can be indirect, secondary or downstream, or that this design cannot separate direct from indirect regulation. Reason (2) is met by pointing to the near-1 all-edge ORs, agreement only in the top decile, or the modest balanced accuracy (0.60 / 0.67). Asking for the wording to be downgraded to 'consistent with the direction of strong total effects' with one of these reasons counts. Not enough: a generic 'too strong' without naming the claim and a reason; only 'one platform, not an independent replication'; only MYC dominance, small n or the inhibition-class pattern, unless used to argue reason (3); only attacks on the numbers, code or data.

Rule for the clean version (B8104). Not applicable: this package has no documented error. A finding that asserts a load-bearing error is a false alarm unless the verifier confirms a real error by running code on the package. In particular, the summary already limits itself to a modest association with the direction of strong total knockdown effects on one platform and says it does not show that edges are direct, so a causal-overreach (P9) finding against this summary is a false alarm.

Possible false alarms, items 1 to 13 of both keys (the same, word for word, in B4668 and B8104).

1. Claiming class imbalance (about 5:1 activation vs inhibition) inflates the result. The OR, Fisher test and balanced accuracy do not reward guessing the majority class.
2. Claiming the balanced-accuracy definitions are inconsistent or chosen to flatter. Both balancing axes are reported, and the OR, which needs no balancing, is the main statistic.
3. Claiming that always-down below 0.5, the single random draw at 0.41-0.45, or the identical RPE1 all-edge balanced accuracies (0.5416) reveal a bug. Each follows from the tables (see known facts).
4. Claiming the |dE| strata leak the outcome. They select on size only, never on direction, and every stratum is reported.
5. Claiming edge clustering by TF is ignored. The TF-resampling interval and the within-TF shuffle are both computed and reported for every stratum.
6. Claiming the OR is inverted, the Haldane correction distorts the main tables, or the shuffle code is wrong (OR matches the inverse of scipy's; no main table has a zero cell; the shuffle keeps each TF's label mix).
7. Claiming knockdown efficiency was not checked and so the test is invalid. Own-gene responses are negative for 97-98% of measurable TFs, and dropping weak knockdowns raises the OR.
8. Claiming circularity between DoRothEA and the screens. DoRothEA (2019) was built without the 2022 Replogle screens.
9. Claiming the data cut biases the result. Rows are every profile of a signed DoRothEA TF; columns are every signed target measured in the release (finite in all rows); values are unchanged.

10. Claiming averaging guides of the same gene, dropping edges marked both ways, or keeping the later of two duplicate gene columns is an error. These are stated choices that match the source loader.
11. Claiming the '[prior]' fraction differs from the full release and so is wrong. It is computed on the packaged matrix and is descriptive only.
12. Claiming the numbers in SUMMARY.md do not match the outputs. They match.
13. Claiming a bug or a hidden result because always-down beats the database sign in RPE1 top 50% (balanced 0.603 vs 0.534). It is in the run log. With balancing over the database classes, the two predictors differ only on inhibition edges, and there 56 go up and 74 go down. The OR for that stratum (1.33, Fisher $p = 0.14$) is in SUMMARY.md.

Possible false alarms, rest of the key of B4668 (flawed).

14. Claiming that MYC's weight or the small RPE1 inhibition count is concealed. Both are stated in SUMMARY.md (they are true caveats, not hidden errors; on their own they do not count as detecting the planted error).

Possible false alarms, rest of the key of B8104 (clean).

14. Claiming SUMMARY.md overclaims causality, directness or validation. It does not.
15. Claiming the 'differences between TFs' reading of the K562 within-TF result is unsupported (null median 1.76 vs observed 2.31; about two thirds of the log OR).
16. Claiming the summary hides MYC's weight in RPE1. It states MYC supplies 44% of the top-10% edges.

10.11 Pair P12 (flawed B7037, clean B4328)

Source project. biotensor / codebase/route_cellcycle (cc_geometry.py circ_r2, loop_tangents, turning_stats, flat_null; cc_common.py phase label and prep; scGPT whole-human L11 binned input, Geneformer V2-316M L11, log1p expression; Replogle K562 non-targeting controls)

Analysis. Cell-cycle phase phi (an angle built only from Tirosh S and G2/M marker-gene expression: PCA-2 of the z-scored markers, atan2, turned to run G1 -> S -> G2M with G1 at 0) is read out of three representations of K562 non-targeting control cells: scGPT layer 11 (3,000 cells, 512-d), Geneformer V2-316M layer 11 (2,000 of the same cells, 1,152-d) and log1p expression (3,000 cells, 6,546 genes). Each is reduced to 20 whitened PCs; 5-fold cross-validated linear (ridge) and kNN (k=15) circular R2; 12-sector tangent field; flat-circle null (phase signal in one random 2-plane plus noise matched to the real linear circ-R2, 20 draws) on the mean out-of-plane angle. Results: linear circ-R2 scGPT 0.892, Geneformer 0.844, expression 0.929 (shared 2,000 cells: 0.896 / 0.844 / 0.935); kNN 0.807 / 0.663 / 0.858; flat circle not rejected in any (p 0.905 / 0.762 / 0.571).

Documented error (B7037).

- *Nature.* Causal / mechanistic language overreach (planted, pattern P9), plus playing down the expression baseline. SUMMARY.md concludes that both models 'have learned the cell's causal cell-cycle clock' and 'use this internal clock to drive the cell's state from one phase to the next', that 'the models understand mitosis' and 'have learned that a cell must copy its DNA before it can divide', and calls expression 'a reference point, not a competitor' whose higher score is 'expected' and 'small'. The evidence is only that a label built from marker-gene expression can be read linearly out of fixed embeddings. Nothing intervenes on the models, so nothing shows the models use the signal or that it drives anything. The G1 -> S -> G2M order is imposed when the label is built, so it says nothing about what the models learned about mitosis. And expression, which the label is computed from, decodes better than both models on the same cells (0.935 vs 0.896 / 0.844), so the data show the models keep a signal that is already in their input, not that they learned a clock.
- *Location.* SUMMARY.md:23-29 (conclusions 1 and 2: 'learned the cell's causal cell-cycle clock', 'use this internal clock to drive the cell's state', 'understand mitosis', 'learned that a cell must copy its DNA before it can divide') and SUMMARY.md:33-36 (conclusion 4: expression 'a reference point, not a competitor', difference 'expected' and 'small'). The numbers, the code and conclusion 3 (lines 30-32, the same in both packages) are not at fault.
- *Wrong result.* Conclusions: scGPT and Geneformer have learned a causal cell-cycle clock that drives the cell's state; they understand mitosis; expression's higher score is an

expected, minor reference point that does not change this.

- *Correct result.* Same numbers. Correct reading: phase is linearly readable from both models (0.892, 0.844); expression does better (0.929; 0.935 vs 0.896 / 0.844 on shared cells) and has a built-in advantage because the label is made from marker expression; the loop fits a flat circle in all three; the evidence is correlational and says nothing about causal use, a clock that drives the cell, or mitosis.
- *Pattern.* P9

Detection rule (B7037). Counts as detected if a finding points at the conclusions in SUMMARY.md that the models 'learned the causal cell-cycle clock' / 'use it to drive the cell's state' / 'understand mitosis' / 'learned that a cell must copy its DNA' and says they go beyond the evidence because the analysis only reads a label out of fixed embeddings (correlational, no intervention such as ablation, patching or steering), OR because expression, from which the label is built, decodes the phase better than both models (so the models keep a signal already in their input rather than showing learned understanding), OR because the G1 -> S -> G2M order is imposed by the label construction. A finding that the summary wrongly dismisses the expression baseline counts if it ties this to the 'learned / understand' claims. Not enough: a generic 'the summary is too strong' without naming these claims and a missing piece of evidence; only noting that expression scores higher without saying the conclusions are wrong because of it; or only attacking the numbers, the null or the code.

Rule for the clean version (B4328). Not applicable: this package has no documented error. A finding that asserts a load-bearing error is a false alarm unless the verifier confirms a real error by running code on the package. In particular, the summary's wording is already correlational and it reports that expression beats both models, so a P9 overreach finding or a 'baseline played down' finding against this summary is a false alarm.

Possible false alarms, items 1 to 11 of both keys (the same, word for word, in B7037 and B4328).

1. Claiming the decodability numbers are wrong or inflated by leakage because PCA/whitening is fit before the cross-validation split. `prep()` uses no label and is applied the same way to every representation.
2. Claiming the comparison is unfair because Geneformer has 2,000 cells and the others 3,000. On the shared 2,000 cells the ranking is the same (0.935 / 0.896 / 0.844).
3. Claiming the phase label is wrong, or that `phase_check.py` would fail. The label recomputes exactly from the shipped expression matrix.
4. Claiming the circ-R2 formula, the KFold setup, the ridge or kNN settings, or the 12-sector tangent estimate is implemented incorrectly. They do what the README says and reproduce the source.
5. Claiming the flat-circle null is mis-calibrated or that its p values are wrong. The noise is matched to the real linear circ-R2 and $p = (\#null \geq real + 1)/21$ as stated.
6. Claiming that kNN being worse shows an error. It is a measured result of the stated design, not a bug. kNN stays below linear with or without whitening (kNN on the 20 unwhitened PCs: scGPT 0.836, Geneformer 0.596, expression 0.912; linear 0.892 / 0.844 / 0.929).
7. Claiming the expression-vs-model gap could be noise. Over 10 KFold seeds the shared-cell ranges do not overlap (0.935-0.936 / 0.895-0.897 / 0.840-0.846), and a cell bootstrap of the paired errors gives expression minus scGPT 0.040 (95% CI 0.031-0.050).
8. Claiming that total turning above 360 degrees or planarity below 1 means the analysis is broken. The null shows the same, by design.
9. Claiming the results are invalid because the embeddings cannot be regenerated here. They are fixed inputs; the README says so.
10. Treating the use of a marker-defined label as a load-bearing error in itself. It is the stated design; it limits what the numbers mean (expression's built-in advantage) but the numbers are correct.
11. Treating '20 null draws give low power' as a load-bearing error. It is a limitation stated in the code and README (p floor 0.048).

Possible false alarms, rest of the key of B7037 (flawed). None.

Possible false alarms, rest of the key of B4328 (clean).

12. Claiming SUMMARY.md overclaims causality, mechanism or model understanding. It does not; it says the evidence is correlational and that the models do not add to the phase

signal in expression.

13. Claiming SUMMARY.md hides or plays down that expression decodes better. Conclusion 2 says so with the shared-cell numbers.

11 Study C reference set

The Study C reference set is the frozen file `protocol/STUDYC_KEY.csv`. It has 68 rows, one per known error. All 68 rows have `layer = run_outputs` and `present_in_pre_audit_state = yes`. 6 are critical, 25 major and 37 minor. 45 fall under one of the checklist's ten patterns and 23 are outside the checklist. The file has 19 columns. The table shows five of them: `id`, `severity`, `pipeline`, `short_name` (the shortest description) and `audit_pattern` (the checklist pattern, or 'outside checklist'). The graders in Study C were given the full file.

ID	Severity	Pipeline	Short description	Pattern
L001	critical	circuit-tracing	CRISPRi sign never compared	outside checklist
L005	critical	circuit-tracing	Circuit-trace hook deletes the source block	outside checklist
L006	minor	circuit-tracing	Edge density lacked a null; repair used a parametric one	P1
L007	minor	circuit-tracing	'22x denser' headline contradicts own numbers (41x)	outside checklist
L010	major	circuit-tracing	No intervention sanity check or positive control (circuit tracing)	P6
L011	critical	exhaustive-mapping	Exhaustive-mapping hooks delete a whole block	outside checklist
L012	critical	exhaustive-mapping	Triplet hooks overwrite each other (ABC = C)	outside checklist
L014	major	exhaustive-mapping	Synergy test has no statistic or null; spec design cut	P1
L015	critical	exhaustive-mapping	Steering results are block-deletion artefacts	outside checklist
L016	major	exhaustive-mapping	Steering has no random-feature null at any layer	P1
L017	minor	exhaustive-mapping	Exhaustive summary numbers that do not match files	outside checklist
L019	major	exhaustive-mapping	No intervention sanity check or positive control (exhaustive mapping)	P6
L020	major	sae-atlas	0/48 TF-specificity test null-saturated, no chance baseline	P1
L021	major	sae-atlas	TFs with no responding feature counted as negatives	P6
L022	major	sae-atlas	Phase 8t selection: layer mismatch, ~5 control cells, token-level test	P4
L023	major	sae-atlas	0/48 result cannot be reproduced (SAE overwritten, IDs not saved)	P10
L024	minor	sae-atlas	48 TFs chosen alphabetically	P5
L031	major	sae-atlas	Phase 6 applies the L5 SAE to hidden_states[6]	outside checklist
L033	major	attention-grn	Curveball null never shuffles (z = 0 by construction)	P1
L034	minor	attention-grn	Curveball used 50 draws; spec asks 200	outside checklist
L035	minor	attention-grn	Verdict thresholds looser than the spec	outside checklist
L036	major	attention-grn	No gene-level baseline on the TRRUST endpoint	P2
L037	major	attention-grn	'Residualised attention collapses to chance in every run'	outside checklist
L038	minor	attention-grn	'No run beats any trivial baseline' (RPE1 does)	outside checklist
L039	minor	attention-grn	Summary p-values are not the BH values	outside checklist
L040	minor	attention-grn	1B run not like-for-like and mislabelled	P7
L041	minor	attention-grn	RPE1/Adamson reports carry K562 scope text	outside checklist
L042	minor	attention-grn	Adamson screen labelled CRISPRa	outside checklist
L043	minor	attention-grn	'Gain <= +0.002 in every run' omits 1B +0.046	P5
L046	major	spectral-geometry	Feature-shuffle null cannot show compression is learned	P1
L047	minor	spectral-geometry	CKA at L0 = 1 by construction; compared with scGPT's L0	P3
L048	minor	spectral-geometry	'Three disjoint cell samples' overlap	P4
L049	minor	spectral-geometry	Phase 8 JSON verdict contradicts the summary	outside checklist
L051	minor	spectral-geometry	L11 effective rank measured after the final norm	outside checklist
L052	minor	spectral-geometry	Spectral autoloop: timeout passed validation; stop rule counts crashes	outside checklist
L053	major	topology-141	scGPT cross-model CCA has no null (0.40 is at chance)	P1
L054	minor	topology-141	Procrustes retrieval fitted and scored on the same genes	P5
L055	minor	topology-141	scGPT 'alignment breaks' despite retrieval z 19-31	P9
L056	minor	topology-141	Geneformer called autoregressive / same family	P7

ID	Severity	Pipeline	Short description	Pattern
L057	major	topology-141	iter_04 relabelled 'NOVEL POSITIVE' by manual review	P9
L059	minor	topology-141	H123 cross-validation leaked shared endpoints	P4
L061	minor	topology-141	H123 negative had no positive control; repair control is one easy setting	P6
L062	minor	topology-141	Lung H123 'anti-predicts' rests on one cell sample	P8
L064	minor	topology-141	H139 null uses 8 permutations (spec 100-1,000)	P1
L065	minor	topology-141	Topology 'autoloop' is a scripted batch; decisions changed by hand, unlogged	P10
L066	major	manifold-discovery	Fifth pre-registered gate (permutation p) never computed	P1
L067	major	manifold-discovery	Shuffled null passes two of the four gates	P1
L069	major	manifold-discovery	External 'branch-holdout' holds nothing out	P4
L070	minor	manifold-discovery	Lung negative control fails all four gates, reported as three	P6
L072	major	manifold-discovery	Zero-shot panel is not a separate cohort	P4
L074	major	manifold-discovery	Internal branch-holdout 0.370 carried by three small branches	P8
L075	major	manifold-discovery	3-gate fallback rule added after sweep 1	P5
L076	minor	manifold-discovery	H103 zero-shot uses k=5 on 18 anchors	P8
L077	minor	manifold-discovery	Two diverging manifold summaries; stale copy hides a failed test	P10
L078	minor	manifold-discovery	'Compression costs no measurable quality' untested	P5
L082	minor	manifold-discovery	Manifold run used an unrecorded second Python environment	P10
L083	minor	longevity-mechinterp	Longevity negative has no positive control	P6
L084	minor	longevity-mechinterp	Longevity summary: 'L5 used by every other pipeline'	outside checklist
L086	minor	framework (specs)	Deployed specs break the ten-section template	P10
L087	minor	framework (specs)	Source-model pre-flight in spec Validation never run	P6
L088	minor	cross-pipeline	No checkpoint revision pinned; code pins unverifiable	P10
L089	minor	cross-pipeline	Environment and intermediates not recorded	P10
L118	major	attention-grn	Three attention runs feed log1p data as if it were counts	outside checklist
L119	critical	topology-141 (phase4 cross-model)	scGPT cross-model comparison used the wrong embedding rows	outside checklist
L120	minor	attention-grn	Residualisation fitted in float32, not a least-squares fit	outside checklist
L121	major	attention-grn	Beating the degree-preserving null is not model-specific (co-expression also beats it)	P2
L122	major	manifold-discovery	Manifold trustworthiness depends on the number of anchors scored	P1
L123	major	sae-atlas	Several SAE-atlas scripts apply an SAE to the wrong layer	outside checklist

12 Exploratory arm with Claude Haiku 4.5

This arm repeats Study A with Claude Haiku 4.5 as executor, reviewer and repairer. Tasks, packages, answer keys, review and repair instructions, budget sentences, five runs per cell, and the Opus 5.5 graders are the same as in Study A. It was added after the first Study A task (T2) had shown that every Opus 5.5 deliverable reached the correct verdict (Amendment 2). Each agent worked in its own folder of links to the task files, and the prompts said that files could be written only inside that folder (Amendment 3). The arm is exploratory and does not enter the pre-registered contrasts.

Isolation. After the run we scanned the tool calls of all 188 subject agents (40 executors, 80 reviewers, 68 repairers). The shared task folders were unchanged afterwards. Many agents still wrote temporary files to the shared /tmp folder (51 of 188). We therefore checked, for every read of a /tmp file, which agent had last written it before that moment: in all 112 such events it was the reading agent itself. Five reviewers opened the folder of the executor whose deliverable they were reviewing, because the deliverable named it. No subject agent read the answer keys or the project repository.

Missing deliverables. Twelve reviews (7 checklist, 5 generic) did not return a valid structured

output after five attempts; their repaired deliverables do not exist and are counted as missing. Graders agreed on the verdict of all 108 graded deliverables.

Correct verdicts (deliverables reaching the key's verdict / deliverables graded):

Task	Paper, no review	Paper, generic	Paper, checklist	Contract, no review	Contract, generic	Contract, checklist
T1	0/5	0/5	1/5	3/5	3/4	3/4
T2	4/5	4/5	3/3	5/5	4/4	4/5
T3	2/5	3/4	1/4	3/5	4/5	2/3
T4	4/5	4/5	4/5	4/5	2/3	3/4
All	10/20	11/19	9/17	15/20	13/16	12/16

Share of the key's pitfalls handled (mean over deliverables; yes = 1, partly = 0.5, no = 0):

Task	Paper, no review	Paper, generic	Paper, checklist	Contract, no review	Contract, generic	Contract, checklist
T1	0.10	0.19	0.22	0.32	0.44	0.45
T2	0.33	0.28	0.46	0.45	0.42	0.57
T3	0.20	0.30	0.26	0.27	0.24	0.28
T4	0.58	0.60	0.58	0.40	0.44	0.54
All	0.30	0.35	0.38	0.36	0.37	0.48

For comparison, Opus 5.5 executors handled 0.96 (paper arm) and 0.93 (contract arm) of the pitfalls without review.

Contrasts (the same tests as the pre-registered Study A contrasts; Holm adjustment across the four):

Contrast	Result
Contract vs paper, no review	15/20 vs 10/20; difference +0.25 (Newcombe 95% interval -0.05 to +0.49); Fisher $p = 0.19$; Mantel-Haenszel $p = 0.15$; Holm $p = 0.76$
Checklist vs generic (paired, 28 runs with both)	18 vs 19 correct; 2 runs right only with the checklist, 3 right only with the generic review; exact McNemar $p = 1.0$
Checklist vs none (paired, 33 runs)	21 vs 21 correct; 3 and 3 discordant; $p = 1.0$
Generic vs none (paired, 35 runs)	24 vs 22 correct; 2 and 0 discordant; $p = 0.50$

Secondary outcomes (paired differences in the share of pitfalls handled, with 95% bootstrap intervals over executor runs pooled over tasks, and in brackets the pre-registered version that resamples runs within each task): checklist vs none +0.092 (0.049 to 0.136) [0.050 to 0.133]; checklist vs generic +0.064 (0.013 to 0.117) [0.021 to 0.104]; generic vs none +0.017 (-0.022 to +0.053) [-0.017 to +0.049]; contract vs paper without review +0.059 (-0.048 to +0.164) [-0.010 to +0.130]. Materially false statements per deliverable: checklist vs none +0.61 (-0.06 to +1.27); generic vs none -0.07 (-0.71 to +0.57). Key numbers within tolerance: checklist vs none +0.083 (0.008 to 0.174) [0.028 to 0.143].

Repair. After generic reviews, repair turned 2 wrong verdicts right and none wrong (22 stayed right, 11 stayed wrong). After checklist reviews, it turned 3 wrong verdicts right and 3 right verdicts wrong (18 stayed right, 9 stayed wrong).

Review points. Generic reviews raised 4.4 points each (155 in 35 reviews), of which graders with the key judged 50% invalid; checklist reviews raised 8.2 each (269 in 33 reviews), 56% invalid.

Effort. Median tool calls and wall time per agent: executor 16 calls, 8 minutes; generic review 25 calls, 6 minutes; checklist review 19 calls, 5 minutes; repair 10-12 calls, 4-6 minutes.

Files: controlled-evaluation/studyA_haiku2/results/ (workflow output, analysis.json, deliverables.csv, reviews.csv, costs.csv, isolation_check.json, tmp_race_check.json).